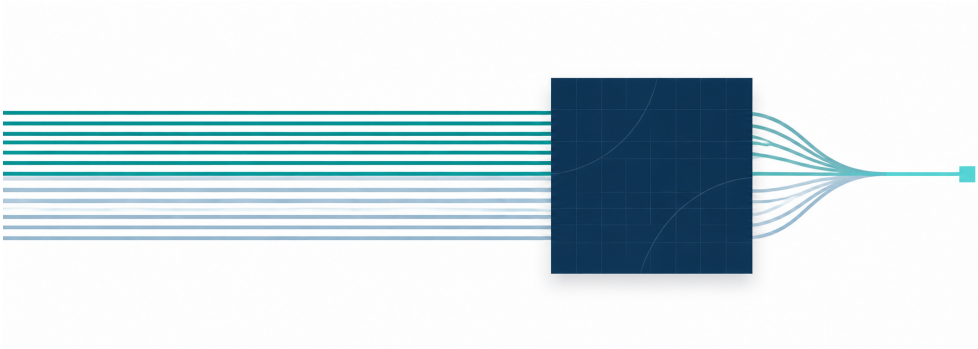
香山昆明湖 V2

自定义向量点积指令

三阶段技术文档合订本

技术报告



AI 生成概念插图；不表示实际芯片版图或实测数据。

工程组合： 0f ＋ firstdot V2 ＋原 C8

团队名称： 待团队确认

成员： 待团队确认

学校： 待团队确认

材料日期： 2026 年 11 月 2 日

图像修订： 2026 年 11 月 5 日

已补齐原 9 处空图位；身份信息与波形补证范围见正文。

目录

第一阶段执行日志与过程文档 1

1.1 实验目的 1

1.2 环境与部署过程 2

1.3 hello 与 vadd 验证结果 3

1.4 波形与执行过程 4

1.5 结论及附件 6

1.6 实际执行日志摘录 6

1.6.1 构建和运行时间记录 7

1.6.2 程序构建 7

1.6.3 差分运行与结果 7

1.7 从原波形提取的事件时间线 8

1.8 日志附件索引 8

1.9 工程实现与验证方法补充 9

1.9.1 部署任务的依赖关系 9

1.9.2 内存、swap 与并发的工程含义 9

1.9.3 固定源码与依赖恢复 10

1.9.4 程序装载、启动与串口输出 10

1.9.5 启动差分故障的定位过程 10

1.9.6 向量加法结果的解释 11

1.9.7 日志附件的阅读和复查顺序 12

1.9.8 环境复现的输入层级与检查顺序 12

1.9.9 启动一致性问题的反事实分析 13

1.9.10 向量用例如何支持结论 13

1.9.11 构建资源与失败证据的管理 14

1.9.12 阶段一成果向后续阶段的传递 14

1.9.13 故障定位时的观察边界 15

1.9.14 可重现结果与不可推广结果 15
1.9.15 对阶段二验证设计的具体启示 16
1.9.16 阶段一验收记录的阅读方式 16

1.10 环境部署详细方法 17
1.10.1 Windows/WSL 原生文件系统与工具分工（工程实践） 17
1.10.2 WSL 内存 18
1.10.3 固定源码 19
1.10.4 已验证工具 19
1.10.5 新机器安装 20
1.10.6 使用现有环境 21

1.11 NEMU 构建原始日志 21

1.12 EMU 构建原始日志 24

1.13 工具与编译链版本原始输出 25

1.14 固定子模块版本清单 26

1.15 原始波形事件记录 29

1.16 向量运算单元事件记录 30

第二阶段设计与基础测试文档 32

2.1 设计目标 32

2.2 指令语义与编码 32

2.3 数据通路和两槽微架构 33

2.4 与 DIV 共用父级的处理 34

2.5 功能验证 35

2.6 设计取舍与扩展 37

2.7 工程实现与验证方法补充 37

2.7.1 从数学定义到硬件位宽 37
2.7.2 译码、微操作和配置依赖 37
2.7.3 P 槽、R 槽与同拍重叠 38
2.7.4 反压期间的身份稳定性 39
2.7.5 取消、复位及环形年龄 40
2.7.6 共享 DIV 资源与时钟使能 41
2.7.7 定向、随机和全核测试的分工 42
2.7.8 设计演进与最终采用 43
2.7.9 指令语义与实现边界的逐层核对 44
2.7.10 算术边界为何需要单独覆盖 44
2.7.11 配置依赖和写回路径的风险 45
2.7.12 两级弹性管线的状态推导 45
2.7.13 父级共享资源与门控检查 46
2.7.14 测试证据如何形成可复核结论 46
2.7.15 优化筛选与交付判据 47
2.7.16 回归矩阵中的组合关系 47
2.7.17 从日志到源码的审查步骤 48
2.7.18 时序与吞吐取舍的解释 48
2.7.19 面向下一阶段的接口冻结 48
2.7.20 回归失败的归属与收口 49
2.7.21 写回身份与功能结果的联合检查 49

2.8 内部 ISA 合同、数学边界和测试向量 50
2.8.1 数学语义与目的位布局 50
2.8.2 合法配置、别名、异常与状态 51
2.8.3 候选机器编码 53
2.8.4 RTL、NEMU 与验证共同遵守的接入要求 54
2.8.5 测试与验收合同 56
2.8.6 RVV 性能基线必须保持的等价性 59
2.8.7 歧义清单与变更流程 59

2.9 两槽弹性状态、父级协议及验证不变量 60
2.9.1 接口与状态 60
2.9.2 每拍转移定义 61

2.9.3 父级资源、派发与除法切换 63
2.9.4 门控、唤醒与写回 64
2.9.5 观察协议与不变量 64
2.9.6 分阶段验收与负控 65

2.10 本轮已提供截图补充 66

第三阶段协同仿真与性能分析文档 71

3.1 验证与评估目标 71

3.2 NEMU 扩展 71

3.3 差分、拒错与恢复 72

3.4 等价 RVV 基线与测量方法 73

3.5 硬件微基准结果 74

3.6 软件调用与 C8 结果 75

3.7 面积、时序与设计成本 77

3.8 优化探索及最终取舍 78

3.9 结论 79

3.10 最终版本与性能证据身份 79

3.11 工程实现与验证方法补充 80

3.11.1 架构参考模型的独立性 80
3.11.2 从全核日志判断完成状态 80
3.11.3 为何要审计计时边界 80
3.11.4 配对样本和加速比的含义 81
3.11.5 普通 C 调用和软件合同 81
3.11.6 对象等价和镜像等价的层次 81
3.11.7 面积和时序结果怎样支持取舍 82
3.11.8 优化失败也需要可复核结论 82

3.11.9 同机干净复现和最终工程验收 82
3.11.10 独立参考模型的输入与输出合同 82
3.11.11 差分测试的结果如何判读 83
3.11.12 计数窗口的工程定义 83
3.11.13 配对比较和数据汇总 84
3.11.14 四类工作负载各自回答的问题 84
3.11.15 软件接口、等价验证与状态影响 85
3.11.16 成本分析与后续优化资格 86
3.11.17 复现、限制与交付判断 86
3.11.18 性能数字的实际使用边界 87
3.11.19 从异常样本到结论修订 87

3.12 周期测量边界与统计合同 88
3.12.1 两条路径必须计算同一结果 88
3.12.2 为什么不能直接使用一对 rdcycle 89
3.12.3 工作负载矩阵 90
3.12.4 汇总与结论边界 91

3.13 O1 与 O2 同镜像验证过程 91
3.13.1 本次实测范围 91
3.13.2 硬件与工具身份 92
3.13.3 交付复核 92

3.14 0f 硬件性能与五档成本详细记录 93
3.14.1 受测身份与公平条件 93
3.14.2 同镜像周期 93
3.14.3 同约束综合与面积 94
3.14.4 STA 及敏感性 95
3.14.5 结论与限制 96
3.14.6 复查、失败记录与交付 96

3.15 C8 调用 ABI、对象及回退合同 97
3.15.1 入口与选择 97
3.15.2 普通 C ABI 97
3.15.3 构建与资格边界 98

3.15.4 回退 98

3.16 C8 最终链接和镜像等价验证 99

3.16.1 接入内容 99

3.16.2 三方等价结果 99

3.16.3 检查器拒错与输入保护 100

3.16.4 已完成与未新增的结论 101

3.17 R5 组合验收和证据复用 102

3.17.1 clean-02 墙钟问题的处理 102

3.17.2 验收矩阵 102

3.17.3 当前可使用的性能结论 104

3.17.4 已知边界与下一步 105

3.18 本轮已提供截图补充 106

第一阶段执行日志与过程文档

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

赛题为面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计。本文初稿日期为 2026 年 11 月 2 日，阶段一原实验日期为 2026 年 10 月 9 日，最终工程版本按《最终版本确认》绑定。

1.1　实验目的

本阶段建立香山硬件仿真、NEMU 参考模型与裸机程序之间的协同验证环境。以 hello 输出检查程序启动和串口链路，以向量加法检查向量状态、访存、执行、写回及提交过程，为后续自定义点积指令提供基线。

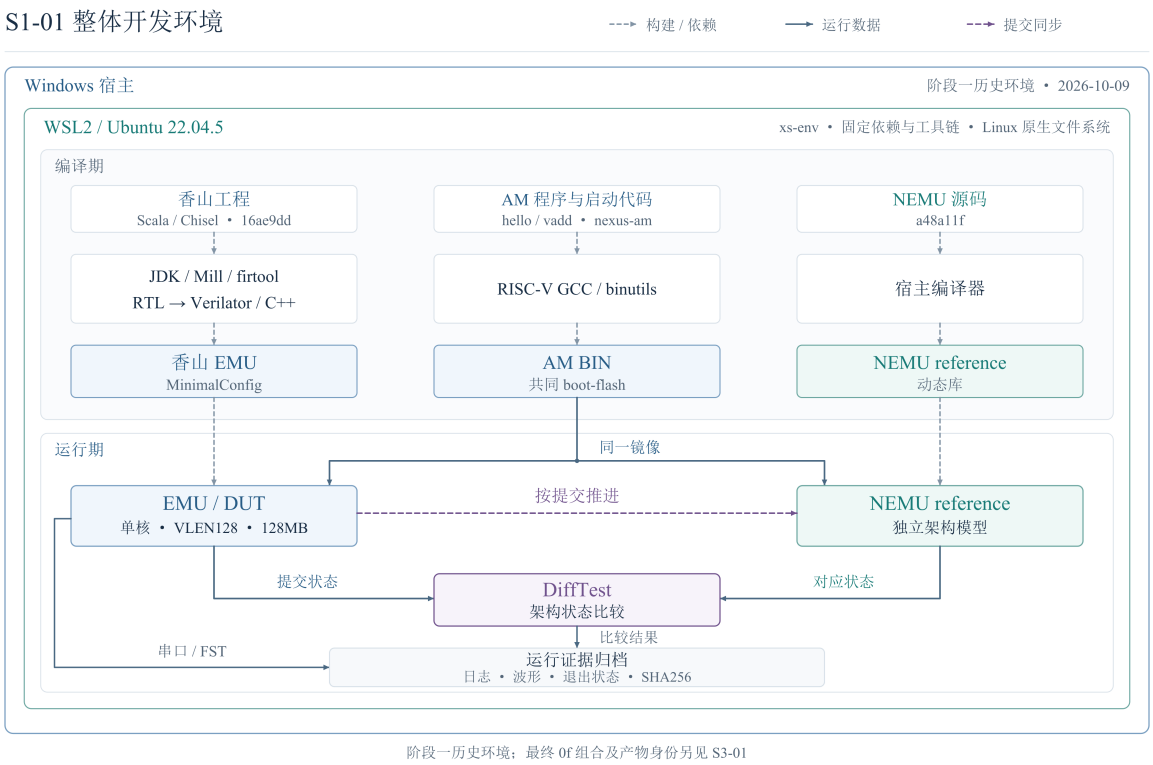


图 1-1　开发环境与软硬件协同验证框架

注：示意图展示 Windows/WSL 环境中 RTL、AM 程序、NEMU 与 DiffTest 之间的构建依赖和运行数据流。阶段一历史环境日期为 2026-10-09。

来源与范围：用户提供示意图；依据：环境记录、team/scripts/stage1.sh 及已固定版本表。版本范围：阶段一历史环境；与最终 0f 组合分别标注。

1.2 环境与部署过程

实验使用 Windows 下的 WSL2 Ubuntu 22.04.5，处理器为 Ryzen 7 7840H。原机器物理内存约 27.7GiB，WSL 配置 20GB 内存及 16GB swap。硬件与仿真工作放在 WSL 原生文件系统。

表 1-1 环境与部署过程

组件	固定版本或设置
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
原香山基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
原 NEMU 基线	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
JDK / Mill / firtool	21.0.12 / 0.12.3 / 1.62.1
Verilator	5.052，固定 ea338be 提交
RISC-V GCC / binutils	11.4.0 / 2.38
DUT	MinimalConfig、单核、VLEN128
原波形构建	EMU_THREADS=1、EMU_TRACE=fst

部署依次完成 WSL 资源调整、固定仓库与递归依赖检出、工具版本核对、NEMU reference 构建、香山仿真器构建、hello/vadd 编译和差分运行。各步保存命令、标准输出、退出码、版本与 SHA。环境配置详情与完整锁文件作为附件交付。

以下是既有脚本的操作顺序，实际操作应在已核对身份的专用输出目录执行。

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID= 正式团队标识 bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```

原始启动曾在 PC 0x10000004 发生寄存器差异。定位发现两端默认 flash 分别含 3 条和 6 条启动指令，随后由同一 boot-flash.S 生成 12 字节镜像，并显式传入共同 flash、128MB RAM 和固定 reference。保存的失败与修复日志可说明启动条件对差分验证的影响。

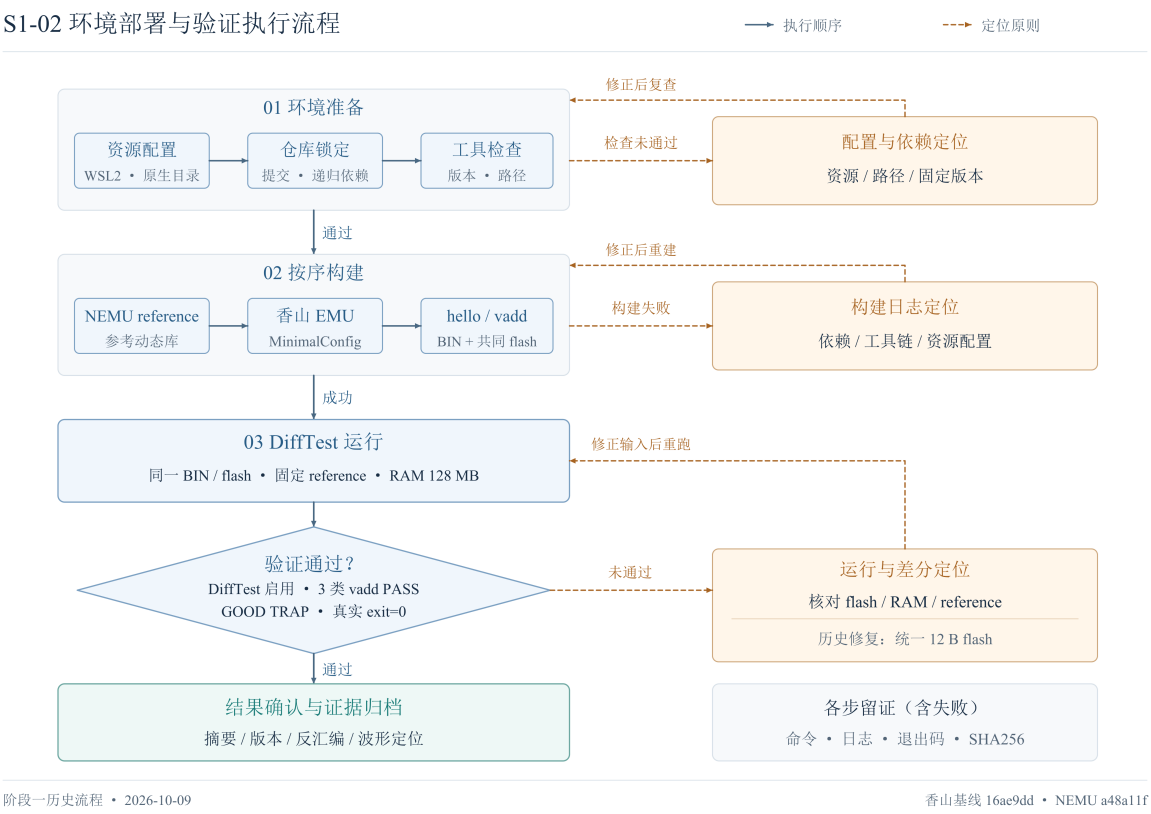


图 1-2　环境部署与验证执行流程

注：流程依次包括环境配置、命令检查、工具核查、NEMU/EMU 与工作负载构建，以及 DiffTest 验证和证据归档；失败时返回相应定位环节。

来源与范围：用户提供示意图；依据：阶段一真实执行顺序、构建和运行日志。版本范围：2026-10-09 阶段一流程。

1.3　hello 与 vadd 验证结果

hello 输出包含团队标识和 WSL IPv4。历史记录使用 riscv-vdot-team，正式录屏应呈现确认后的团队标识及相应真实输出。下文附真实历史输出，正式团队标识由成员确认后补入。

vadd 程序设置 SEW8、LMUL1、VL8，执行两次向量加载、一次 vadd.vv 和一次向量存储，逐字节对比标量参考。

表 1-2　hello 与 vadd 验证结果

用例	输出字节
正数	9 9 9 9 9 9 9 9
负数补码	254 252 250 248 246 244 242 240
边界回绕	128 127 0 0 0 0 254 254

原运行同时具备 HELLO_VADD_PASS cases=3 lanes=24、GOOD TRAP、启用 DiffTest 及 exit0。程序整体统计为 16557 条指令、42962 周期，包含启动、UART 和校验。真实日志摘录见附录。

Historical hello/vadd output; original run 2026-10-09

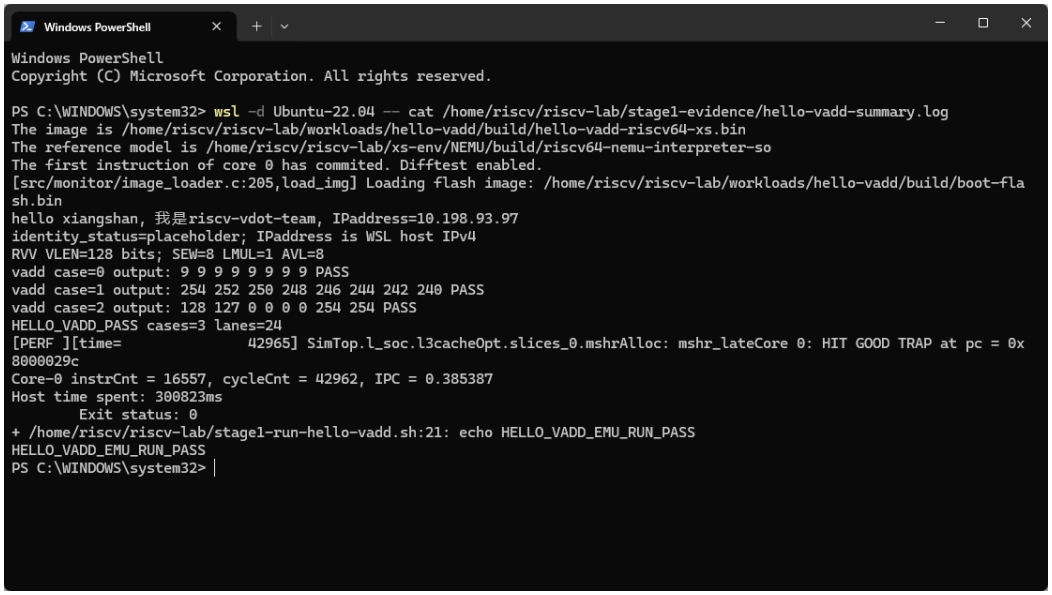


图 1-3 hello 与 vadd 正常结束输出

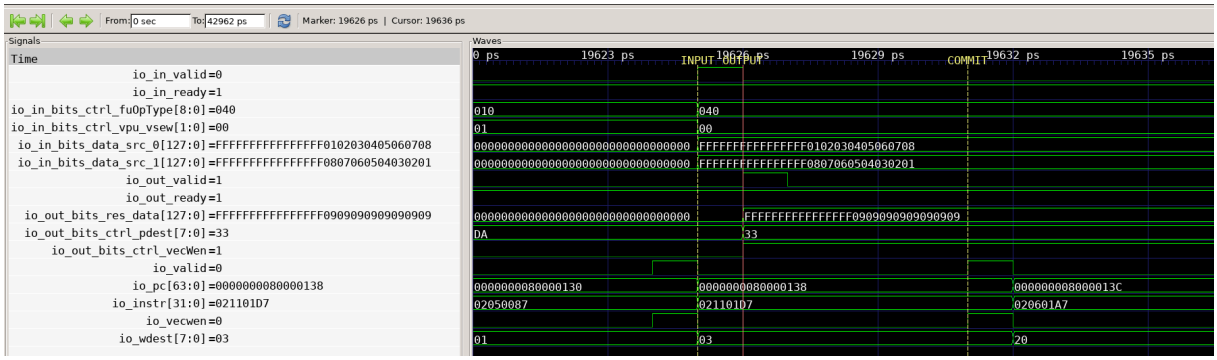
注：已插入用户提供的历史运行日志截图，包含 DiffTest、三类 vadd 输出、GOOD TRAP 及原退出状态。团队名仍为占位标识，正式团队标识输出待补。

来源与范围：用户截图 WindowsTerminal_m5hXeDJnV1.png；原 hello-vadd-summary.log。版本范围：2026-10-09 历史运行回放，不是本次重新运行。

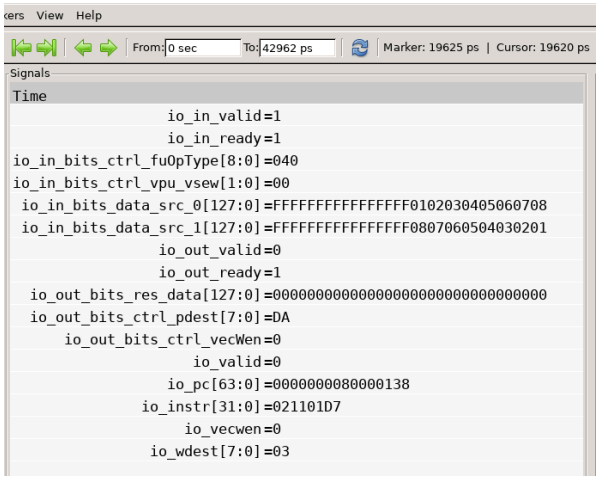
1.4 波形与执行过程

处理器完成取指和译码后，由重命名、调度和操作数准备驱动向量执行。两个 vle8.v 提供输入，vadd.vv 完成按元素加法，结果经写回进入目的向量寄存器并在 ROB 提交。DiffTest 在架构状态层与 NEMU 对应执行结果比较。

(a) Output handshake, marker 19626 ps; all three event positions visible



(b) Input marker 19625 ps



(c) Commit marker 19631 ps

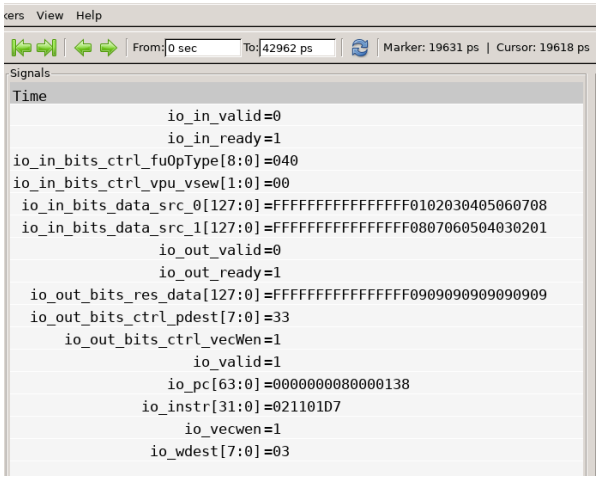


图 1-4　vadd 正数用例的执行与提交波形

注：(a) 输出事件及输入/提交位置；(b) 输入游标处信号值；(c) 提交游标处信号值。截图来自原 FST 提取的 selected VCD，未显示时钟；19625/19626/19631 为文件中事件坐标，截图单位为 ps，不能据此推断芯片物理时钟或实际延迟。

来源与范围：用户上传正数输出截图及 riscv-lab/picture 中的输入、提交截图；保留原始图并仅裁剪外部空白。版本范围：2026-10-09 原基线 vadd；时钟截图仍待补，不是 0f vdot。

原波形提取的 vadd 提交 PC 为 0x80000138，机器码为 0x021101d7，vecwen=1、wdest=3。

表 1-3　波形与执行过程

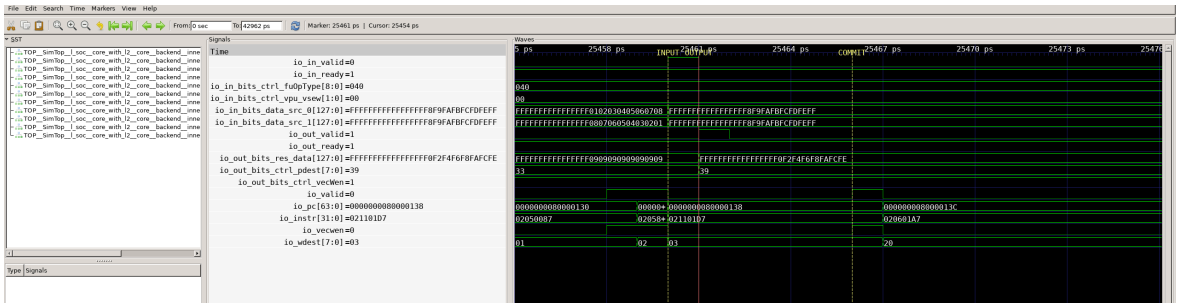
用例	输入事件坐标	输出事件坐标	提交事件坐标	运算输出低 64 位
正数	19625	19626	19631	0x0909090909090909
负数	25460	25461	25466	0xf0f2f4f6f8fafcfe
边界	33009	33010	33015	0xfefe000000007f80

按原波形展开时，应展示时钟、valid/ready、提交 PC 和结果信号，并用游标标注表内事件。具体信号层级以原波形与事件提取脚本为准。表内三列事件数值为采样坐标，结果可说明运算单元输出早于架构提交的流水关系。

完整 FST 位于本机/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，大小 220295598

字节。历史波形按 2026 年 10 月 9 日基线标注。最终 0f 环境的 hello/vadd 资格由 R5 组合验收另行绑定。

(a) Negative case: output marker 25461 ps



(b) Boundary case: output marker 33010 ps

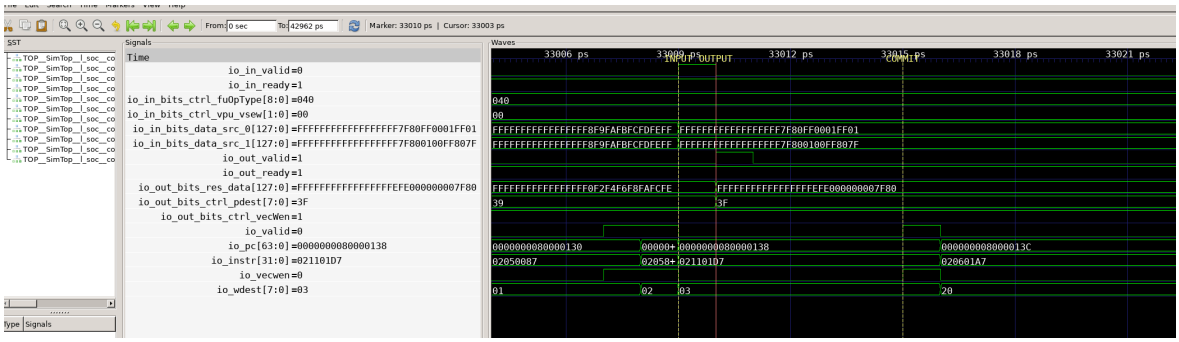


图 1-5　vadd 负数与边界用例结果波形

注：(a) 负数输出低 64 位 F0F2F4F6F8FAFCFE；(b) 边界输出低 64 位 FEFE000000007F80。两图保留各自输入、输出和提交事件位置；未显示时钟，坐标单位按截图为 ps。

来源与范围：用户上传 msrdc_yWBpPeYM8q.png 与 msrdc_6cIzJ9aIVJ.png。版本范围：同一 2026-10-09 历史 vadd 波形；低 64 位按有效 8 个元素解释。

1.5　结论及附件

阶段一已完成部署、共同启动条件、3 类向量加法用例与差分检查。提交报告应附操作日志索引、工具/子模块版本、源文件、反汇编、事件 CSV 及波形定位信息。本稿提供原波形提取表与事件时间线，团队标识待确认。

附件位置为 team/evidence/stage1-20261009，包含 hello-vadd-summary.log、工具版本、子模块锁、boot-flash 反汇编、vadd-commit-events.csv 及 vector-fix-alu-events.csv。运行源码位于 team/workloads/hello-vadd。

阶段一原记录 • 环境记录

1.6　实际执行日志摘录

下列片段直接取自原始归档，日期及路径按原记录保留。完整日志随本文源文件的 evidence 目录提供。

1.6.1 构建和运行时间记录

- 2026 年 10 月 9 日 20:42:15，主动中断并恢复 C++ 构建，将并行度调为 4，已完成对象保留。
- 2026 年 10 月 9 日 20:53:32，EMU 构建结束，状态文件记录 exit_code=0。
- 2026 年 10 月 9 日 20:55:26，workload 与共同 flash 构建结束，记录 WORKLOAD_BUILD_PASS。
- 原正常运行至 21:00:28 结束，退出状态为 0。

1.6.2 程序构建

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:11: date -Is
2026-10-09T20:55:26+08:00
+ /home/riscv/riscv-lab/stage1-build-workload.sh:12: cd /home/riscv/riscv-lab/workloads/hello-vadd
+ /home/riscv/riscv-lab/stage1-build-workload.sh:13: python3 -
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME {/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c -march=rv64gc -mabi=lp64d
boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy -O binary -j .text
build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump -d build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:24: cp build/boot-flash.bin
/home/riscv/riscv-lab/stage1-evidence/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:25: sha256sum build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:26: cp build/hello-vadd-riscv64-xs.txt
/home/riscv/riscv-lab/stage1-evidence/hello-vadd-disassembly.txt
+ /home/riscv/riscv-lab/stage1-build-workload.sh:27: sha256sum build/hello-vadd-riscv64-xs.elf
build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s end=%s\n' 0
2026-10-09T20:55:26+08:00
```

1.6.3 差分运行与结果

```
The image is /home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
```

```
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=              42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc: mshr_lateCore 0: HIT GOOD
TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
      Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

1.7 从原波形提取的事件时间线



图 1-6 vadd 输入、输出与提交事件时间线

注：按 PC 0x80000138、指令 0x021101D7 及目的 v3 精确关联三类 vadd 用例。数字为原 CSV 事件坐标；三列按事件阶段排列，间距不表示时间长度。

来源与范围：evidence/vector-fix-alu-events.csv 及 vadd-commit-events.csv；生成源与哈希见 figures/evidence-20261105。版本范围：2026-10-09 历史 vadd 事件；不能推导物理时钟或新硬件延迟。

图中坐标来自原 FST 提取 CSV，为事件重绘示意。负数和边界用例的对应坐标列于正文表格，完整原波形以 SHA 绑定。

1.8 日志附件索引

evidence 目录保留 NEMU 构建、EMU 构建、workload 构建、原正常运行、原启动失败及并行度调整日志，并包含工具版本、子模块锁、两份事件 CSV 及信号索引。原始日志用于追溯实际操作，本文摘录用于阅读。

1.9 工程实现与验证方法补充

1.9.1 部署任务的依赖关系

环境部署包含宿主运行时、硬件生成工具、RISC-V 程序工具链和参考模型四条依赖链。Java 与 Mill 负责解析 Scala 和 Chisel 工程，firtool 将中间表示转换为可仿真的硬件描述，Verilator 与宿主 C++ 编译器生成 EMU。RISC-V GCC 和 binutils 处理目标程序及启动镜像。NEMU 在参考侧执行同一条架构指令，为差分检查提供状态。

四条依赖最终在一次真实运行中汇合。若只验证各工具能够打印版本，仍无法确认其组合产物相互兼容。因此阶段一把工具检查、实际构建、共同启动镜像、程序执行及差分结果串成完整链路。各层分别保存输入和输出摘要，使后续故障能够定位到对应环节。

表 1-4 部署任务的依赖关系

部署对象	上游依赖	主要产物	核对内容
NEMU 参考库	固定源码、配置、宿主编译器	动态参考库	配置、编译退出码、加载路径、SHA
香山 EMU	固定硬件与子模块、Java/Mill、firtool、Verilator	宿主仿真器	DUT 参数、生成命令、编译器、SHA
裸机程序	nexus-am、klib、RISC-V 工具链	ELF/BIN、反汇编	ISA/ABI、入口地址、段、机器字
共同 flash	启动汇编与 binutils	12 字节启动镜像	两端加载同一实物
差分运行	上述四类产物	原始日志、退出状态、波形	完整通过标记及实际状态比较

1.9.2 内存、swap 与并发的工程含义

香山仿真器构建会同时消耗 Java 堆、硬件生成工具内存、多个 C++ 编译进程及文件缓存。仅提高 Java 堆并不能保证构建更快，反而可能挤压宿主和其他阶段可用内存。本项目在约 27.7GiB 物理内存的个人电脑上，将 WSL 上限配置为 20GB，并设置 16GB swap 作为资源缓冲。

并发度需要结合当前阶段选择。原始记录中将 C++ 构建调整到-j4，并明确保留已完成对象。日常复现脚本采用较低默认并发，以便控制内存压力。两种配置的具体发生时间和真实命令分别记录，报告只据此说明实际操作。运行阶段每个 DUT 仍按单核、单仿真线程进行，宿主同时管理多个任务的数量另按对应合同约束。

swap 发生时，墙钟耗时可能显著增加。构建速度和模拟吞吐用于安排工作，性能表

中的 cycles 由处理器仿真内部计数器给出。对于较长作业，还应保存内存峰值、日志大小、单调时钟限额和进程回收结果，以便判断完成状态。

1.9.3 固定源码与依赖恢复

xs-env 协调硬件、NEMU、nexus-am 和 DRAMsim3 等依赖。项目记录每个组件的完整提交，而不是只记录分支名。相同分支在后续日期可能移动，固定提交则可以明确回答当时使用了哪一份代码。

恢复环境时先读取版本锁，再检查已有目录的提交与修改状态。遇到不一致时应保留现场并使用隔离目录，避免覆盖旧实验。递归子模块的身份同样属于硬件输入，单独更新主仓库并不能保证恢复了同一设计。

新成员需要区别三种版本信息。第一种是香山上游基线，说明开发起点。第二种是包含 vdot 修改的硬件提交，说明最终设计。第三种是后续文档与封存提交，说明交付材料。三者共同构成可追溯记录，后两者之间必须通过源码树和产物摘要连接。

1.9.4 程序装载、启动与串口输出

目标程序以 ELF 承载符号、段信息和调试资料，以 BIN 供加载器装入。反汇编用于核对实际机器字和指令地址。即使 C 源代码未变，链接参数、数据布局或库版本发生变化，也可能改变指令 PC，因此观察波形时应使用与被测 BIN 同次构建的反汇编。

hello 输出先确认程序已经经过启动代码到达 C 入口，并检查 UART 可见结果。随后程序启用并核对向量状态，执行三类 vadd 用例。输入数据、标量期望和完整输出比较均由程序实际执行，输出 PASS 之前已完成逐元素检查。

原 hello 包含 WSL 宿主 IPv4，该地址用于识别当时开发环境。它与仿真处理器中的网络设备能力分别表述。团队标识目前仍使用历史占位符，填写正式队名后应保留新镜像和输出的身份，原历史记录继续作为原实验凭据。

1.9.5 启动差分故障的定位过程

原故障发生在启动阶段 PC 0x10000004，程序尚未进入用户向量运算。检查差分首次不一致的位置后，进一步比较两端启动 flash，发现默认启动指令数量不同。若直接调整测试期望，无法解决两端起始状态和执行序列的根本差异。

最终处理使用一份 boot-flash.S 构建共同镜像，并在启动命令中显式指定该文件。修复之后，两端经过相同启动序列，再加载同一程序，在保持 DiffTest 启用的条件下完成验证。该过程说明最早的差分错误应先从共同输入、复位和装载条件检查。

表 1-5 启动差分故障的定位过程

排查步骤	所查证据	得到的判断
定位首次差分	原失败日志、PC、整数状态	发生在启动代码
比较输入	默认 flash 及反汇编	两端默认序列不一致
统一启动实物	共同 boot-flash.bin	输入条件明确
使用原比较域恢复运行	正常输出、GOOD TRAP、退出状态	hello/vadd 通过

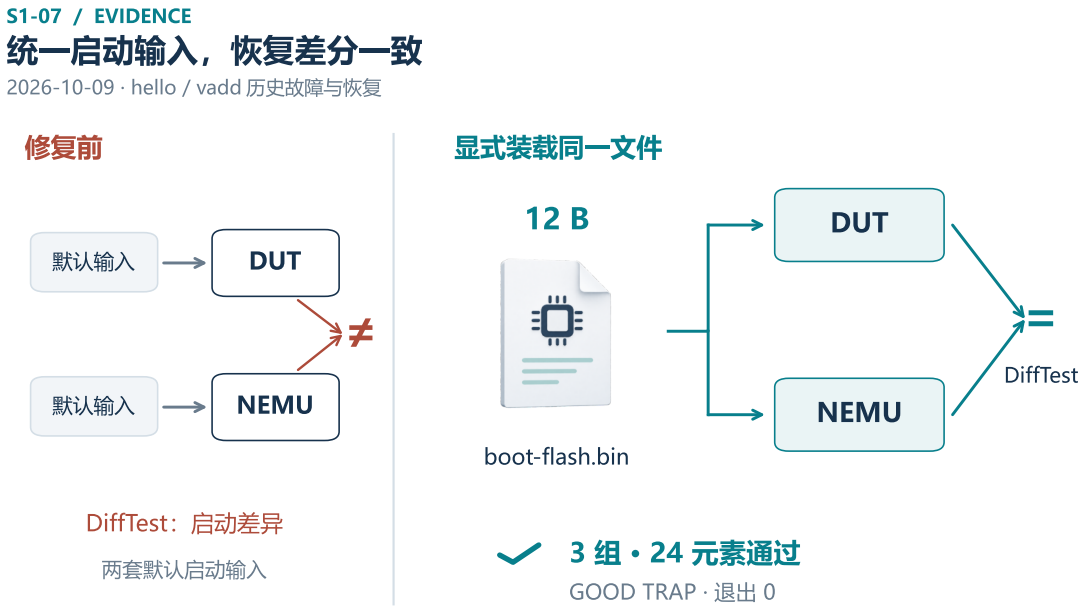


图 1-7 共同启动镜像修复前后的差分定位

注：首次差异在启动 PC 0x10000004、t0，原 instrCnt 为 2；随后两端通过显式 flash 参数使用同一 12 字节文件，三条启动指令及文件偏移见图数据附件。恢复日志记录三类 24 lane 通过、GOOD TRAP 及 exit 0。**来源与范围：**2026-10-09 原失败日志行 164/166、boot-flash-disassembly.txt、完整成功日志行 7/10/16/17/19 及 hello-vadd-summary.log；完整出处和哈希见图数据附件。版本范围：历史启动问题定位；不是 vdot 错误或新运行。共用文件不等于两端物理映射地址相同。

1.9.6 向量加法结果的解释

SEW8 下每个元素保存 8 位加法结果，运算按模 256 保留低 8 位。负数补码的无符号打印值应结合原输入解释。例如十进制 254 对应 8 位补码 −2，打印成 254 不表示数学结果变为正数。边界用例专门包含回绕，以检查程序的参考计算与向量指令行为一致。

结果进入执行单元输出后，仍需经过写回和乱序提交。本项目把输入事件、输出事件和 ROB 提交分别标出。正数用例的输入 19625、输出 19626、提交 19631 说明三个事

件属于不同流水位置。表中单次差值用于解释该次观测，不据此给出所有操作的固定提交延迟。

1.9.7 日志附件的阅读和复查顺序

建议先查看退出状态和通过摘要，确定记录是否完整。再沿日志开头的镜像、reference 与 flash 路径核对输入。随后检查反汇编和事件 CSV，最后在需要时打开完整波形。这样可以避免仅凭局部 PASS 文本就扩大结论，也能减少在大波形中盲目搜索的时间。

本报告将精简日志放在正文，原始构建和运行日志作为附件保存。构建日志中的工具调用与运行日志中的架构输出承担不同用途，不能互相替代。附后的工具版本和子模块锁用于重建依赖身份，完整 FST 按 SHA 定位，便于他人核对真实信号。

1.9.8 环境复现的输入层级与检查顺序

复现阶段一时，首先确认工作树与子模块的源码身份，其次确认工具链身份，最后检查运行输入。这个次序对应由源到产物的因果关系：源码决定应当构建什么，工具决定怎样生成，镜像和启动参数决定实际执行什么。仅保存最终屏幕输出，无法区分输入变化与工具变化；仅保存版本号，也无法证明运行确实加载了固定产物。报告把这些信息分别列出，是为了让后来者能够从差异发生的最早环节向下排查。

同名目录不能代表同一环境。一个已经存在的 xs-env 目录可能包含未提交修改，子模块也可能停在另一提交。复现者应在启动构建前记录主仓库及递归子模块的完整提交，并检查脏文件；若发现与报告锁定值不同，应先留存差异，再在隔离目录恢复。此规则同时保护旧实验，因为直接覆盖原目录会使后来者无法重读当时的失败证据。

工具版本清单也需要分清直接工具与间接依赖。Java、Mill、firtool、Verilator 和宿主编译器处于硬件生成链；RISC-V GCC、汇编器和链接器处于目标程序链；NEMU 的构建还依赖宿主编译工具。某一工具升级后，即使硬件源文件不变，生成 RTL、C++ 仿真器或 ELF 仍可能变化。阶段一并不凭工具版本推断语义改变，而是在复验时将版本、命令、产物摘要和运行结果一起核对。

当构建失败时，错误发生的位置比最后一行提示更重要。Java 依赖解析失败意味着尚未进入硬件生成；firtool 失败意味着无法获得可供 Verilator 使用的目标 RTL；C++ 编译失败则需要区分缺头文件、资源不足和实际源错误。每次修复后只重新执行受影响的下游步骤，同时保留失败日志与原命令。这样可以解释为什么最终日志出现成功，并防止把一次不同配置的成功结果误接到失败尝试后面。

运行所需的镜像还存在不同层级。ELF 提供符号、段布局与反汇编线索，BIN 是加载器使用的内容，flash 是启动入口的独立输入。研究一次具体提交事件时，应先用 BIN 和 ELF 确认机器码，再用实际启动命令确认两侧加载了哪些文件。文件名可以被复用，SHA 和文件长度才可以约束具体实物；入口地址和装载位置则约束这些字节进入处理器的方式。

阶段一的验收采用递进关系。环境检查通过只说明依赖齐备，构建通过说明产物已生成，hello 成功说明程序和串口链路可运行，vadd 逐元素成功说明这组向量输入得到了预期结果，DiffTest 正常结束才为该运行增加参考模型一致性证据。各级结论不能倒置。例如看到 GOOD TRAP 但没有找到输出比较或真实退出状态，仍不足以断言向量功能已被检查；看到某一工具版本，也不能推导完整 EMU 已经重建成功。

1.9.9 启动一致性问题的反事实分析

启动处首次差分位于 PC 0x10000004，这一位置早于用户测试代码。因此排查优先级从指令算法转向复位状态、flash 内容与装载配置。复核时同时核对该地址在两侧实际读取的机器字，并用对应镜像的反汇编确认指令；地址本身不足以证明两次运行使用了相同内容。若在此处修改 vadd 预期值，错误仍会在进入 vadd 以前发生；若关闭 DiffTest，程序可能继续运行，却失去原本用于识别启动差异的门禁。保留原失败、机器字及镜像身份，使修复原因可以被独立检查。

比较两侧默认启动内容后，项目选择共同 boot-flash.S 生成一份明确实物，再将该实物传入运行。此时“共同”有两个含义：字节身份相同，且加载参数指向相同实物。单纯在源码中写出相同汇编，仍可能因工具或构建时点不同得到不同二进制；只传相同路径，仍可能因目录映射或文件被覆盖而不一致。因此日志中同时记录生成命令、输入摘要和运行参数。启动脚本或镜像更新后，应重新核对地址到机器字的映射，不能沿用旧反汇编。

修复后的正常运行构成因果链中的最后一环。两侧先经过共同 flash，再到达同一程序入口，随后观察 hello、向量用例与差分终态。失败和成功是两个独立运行，报告没有把前者的局部现象说成后者已经通过，也没有把后者覆盖前者。未来若更换启动镜像或参考库，应重新核对首次差分位置与终态，而不能沿用本次的成功标签。

1.9.10 向量用例如何支持结论

三个 vadd 用例承担不同角色。正数输入便于检查基本取数、计算和写回；负数输入检查补码解释以及打印形式与算术含义的区别；边界输入检查定宽结果的回绕。用例组合的目的不是穷举全部向量 ISA，而是在自定义指令开发之前证明共同仿真链路可以观察到向量运算的关键状态，并能对典型错误给出可辨结果。

参考计算应与硬件结果采用相同的元素宽度和截断规则。SEW8 的加法输出只保留低 8 位，一个数学和超过 255 时会按模 256 回绕；负数以补码存储，若按无符号十进制打印，就会得到大于 127 的数。报告保留输入、期望和实际输出，而不只列 PASS，是为了让读者能够在不用重新运行仿真的条件下核对这一解释是否成立。

波形事件表提供的是一次实际执行的定位点，而不是固定微架构时延合同。输入、输出和提交落在不同拍，说明指令在执行单元内完成后，还需要经过父级选择、写回及 ROB 退休。由于乱序状态、下游 ready 和周边指令可以改变这些间隔，不能从一次正数

用例的三个事件坐标推导所有 vadd 的固定延迟。报告同时给出 PC 和事件类型，方便将波形与反汇编、运行日志连接。

波形截图和 CSV 的证据能力也不同。截图适合展示某几个关键信号在局部时间范围内的相对位置；CSV 可用于逐事件复算和脚本检查；完整 FST 保留更广的信号与时间线。若截图缺少时钟，仍可以说明它来自原波形数据，但不能据坐标差推算芯片物理时间或频率。阶段一明确保留待补时钟图位，避免用一个没有时钟的局部截图替代时序证据。

1.9.11 构建资源与失败证据的管理

硬件构建耗时较长，且 C++ 编译能够产生大量并行任务。出现资源压力时，先记录实际内存限制、swap 配置和并行度，再观察进程退出码与构建日志。若只提高并行度，可能使多个编译进程同时争抢内存；若只增加 swap，可能使作业从失败变为极慢完成。报告中的资源配置是该主机上的实际条件，不推广为每台电脑都应采用的固定值。

Windows 工作树与 WSL 原生目录的文件访问特征不同，仓库规模又较大。Git 索引、硬件生成和大规模 C++ 编译应各自选择有证据支持的放置位置，不能因为某一次超时就给所有操作规定一个加速比例。若跨系统复制源码，必须重新核对源码快照和必要文件摘要，并隔离可写构建目录。这样可以保持历史实验可读，也能让新的构建身份独立于旧缓存。

失败日志不是应当清除的噪声。它们解释为什么后续命令改变了参数、并发或启动镜像，也帮助识别一个看似成功的结果是否来自另一轮运行。每项日志至少应有任务、时间、命令、退出码和所用输入的对应关系。对后续阶段来说，保留失败位置尤其重要，因为新 vdot 的差分失败可能来自指令语义，也可能再次来自启动或参考模型装载；阶段一的排查记录给出了可重复的分层方法。

1.9.12 阶段一成果向后续阶段的传递

阶段一交付的核心不是某个固定波形画面，而是已验证的运行入口和输入身份。阶段二可在该入口上增加 vdot 译码、算术与父级控制检查；阶段三可以继续使用共同镜像和差分终态检查，同时增加自定义指令参考模型。每次增加新能力都要说明哪些旧证据仍适用，哪些输入和源码变化要求重测。这样的传递规则避免将早期 hello/vadd 通过误写成新 vdot 或新性能候选已经通过。

对后来复核者，最短路径是先打开阶段一版本表与最终运行摘要，核对镜像、flash、reference 和 EMU 的身份，再查看三组向量输入及期望，最后针对疑点深入日志与波形。若摘要与原始日志不一致，应以原始运行事实和退出状态为准，暂停扩大结论。这个阅读顺序使报告既能快速说明已完成的基础验证，也能支持对关键环节的独立复查。

1.9.13 故障定位时的观察边界

在仿真系统中，“没有看到预期输出”可以对应许多不同情况：构建没有完成、程序没有加载、启动未到达 C 入口、UART 尚未刷出、参考模型提前拒绝，或者真正的算术结果错误。报告因此把观察边界分为构建完成、启动到达、指令执行、架构比较和进程结束五个环节。每个环节都有可核对的证据，不能只用一个终端窗口的最后几行去解释全部过程。

构建完成的边界由工具退出码和产物检查给出。进程结束的边界由 EMU 返回值、GOOD TRAP 和差分总结共同给出。两者之间仍有大量可能的状态，因此启动日志中的 PC、flash 路径、装载地址和镜像摘要不能省略。若程序实际加载的 ELF 与反汇编来自不同构建，研究者可能把正确的事件错误地归到另一条指令；若终端输出被截断，也可能把未完成任务误当成正常通过。

观察点还要区分架构状态与微架构状态。UART 文本是软件可见输出，向量寄存器和 CSR 是架构状态，ready、valid、ROB 标签和内部时钟是实现状态。阶段一的 vadd 结果主要支持架构功能判断，波形中的握手与提交点帮助解释发生过程，却不能单独代替完整架构比较。到了 vdot 阶段，必须继续把这两类状态分别验证，尤其不能因内部 result 出现正确数值，就跳过完整写回与提交检查。

即使两个观测源给出相同的时间数字，仍需确认单位和事件坐标。FST/VCD 中的时间刻度、仿真器周期数和主机墙钟属于不同量。日志打印顺序也可能受缓冲影响，不能直接当成硬件边沿顺序。阶段一将事件 CSV 绑定到原波形和实际运行，说明某个信号在给定时间坐标发生变化；关于性能或时序的结论，则要等后续阶段建立专门的计数窗口与综合合同。

1.9.14 可重现结果与不可推广结果

本轮环境验证在固定主机、固定工具和固定输入下完成。复现成功支持这些条件下的可执行性，但不会自动证明另一操作系统、另一子模块提交或另一工艺库也能得到相同输出。报告列出本机内存、WSL 配置和路径，是为了复核实际条件；新机器可以采用等效配置，却必须重新核对产物和运行结果。

hello 和三组 vadd 说明基本链路已经闭合，但测试范围依旧有限。它们没有覆盖完整 RVV 指令集，也没有提供 vdot 自定义编码的通过证据。特别是 vadd 的数学和写回宽度与后续 vdot 不同，不能把阶段一的负数或边界用例视为 vdot 极值测试。将结论限制在实际执行的程序和观察域内，是后续阶段继续扩展测试的前提。

对于启动故障，统一 boot flash 证明本次失败原因得以排除，但并不表示启动流程永远不会再次变化。若更新 EMU、NEMU 或加载脚本，仍要检验两侧是否加载同一启动实物。若将来重新编译工作负载，也要使用新镜像对应的反汇编和 PC 表。报告保留这些条件，避免“曾经通过”成为忽略输入变化的理由。

原始证据中的历史日期和主机路径不会为了当前文档外观而改写。材料日期描述报

告整理时点，日志日期描述实验执行时点，二者承担不同身份。后续若团队补充正式截图、署名或报告封面，应将新增内容标为新的材料修订，并保留原始记录与摘要关联。这样既能改善交付可读性，也不会破坏对当时实验的追溯。

1.9.15 对阶段二验证设计的具体启示

阶段一确认了程序能够启动、向量配置能够进入执行路径、差分系统能够观察架构结果。阶段二增加 vdot 时，可沿用同一分层原则设计用例：先用独立软件模型核查数值，再用局部 RTL 检查位宽和握手，随后在真实父级核查取消、反压与时钟，最后在全核检查译码、配置依赖、异常和 128 位写回。每一层都有独立目标，出现失败时能够回退到更小的复现范围。

阶段一的正数、负数和边界三类组织方式可以借鉴，但 vdot 需要新的极值组合。点积有八项乘法与求和，乘积符号和归约位宽比单元素加法复杂；完整向量结果的高位还必须按自定义指令合同处理。一个 vadd 用例可以证明环境能够运行向量程序，却不能替代这些新语义检查。报告在此明确两阶段的证据传递边界，减少重复部署的同时保留新功能的独立验收。

测试入口应允许对失败状态进行负控。阶段一已有启动差异这一真实失败，说明比较机制能在错误起点停止；阶段二还需要对算术位、写回高位、身份和取消行为分别注入或构造错误，证明检查器会拒绝。只有正常通过而没有拒错能力说明，某些不被观察的错误可能仍然混入结果。负控的对象和期望失败位置必须明确，不能把预期失败记作设计失败。

最后，构建与运行成本不应混入指令性能。阶段一的墙钟主要反映源码生成、C++ 编译、模拟执行和宿主资源；后续性能报告使用处理器内部周期与退休计数，并要求同镜像、同工作量配对。即使某台机器编译得更快，也不会改变同一 EMU 中已记录的架构周期。把这两种数字分开陈述，才能让环境改进和硬件优化各自得到正确评价。

1.9.16 阶段一验收记录的阅读方式

阶段一的交付物同时包含环境部署记录、失败定位过程和成功运行日志。审阅时应先确认每份日志对应的源码、脚本与镜像，再确认运行命令及退出状态，最后检查程序自己的通过标记。若只阅读汇总中的“成功”二字，可能遗漏中途替换过的 boot flash 或某次构建使用的旧产物。报告保留失败前后两组证据，是为了让修复因果链可以复算，而不是把失败文件当作无关噪声删除。

对于向量用例，还要区分输入覆盖和观察覆盖。输入覆盖说明测试提供了哪些带符号数、边界值和寄存器初态；观察覆盖说明实际比较了哪些向量元素、异常字段和终止状态。输入看起来复杂并不保证所有结果都被断言，单条通过日志也不保证其余向量状态没有变化。每个用例应能从初态推导预期，并在日志或检查器中找到对应的比较点。若缺少观察点，只能报告“执行过”，不能报告“验证过”。

复现实验时，最容易被忽略的是运行输入和编译输入之间的版本断层。源码更新后若复用旧 EMU，现象可能与新代码无关；镜像更新后若沿用旧反汇编，PC 定位也会发生偏差。阶段一把可执行文件、启动镜像、程序镜像和反汇编视为一组关联产物，并记录生成顺序。新复现者可以使用不同的绝对路径，但需要在同一轮运行中重新确认这组产物的关联性。

阶段一最终能够支持的结论是：在已记录环境与固定输入下，启动差异得到解释和修正，基本程序及选定向量用例完成验证，后续自定义指令开发具备可重复的运行入口。这个结论不包含新点积指令的语义正确性，也不包含后续父级门控与性能指标。明确边界让阶段二能以现成环境为基础，继续添加真正针对点积的模型、协议和全核测试。

1.10 环境部署详细方法

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/docs/environment.md

1.10.1 Windows/WSL 原生文件系统与工具分工（工程实践）

此处是工程执行的默认选择，不改变任务包、验证合同或资源预算，也不把原生工具选择本身视为功能、性能或 T17 验收。

表 1-6 Windows/WSL 原生文件系统与工具分工（工程实践）

源码 / 工作树和任务	默认执行位置	注意事项
Windows NTFS（如 D:\...）工作树上的 Git status/diff、完整索引检查、commit/push	**Windows 原生 Git**，直接使用 Windows 路径	避免经 /mnt/d 跨文件系统遍历大型 Git 工作树；发布仍遵守 Issue 短集成窗口、远端核对和快进规则
Linux/WSL 原生目录（如 /home/...）中的 Git	**Linux 原生 Git**	不因“统一用 Windows Git”而倒置跨文件系统访问
Chisel/Scala、硬件生成、RISC-V 工具链编译、EMU/NEMU、波形 / 综合时序等重型任务	**Linux/WSL 原生文件系统**，使用 Linux 工具	以当前工作包的固定工具和真实硬件 / 软件输入为准；受总体 CPU、内存和其他 Agent 的资源保护约束

表 1-6 Windows/WSL 原生文件系统与工具分工（工程实践）（续）

源码 / 工作树和任务	默认执行位置	注意事项
Windows 工作树与 Linux 运行目录之间的源码、二进制及证据交换	独立检出或固定快照，并在目标环境核验	记录完整 HEAD、未提交差异 / 源码快照、外部依赖和关键文件 SHA-256；不可直接把另一个工作树的 PASS 当作本次 PASS

执行前先确认实际仓库根、工作树位置、分支、完整 HEAD、Git common-dir、未提交改动和正在使用的输入 / 输出；选择**与工作树所在文件系统一致**的 Git。需要 Linux 构建时在原生目录准备独立源码与输出，核对身份后执行已有合同；不要仅为省一次复制就反复扫描 Windows 挂载目录。回写报告、准备提交时由目标 Windows 工作树使用原生 Git 检查实际落盘字节与索引，必要时按实验合同核对 Linux 运行快照。多个 Agent 不共用可写工作树或 Git index，既有运行和冻结证据保持受保护。

来源与限制： 2026-10-24 R4 调用入口发布前，一次从 WSL 跨到 Windows 工作树的最终检查在约 300 秒超时，确认未提交、无残留进程和索引锁后，使用 Windows 原生 Git 完成剩余检查与提交（原始发布检查、Issue 记录）。两次执行并非严格的同命令 A/B 基准，不能据此宣称任意 Git 操作有固定加速比例。若某任务实测另一种放置更合理，可保留证据后调整；正确性与可复现性优先。

1.10.2 WSL 内存

在 Windows 的%UserProfile%\.wslconfig 中设置：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行 wsl –shutdown，重新进入 Ubuntu，用 free -h 核对。原机器 swap 文件放在 D:\WSL\wsl-swap.vhdx，新成员无需照搬该路径。较小内存的机器相应降低 JVM 堆和构建并行度，不能同时把 JVM 堆和构建进程占满可用内存。

1.10.3 固定源码

表 1-7 固定源码

组件	已验证提交
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
香山硬件基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
DRAMsim3	1d2a9bf6da8e050d975ef2669c42faa93943f489
DiffTest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
yunsuan	cadd3c2f43096e253d61296b33ac697be8354e29

在团队仓库运行 bash team/scripts/prepare-dependencies.sh：固定检出 xs-env，仅初始化配套 NEMU、nexus-am、DRAMsim3 及其递归子模块，然后锁定 NEMU 额外依赖。团队仓库自己的香山子模块通过 git clone --recurse-submodules 获取。脚本遇到已有依赖版本不同或已跟踪文件修改会停止，不丢弃队友修改。

NEMU 额外依赖的实际提交见锁定脚本。SimPoint 是 NEMU 已记录的子模块，通过递归初始化固定；其他额外下载仓库不能依赖当天默认分支，构建前必须执行锁定脚本。

1.10.4 已验证工具

表 1-8 已验证工具

工具	版本
Oracle GraalVM JDK / javac	21.0.12
LLVM / Clang	19.1.7
Mill 项目版本	0.12.3
Scala / Chisel	2.13.15 / 6.6.0，由项目解析
firtool	1.62.1，由本次项目依赖解析
Verilator	5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c
RISC-V GCC	11.4.0，riscv64-linux-gnu- 前缀
GNU binutils	2.38
宿主 GCC / G++	11.4.0

表 1-8 已验证工具（续）

工具	版本
Make / CMake / Ninja	4.3 / 3.22.1 / 1.10.1
Python / GTKWave	3.10.12 / 3.3.104

完整工具输出见 tool-versions.txt。Ubuntu 软件包版本清单提供历史来源信息，不建议无差别恢复整台主机的软件包。

1.10.5 新机器安装

先完成配套源码检出，然后加载路径。在团队仓库根目录执行：

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

以上安装入口来自固定 xs-env 的 setup-tools.sh。软件源提供的补丁版本可能变化，因此安装后必须运行团队环境检查。

JDK 原安装脚本使用 Oracle 21/latest URL；它会随时间变化，不能据此保证重装仍是 21.0.12。本队提供固定 GraalVM 安装脚本，使用已核对可下载的官方 21.0.12 Linux x64 归档和官方 SHA-256，安装到用户目录，不需要 sudo：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

新终端中继续导出相同的 JAVA_HOME；已安装原本机 /opt/graalvm-jdk-21 的成员可使用默认路径，无需重装。脚本遇到已有 JDK 版本不同会停止，不能覆盖现有安装。新版本需要单独验证后更新团队记录。安装方法参考 Oracle GraalVM 安装说明；固定归档为 21.0.12 Linux x64，校验来源为 官方 SHA-256。

Verilator 在用户目录源码编译，避免官方脚本默认 -j$(nproc) 导致个人电脑资源不足。先安装依赖，再使用已安装的 Clang：

```
sudo apt-get install -y help2man perl autoconf flex bison libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git "$HOME/riscv-tools/verilator"
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++ LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

回到团队仓库根目录：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
```

此检查核对配套源码提交、香山递归子模块与关键工具版本；香山 HEAD 可以包含本队新增文档提交，但必须保留硬件基线祖先。检查通过不等于新硬件改动已通过仿真。不要手动升级全局 Scala；项目 Mill 会读取仓库.mill-version，启动器来源版本不等于项目版本。

1.10.6 使用现有环境

现有主机只需从 /home/riscv/riscv-lab/xs-env/XiangShan 运行脚本。activate.sh 会设置当前团队仓库为 NOOP_HOME，按 TEAM_XS_ENV_ROOT 指定 NEMU 和 AM，设置 LINUX_GNU_TOOLCHAIN=1。AM 默认不会自动启用向量 ISA，因此构建 workload 仍需显式设置 MARCH=rv64gcv。

1.11 NEMU 构建原始日志

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:9: date -Is
2026-10-09T20:24:27+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:10: cd /home/riscv/riscv-lab/xs-env/NEMU
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:11: git rev-parse HEAD
a48a11f208ff81ad765591d83271d1b339fb3861
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:12: make riscv64-xs-ref_defconfig
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/nanopb'...
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint'...
Submodule 'resource/nanopb' (https://github.com/nanopb/nanopb) registered for path 'resource/nanopb'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint/resource/nanopb'...
Submodule path 'resource/nanopb': checked out '671672b4d7994a9b07a307ae654885c7202ae886'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/gcpt_restore'...
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:20: Warning: .config does not exists!
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:21: To build the project, first run 'make menuconfig'.
+ CC confdata.c
+ CC expr.c
+ CC preprocess.c
+ CC symbol.c
+ CC util.c
+ YACC build/parser.tab.h
+ LEX build/lexer.lex.c
+ CC build/lexer.lex.c
+ CC build/parser.tab.c
+ CC conf.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/kconfig/build/conf
+ CC fixdep.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/fixdep/build/fixdep
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:13: /usr/bin/time -v make -j1
Makefile:40: You have set CONFIG_FLASH_IMG_PATH option, but the specified path """ does not exist!
```

```
Cloning into 'resource/softfloat/repo'...
+ CC src/nemu-main.c
+ CC src/device/device.c
+ CC src/device/alarm.c
+ CC src/device/intr.c
+ CC src/device/flash.c
+ CC src/cpu/tcache.c
+ CC src/cpu/cpu-exec.c
+ CC src/cpu/instr_stat.c
+ CC src/cpu/difftest/ame/amuctrl.c
+ CC src/cpu/difftest/ame/msync.c
+ CC src/cpu/difftest/dut.c
+ CC src/cpu/difftest/ref.c
+ CC src/monitor/ui.c
+ CC src/monitor/watchpoint.c
+ CC src/monitor/monitor.c
+ CC src/monitor/fdt_rng_seed.c
+ CC src/monitor/aligncheck.c
+ CC src/monitor/image_loader.c
+ CC src/utils/state.c
+ CC src/utils/iqueue.c
+ CC src/utils/br_log.c
+ CC src/utils/expr.c
+ CC src/utils/log.c
+ CC src/utils/filename.c
+ CC src/utils/dynamic-config.c
+ CC src/utils/timer.c
+ CC src/memory/host-tlb.c
+ CC src/memory/vaddr.c
+ CC src/memory/paddr.c
+ CC src/isa/riscv64/system/mmu.c
+ CC src/isa/riscv64/system/priv.c
+ CC src/isa/riscv64/system/trigger.c
+ CC src/isa/riscv64/system/intr.c
+ CC src/isa/riscv64/system/timer.c
+ CC src/isa/riscv64/system/aia.c
+ CC src/isa/riscv64/reg.c
+ CC src/isa/riscv64/ame/raw_fp4_memory.c
+ CC src/isa/riscv64/clint.c
+ CC src/isa/riscv64/difftest/dut.c
+ CC src/isa/riscv64/difftest/ref.c
+ CC src/isa/riscv64/init.c
+ CC src/isa/riscv64/logo.c
+ CC src/isa/riscv64/instr/priv/system.c
+ CC src/isa/riscv64/instr/fp.c
+ CC src/isa/riscv64/instr/rvk/aes_common.c
+ CC src/isa/riscv64/instr/rvk/crypto_impl.c
+ CC src/isa/riscv64/instr/ame/mcompute_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_transpose_fast.c
+ CC src/isa/riscv64/instr/ame/mreg.c
+ CC src/isa/riscv64/instr/ame/mldst_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_fast.c
+ CC src/isa/riscv64/instr/decode.c
+ CC src/isa/riscv64/instr/rva/amo.c
+ CC src/isa/riscv64/instr/rvv/vldst_impl.c
+ CC src/isa/riscv64/instr/rvv/vfexp2.c
+ CC src/isa/riscv64/instr/rvv/vcompute_impl.c
+ CC src/isa/riscv64/instr/rvv/vreg_impl.c
+ CC src/isa/riscv64/instr/rvv/vcommon.c
```

```
+ CC src/isa/riscv64/instr/rvcbo/cbo_impl.c
+ CC src/isa/riscv64/instr/mp.c
+ CC src/isa/riscv64/instr/rvb/rvintrin.c
+ CC src/isa/riscv64/instr/rvt/sfu.c
+ CC src/isa/riscv64/instr/rvt/matmul_subfloat.c
+ CC src/isa/riscv64/instr/rvt/element_wise.c
+ CC src/isa/riscv64/instr/rvt/convert.c
+ CC src/isa/riscv64/instr/rvt/conv2d.c
+ CC src/isa/riscv64/instr/rvt/type.c
+ CC src/isa/riscv64/instr/rvt/tensor_init.c
+ CC src/isa/riscv64/instr/rvt/tensor_copy.c
+ CC src/isa/riscv64/instr/rvt/pooling.c
+ CC src/isa/riscv64/instr/rvt/register.c
+ CC src/isa/riscv64/instr/rvt/matmul.c
+ CC src/isa/riscv64/instr/vp.c
+ CC src/engine/interpreter/hostcall.c
+ CC src/engine/interpreter/fp.c
+ CC src/engine/interpreter/init.c
+ CC src/device/io/mmio.c
+ CC src/device/io/port-io.c
+ CC src/device/io/map.c
+ CC src/profiling/profiling_control.c
+ CC src/checkpoint/cpt_env.c
+ CXX src/checkpoint/simpoint.cpp
+ CXX src/checkpoint/path_manager.cpp
+ CXX src/checkpoint/serializer.cpp
+ CXX src/checkpoint/semantic_point.cpp
+ CXX src/base/output.cpp
+ CXX src/iostream3/zfstream.cpp
+ CXX src/cpu/difftest/ame/msync_queue_wrapper.cpp
+ CXX src/cpu/difftest/ame/amu_ctrl_queue_wrapper.cpp
+ CXX src/memory/store_queue_wrapper.cpp
+ CXX src/memory/elfloader.cpp
SPECIALIZE_TYPE=RISCV SOFTFLOAT_OPTS="-DSOFTFLOAT_ROUND_ODD -DINLINE_LEVEL=5
-DSOFTFLOAT_FAST_DIV32TO16 -DSOFTFLOAT_FAST_DIV64TO32 -fPIC" make -s -C
/home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC all
mkdir -p resource/softfloat/build
ln -sf /home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC/softfloat.a
resource/softfloat/build/softfloat.a
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
        Command being timed: "make -j1"
        User time (seconds): 22.97
        System time (seconds): 7.18
        Percent of CPU this job got: 101%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:29.76
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 445324
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 1324
        Minor (reclaiming a frame) page faults: 1488436
        Voluntary context switches: 7740
        Involuntary context switches: 491
        Swaps: 0
        File system inputs: 227800
        File system outputs: 96112
        Socket messages sent: 0
```

```
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:14: cp .config /home/riscv/riscv-lab/stage1-evidence/nemu.config
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:15: file build/riscv64-nemu-interpreter-so
build/riscv64-nemu-interpreter-so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
linked, BuildID[sha1]=4841e67452e47681eecaff72b0e3526c02cec25f, not stripped
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:16: sha256sum build/riscv64-nemu-interpreter-so
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:17: date -Is
2026-10-09T20:25:12+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:18: echo NEMU_BUILD_PASS
NEMU_BUILD_PASS
```

1.12 EMU 构建原始日志

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
+ /home/riscv/riscv-lab/stage1-build-emu.sh:12: date -Is
2026-10-09T20:42:28+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:13: cd /home/riscv/riscv-lab/xs-env/XiangShan
+ /home/riscv/riscv-lab/stage1-build-emu.sh:14: git rev-parse HEAD
16ae9ddcda54fc9a2fddffad73174cf793ac7814
+ /home/riscv/riscv-lab/stage1-build-emu.sh:15: free -h
               total        used        free      shared  buff/cache   available
Mem:            19Gi       623Mi        16Gi       2.0Mi       2.3Gi        18Gi
Swap:           16Gi        73Mi        15Gi
+ /home/riscv/riscv-lab/stage1-build-emu.sh:16: /usr/bin/time -v make emu CONFIG=MinimalConfig
NUM_CORES=1 EMU_THREADS=1 EMU_TRACE=fst JVM_XMX=12G -j4
make -C ./difftest emu SIM_TOP=SimTop DESIGN_DIR=/home/riscv/riscv-lab/xs-env/XiangShan
NUM_CORES=1 RTL_SUFFIX=sv
make[1]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
Fri, 09 Oct 2026 20:42:28 +0800
Building emu...
make[2]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
time -avp -o /home/riscv/riscv-lab/xs-env/XiangShan/build/time.log make -s VM_PARALLEL_BUILDS=1
OPT_SLOW="-O0" \
                        OPT_FAST=-O3 \
                        PGO_CFLAGS= \
                        PGO_LDFLAGS= \
                        -C /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile -f
                        /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile/VSimTop.mk
make[3]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[3]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[2]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
make[1]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
        Command being timed: "make emu CONFIG=MinimalConfig NUM_CORES=1 EMU_THREADS=1
        EMU_TRACE=fst JVM_XMX=12G -j4"
        User time (seconds): 2451.21
        System time (seconds): 187.35
        Percent of CPU this job got: 397%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 11:03.88
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
```

```
        Maximum resident set size (kbytes): 1301648
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 479
        Minor (reclaiming a frame) page faults: 50883606
        Voluntary context switches: 5000
        Involuntary context switches: 4568
        Swaps: 0
        File system inputs: 8832
        File system outputs: 3129248
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-emu.sh:17: ./build/emu --help
+ /home/riscv/riscv-lab/stage1-build-emu.sh:18: sha256sum build/emu
+ /home/riscv/riscv-lab/stage1-build-emu.sh:19: cp build/time.log
/home/riscv/riscv-lab/stage1-evidence/emu-time.log
+ /home/riscv/riscv-lab/stage1-build-emu.sh:20: date -Is
2026-10-09T20:53:32+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:21: echo XIANGSHAN_EMU_BUILD_PASS
XIANGSHAN_EMU_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-emu.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: printf 'exit_code=%s end=%s\n' 0 2026-10-09T20:53:32+08:00
```

1.13 工具与编译链版本原始输出

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
java 21.0.12 2026-07-21 LTS
Java(TM) SE Runtime Environment Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96)
Java HotSpot(TM) 64-Bit Server VM Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96,
mixed mode, sharing)
javac 21.0.12
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Ubuntu clang version 19.1.7 (++20250114103320+cd708029e0b2-1~exp1~20250114103432.75)
Target: x86_64-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/lib/llvm-19/bin
Verilator 5.052 2026-09-05 rev v5.052
LLVM (http://llvm.org/):
  LLVM version 18.0.0git
  Optimized build.
CIRCT firtool-1.62.1
riscv64-linux-gnu-gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
```

```
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU assembler (GNU Binutils for Ubuntu) 2.38
Copyright (C) 2022 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or later.
This program has absolutely no warranty.
This assembler was configured for a target of `riscv64-linux-gnu'.
Mill Build Tool version 0.12.3
Java version: 21.0.12, vendor: Oracle Corporation, runtime: /opt/graalvm-jdk-21
Default locale: en, platform encoding: UTF-8
OS name: "Linux", version: 6.6.87.2-microsoft-standard-WSL2, arch: amd64

[Scala/Chisel dependency versions]
val defaultScalaVersion = "2.13.15"
 "chisel"        -> ivy"org.chipsalliance::chisel:6.6.0",
 "chisel-plugin" -> ivy"org.chipsalliance:::chisel-plugin:6.6.0",
 "chiseltest"    -> ivy"edu.berkeley.cs::chiseltest:6.0.0"
 override def scalaVersion = defaultScalaVersion
 def scalaVersion: T[String] = T(defaultScalaVersion)
   def scalaVersion: T[String] = T(defaultScalaVersion)
   def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
   def scalaVersion: T[String] = T(defaultScalaVersion)
   def scalaVersion: T[String] = T(defaultScalaVersion)
 override def scalaVersion: T[String] = T(defaultScalaVersion)
 override def ivyDeps = super.ivyDeps() ++ Agg(ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}")
 def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
```

1.14 固定子模块版本清单

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
1d2a9bf6da8e050d975ef2669c42faa93943f489 DRAMsim3 (heads/cosim-kmh)
68739c5a23cc48c9a13d28793834e4fee95d33ee DRAMsim3/ext/SuperLU_MT_3.1 (68739c5)
a48a11f208ff81ad765591d83271d1b339fb3861 NEMU (a48a11f)
e51a936d7fddfa03c81692039f184ab6c437e99e NEMU/resource/simpoint/simpoint_repo (heads/main)
bba8f3a4e3a15ef135baed37566be3e8a0c7d384 NutShell (bba8f3a)
29e80f8525735d1cb77f30fd2a3ea5eff3e6528c NutShell/difftest (29e80f8)
16ae9ddcda54fc9a2fddffad73174cf793ac7814 XiangShan (16ae9dd)
82079f4081bbc1f0221c790fbf8a29b1f23f48b8 XiangShan/coupledL2 (82079f4)
8faa17c9f7115782da1bd1665bd95587fba525ec XiangShan/coupledL2/HuanCun (8faa17c)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/coupledL2/HuanCun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/HuanCun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat
(d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture/env
(ce70afb)
```

90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/HuanCun/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/rocket-chip/torture/env (ce70afb)
992ce71166405de47c93102100a897726bd93bac XiangShan/coupledL2/utility (992ce71)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/utility/rocket-chip/torture/env (ce70afb)
62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0 XiangShan/difftest (62e4540)
e1bd4695ca7beb36a5ce7357e9527ad9e95b9ec1 XiangShan/fudian (heads/pipeline)
b64af41c3276f97f0e181920400ee056b9c88037 XiangShan/fudian/berkeley-softfloat-3 (b64af41)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/fudian/berkeley-testfloat-3 (06b2007)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/huancun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/huancun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/huancun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/huancun/rocket-chip (e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/huancun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/huancun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/rocket-chip/torture/env (ce70afb)
614ceb4c5d39d32e6ae786693a35da429ea32c12 XiangShan/openLLC (614ceb4)
d3b04fef92092f9b27023bea57cc2ca99a8ffea0 XiangShan/openLLC/coupledL2 (d3b04fe)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/openLLC/coupledL2/HuanCun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/openLLC/coupledL2/HuanCun/Utility (de50694)

175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip (e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/coupledL2/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/rocket-chip/torture/env (ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/coupledL2/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/utility/rocket-chip/torture/env (ce70afb)
546a621ed82302db1bd1edc900b17ba55682c15c XiangShan/openLLC/openNCB (546a621)
c0955429aa3d9fe2be560f28599ece700fb0466c XiangShan/openLLC/openNCB/external/rocket-chip (c095542)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/openNCB/external/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat (d93aa57)

5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/openNCB/external/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/openNCB/external/rocket-chip/torture/env (ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/rocket-chip/torture/env (ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/utility/rocket-chip/torture/env (ce70afb)
ed8c30e1ec8a9c9aa6b116ce54281235d00273fa XiangShan/ready-to-run (ed8c30e)
bea0af5d855a871699b907c1988961141bbf28e7 XiangShan/rocket-chip (bea0af5)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/rocket-chip/torture/env (ce70afb)
9464aee3c77b021a037f28aa1b3fe53c71b516f6 XiangShan/src/main/resources/aia (heads/main)
a8ca0c6564865ee69b564d431ecbc52074ba8060 XiangShan/utility (a8ca0c6)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/utility/rocket-chip/torture/env (ce70afb)
cadd3c2f43096e253d61296b33ac697be8354e29 XiangShan/yunsuan (cadd3c2)
086a238062fdfb3281b745a537b361983d0154b5 nexus-am (086a238)

1.15 原始波形事件记录

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

cycle,scope,pc,instruction,vecwen,wdest
19562,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0x」
cc0472d7,0,5

19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x
2050087,1,1
19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000134,
0x2058107,1,2
19631,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
19647,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000142,0x
8067,0,0
25442,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0x
cc0472d7,0,5
25458,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x
2050087,1,1
25459,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x
2058107,1,2
25466,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,
0x8067,0,0
32991,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0x
cc0472d7,0,5
33007,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x
2050087,1,1
33008,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x
2058107,1,2
33015,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,
0x8067,0,0

1.16 向量运算单元事件记录

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

cycle,scope,phase,fuOpType,vsew,src0,src1,result
19625,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fff0102030405060708,0xffffffffffffffff0807060504030201,
19626,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
f0909090909090909
25460,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
ffff8f9fafbfcfdfeff,0xffffffffffffffff8f9fafbfcfdfeff,
25461,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
ff0f2f4f6f8fafcfe
33009,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fffff7f80ff0001ff01,0xffffffffffffffff7f800100ff807f,

33010,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffffff」
ffefe000000007f80

第二阶段设计与基础测试文档

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

工程采用 0f 硬件，材料版本 V1.0，初稿日期为 2026 年 11 月 2 日。

2.1　设计目标

点积由多次乘法与累加构成。本设计在昆明湖 V2 已有向量路径中增加 vdot 功能，复用处理器的译码、向量寄存器、调度、写回和精确提交机制。优化集中于独立点积吞吐、结果保存成本及与 DIV 共用父级时的取消行为。

2.2　指令语义与编码

内部合同 vdot-s8x8-r32-v0.1 固定 VLEN128、SEW8、LMUL1、VL8 及 vstart0。两个源各取低 8 个字节，按 signed int8 解释。

$S = \Sigma(i=0\cdots7)\ \text{signed8}(vs1[i]) \times \text{signed8}(vs2[i])$

目的低 32 位保存 S 的二补码，高 96 位清零。源高 64 位不参与计算。结果覆盖旧目的，源与目的允许别名。单项乘积范围为 −16256～16384，总和范围为 −130048～131072，32 位结果能够完整保存。

表 2-1　指令语义与编码

字段	位段	取值
funct6	31～26	000000
vm	25	1
vs2 / vs1	24～20 / 19～15	向量源寄存器编号
funct3	14～12	000
vd	11～7	向量目的寄存器编号
opcode	6～0	custom-0，0x0b

MATCH 为 0x0200000b，MASK 为 0xfe00707f。汇编记法使用 vdot.vv vd, vs2, vs1，程序通过校验后的机器字/汇编宏编码。示例 vdot.vv v10,v8,v9 对应 0x0284850b。

合法状态还要求 vill=0 和 vm=1，接受四种 vta/vma 组合。非法配置通过异常路径处理，成功运算保留合同规定的向量 CSR。对目的宽度、合法域等赛题歧义，按内部合同公开说明，并保留 Q01～Q07 待官方澄清状态。



图 2-1　vdot 指令字段与完整结果布局

注：两个源寄存器均只读取低 8 个字节，按 signed int8 相乘并求和；完整 128 位目的结果的低 32 位为补码结果，高 96 位清零。

来源与范围：用户提供示意图；依据：固定 team/spec/vdot.md 及本文编码表。版本范围：内部 vdot-s8x8-r32-v0.1 合同；保留歧义说明。

2.3　数据通路和两槽微架构

数据通路由 8 路有符号乘法、乘积寄存器、归约加法网络及结果槽构成。乘积为 16 位有符号数，归约在足够宽度下累加。

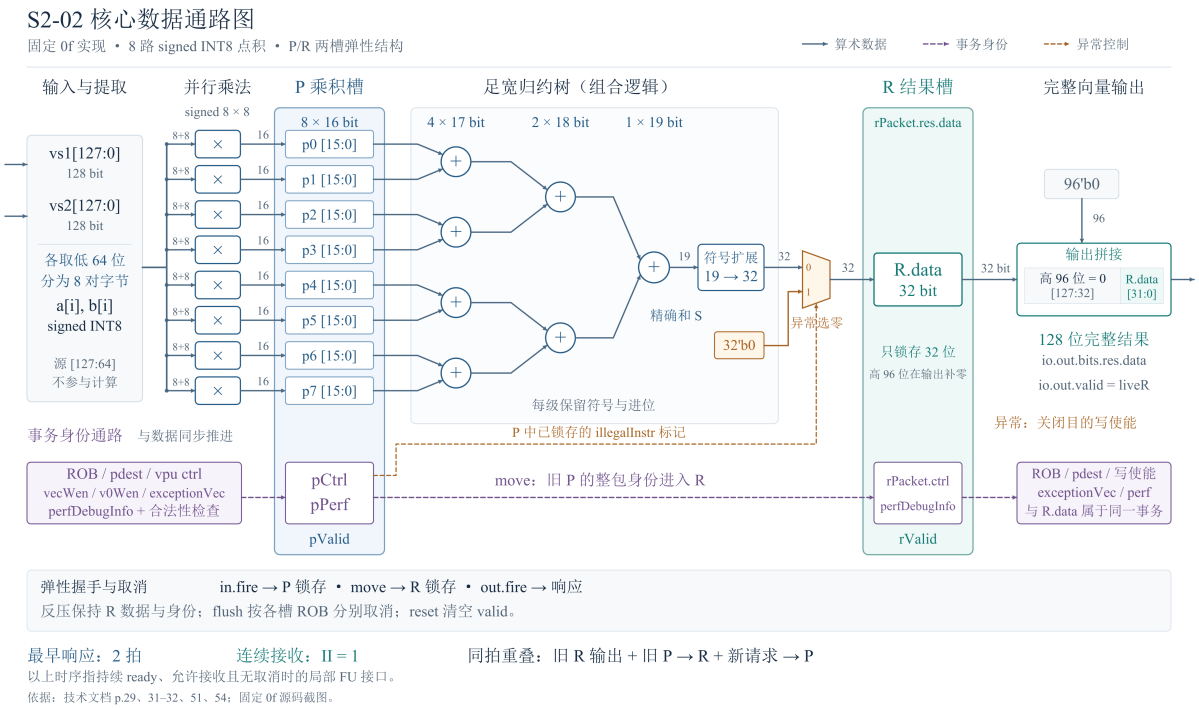


图 2-2 0f 点积算术与 P/R 两槽数据通路

注：8 路乘积经 P 槽、足宽归约及 R 槽输出；ROB、pdest、控制和性能标记随事务传递。图示为稳定 0f 两槽实现，无阻塞时最早 2 拍响应，启动间隔为 1 拍。

来源与范围：用户提供示意图；依据：0f 固定提交的 VDotElastic/VDotO2 及算术模块；旧 TikZ 图可参考。版本范围：最终采用 0f；不展示未采用 P4 一拍候选。

P 槽保存 8 个乘积及对应控制信息，R 槽保存完成结果和完整身份。无阻塞时，旧 R 输出、旧 P 转入 R、新请求进入 P 可在同一拍发生。最早响应为接收后 2 拍，连续接受间隔为 1 拍。

H1 将结果状态保存宽度收窄到 32 位，输出处拼接 96 个零。ROB 标识、目的物理寄存器、向量写使能和性能信息随事务一起推进，反压期间保持待输出数据及身份。flush 分别匹配 P、R 的 ROB 标识，处理部分取消与同时取消。异常请求携带异常信息并关闭目的写使能。

实现路径包括 backend/fu/vector 中的算术模块、wrapper/VDotElastic.scala 及 VDotO2.scala，父级调度和时钟控制位于 backend/exu/ExeUnit.scala。实际浏览以固定 0f 提交的路径为准。

2.4 与 DIV 共用父级的处理

VFEX4 在 vdot 及整数/浮点 DIV 之间管理分派、占用和时钟。0f 修复匹配 DIV 取消之后保持状态未及时释放的问题。验证通过实际父级信号检查取消后的空闲时钟，并覆盖复位、两类 DIV、反压和请求切换。两槽占用信息用于保持正确互斥，控制信号以实际事务身份为依据。

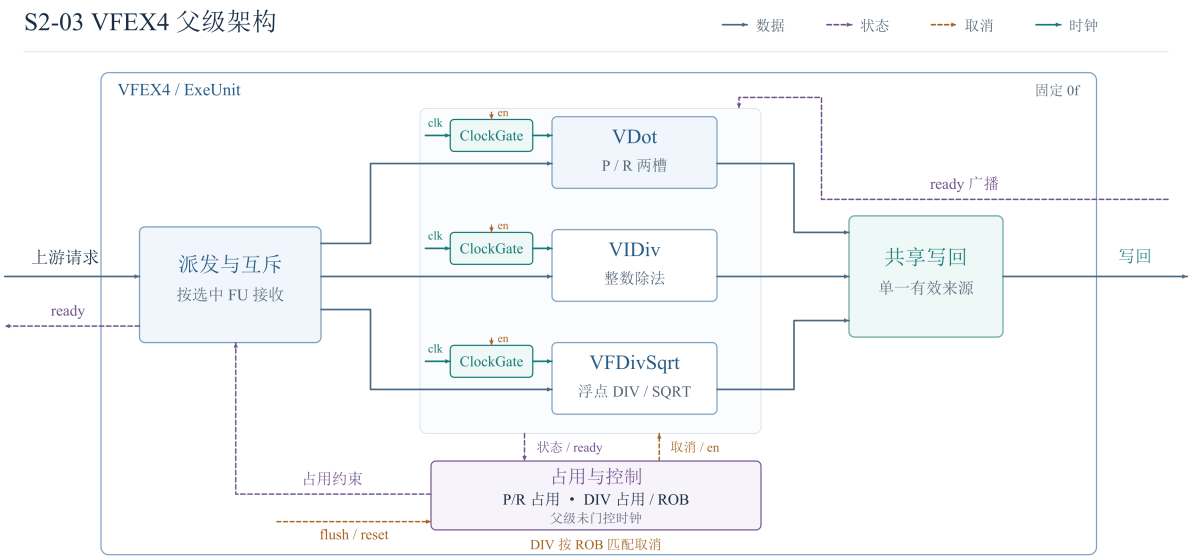


图 2-3 VFEX4 父级的派发、互斥与门控结构

注：VFEX4 共享资源的简化结构示意图，展示 VDot、VIDiv、VfDivSqrt、门控及共享写回之间的关系；控制细节以稳定 0f 实现和正文说明为准。

来源与范围：用户提供示意图；依据：0f ExeUnit.scala、父级验证及 DIV 取消修复证据。版本范围：0f 生产逻辑；不得套用未采用 C1/C2/C3/P4。

2.5 功能验证

测试从独立数学模型推进到算术 RTL、FU、真实父级和全核差分。各层统计分别保存。

表 2-2 功能验证

层级	主要覆盖	已有结果
模型/算术	22 组定向＋ 10000 组随机，完整 128 位	验证通过
FU/真实父级	反压、取消、复位、身份、异常及 DIV 交互	0f 原证据复用，4 ＋ 16 项负控
全核	正负边界、寄存器别名、非法向量状态等	117 个不同用例，按原矩阵分片执行 124 次
总验收	原 13 项与补 1 项组合、负控及恢复	R5/G5 组合验证通过

下面列出便于评审阅读的定向预期值，完整执行证据见原测试归档。

表 2-3 功能验证

输入	数学结果	目的低 32 位
[1,2,3,4,5,6,7,8] 与自身点积	204	0x000000cc
8 个 −1 与 8 个 1 点积	−8	0xfffffff8
8 个 −128 与 8 个 −128 点积	131072	0x00020000
8 个 −128 与 8 个 127 点积	−130048	0xfffe0400
8 个 0 与任意合法输入	0	0x00000000

所有行还需检查高 96 位为零。测试对源高半区和旧目的设置哨兵，以发现错误 lane 选择、目的高位保留和源/目的别名处理问题。

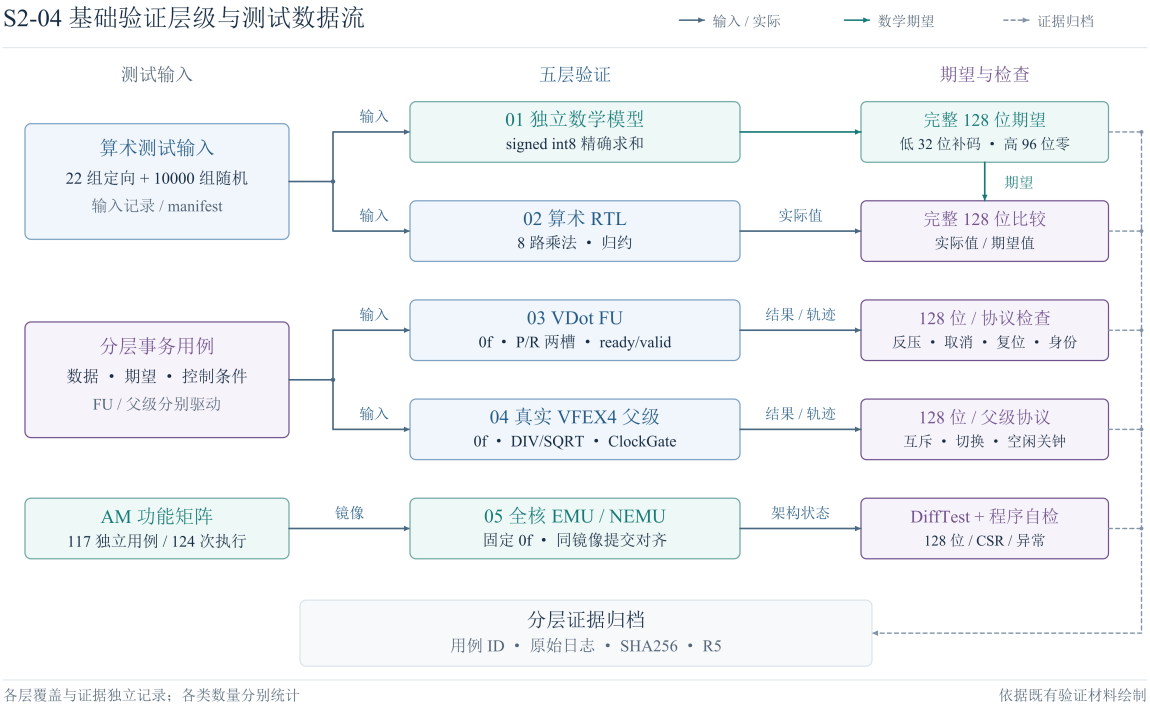


图 2-4 基础验证层级与测试数据流

注：独立参考模型、算术 RTL、FU、真实 VFEX4 父级和全核 DiffTest 分别核验；22 定向加 10000 随机属于算术层，117 独立用例、124 次执行属于所列全核范围。

来源与范围：用户提供示意图；依据：基础测试矩阵、模型数据 manifest、R5 验收矩阵。版本范围：各层覆盖独立记账；Agent 检查不标成人工验收。

2.6 设计取舍与扩展

O2 以两槽容量提高独立吞吐，H1 降低结果寄存器开销，0f 完善 DIV 取消关钟行为。后续一拍 P4 候选经历局部验证和成本筛选，原版 6 项、唯一修订 10 项门槛未过，最终选择 0f。

参数扩展实验另行实现 signed INT16×8、64 位结果，完成 94 组定向和 10000 组随机验证。该证据说明算术与包装的参数化方法，生产实现的 ISA 域按 INT8 合同交付。

指令合同 · 稳定 FU 源码 · R5 验收矩阵 · 参数扩展

2.7 工程实现与验证方法补充

2.7.1 从数学定义到硬件位宽

signed int8 的范围为 −128 至 127。最大正乘积由 −128 乘 −128 得到 16384，最负乘积由 −128 乘 127 得到 −16256。八项累加后的数学范围为 −130048 至 131072。19 位有符号表示覆盖该范围，18 位正上限只有 131071，因此正端点是检验归约位宽的必要案例。

加法树每一级保留进位，宽度依次由 16 增加到 17、18、19 位。最终先形成 32 位补码，再在外部拼接 96 位零。负数结果的高 96 位仍为零，因为项目将低 32 位定义为结果字段，整个 128 位向量按照字段布局写回。完整 128 位检查能够同时发现符号扩展方向错误和高位残留。

源寄存器高 64 位设置非零哨兵，可以检验实现是否误做 16 项点积。两个源的字段提取还需要独立核对，因为点积交换律会掩盖 vs1 与 vs2 交换的问题。数学结果正确、编码字段正确、实际寄存器依赖正确是三项分别检查的条件。

2.7.2 译码、微操作和配置依赖

精确编码匹配后，指令应进入向量类别并生成单个微操作、单次向量写回。编码中寄存器字段采用与 R 型相似的位置，但寄存器类别属于向量体系。若将字段直接按整数寄存器读取，简单数值案例可能偶然通过，架构状态和别名行为仍会错误。

实际 VL 通过重命名后的依赖到达执行端。SEW、LMUL、vill 等来自随指令携带的向量控制。对于乱序处理器，执行时读取一个全局当前 CSR 无法保证对应到本条指令的配置，尤其当多条 vset 与向量运算交错时。测试因此包含密集配置变化、合法非法交错和恢复。

旧目的寄存器不参与点积累计，译码需要避免无用旧目的依赖。目的可能是 v0，也可能与某个源相同，因此源快照和目的写回类型必须明确。完成标记、异常信息和写使能与这一次微操作绑定，防止推测执行在取消后留下架构可见更新。

2.7.3 P 槽、R 槽与同拍重叠

P 槽包含乘积和完整身份，R 槽包含归约结果和完整身份。两个 valid 描述各槽是否拥有有效事务，而不是描述算术结果是否非零。结果为零的合法事务仍必须完成一次响应。

设旧 P 为 A、旧 R 为 B。当下游 ready 且有新请求 C 时，当前边沿可以同时完成 B 输出、A 从 P 转入 R、C 进入 P。非阻塞赋值和实际组合逻辑必须读取旧 P 的内容，使 A 的结果不会被 C 覆盖。验证通过相异数据及相异 ROB/pdest 组合来识别这类覆盖错误。

当 R 被反压时，P 可在空槽条件下接收一个请求，但满容量后必须停止接收。反压解除时，R 响应与 P 推进可以重叠，避免恢复过程引入额外气泡。每次握手、推进和取消均在同一全局周期坐标中复算，不能从日志打印顺序推断真实边沿顺序。

表 2-4 P 槽、R 槽与同拍重叠

旧 P	旧 R	下游 ready	可以发生的主要动作
空	空	任意	接收新请求到 P
有效	空	任意	旧 P 推进到 R，并可接收新请求
空	有效	0	保持 R，可接收新请求到 P
有效	有效	0	保持两槽并反压输入
有效	有效	1	R 输出、P 推进、新输入接收重叠

上表说明正常流的容量关系。reset 与 redirect 具有额外优先级，必须在完整状态转移中覆盖。

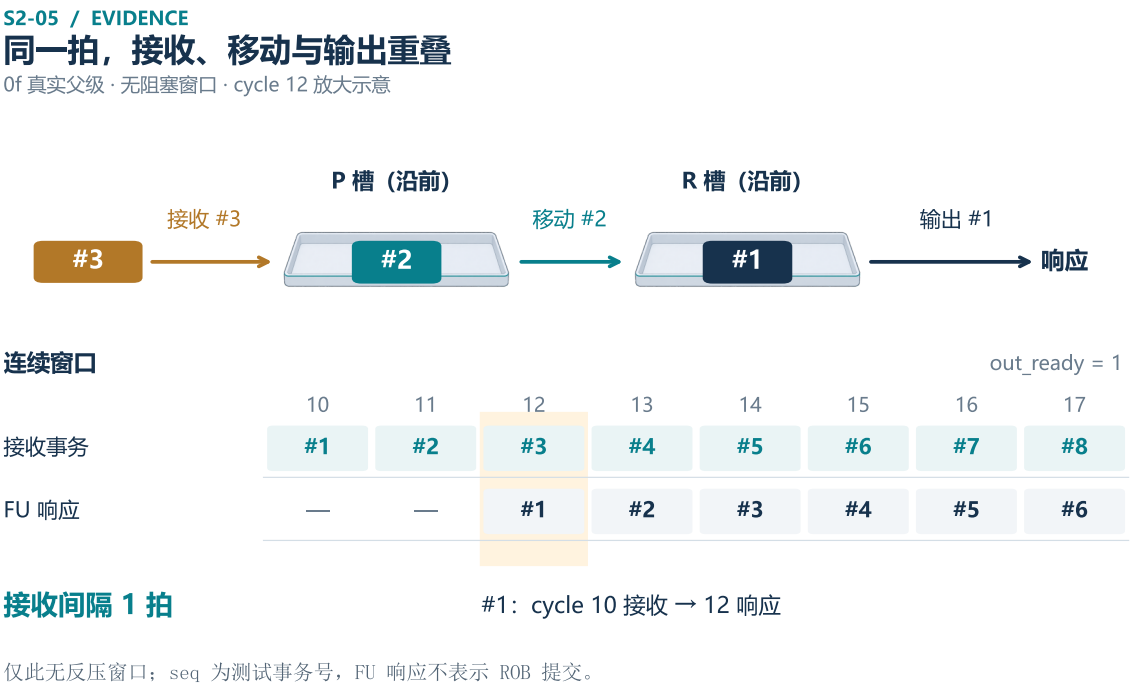


图 2-5　无阻塞连续接收的两槽重叠事件

注：cycle 12 中旧 R 输出、旧 P 转 R 和新请求接收同拍发生；无阻塞时事务 1 在 cycle 10 接收、12 响应，连续接收间隔为 1 拍。seq 为测试记分板事务号，图中槽状态为沿前采样。

来源与范围：0f 真实 VFEX4 父级 green-parent-01/result.log.gz，原行 12–19；case=frozen-vdot-II1。版本范围：历史父级逐拍记录；响应为 FU 握手而非 ROB 提交；未伪造时钟或完整数据总线波形。

2.7.4　反压期间的身份稳定性

io.out.valid 表示 R 中存在可输出事务，其含义独立于 io.out.ready。输出被阻塞时，结果、ROB 标识、目的物理寄存器、异常向量、写使能及性能字段均保持。只检查 data 稳定而忽略控制包，可能产生数值正确但写入错误寄存器的问题。

验证为每个接受事务记录独立身份，响应时与该身份的参考值对应。负控包括交换两事务 pdest、改变 ROB、篡改高位、制造重复响应或漏响应。实际硬件协议测试与检查器日志变异分别记账，前者验证设计，后者验证证据审查能力。



图 2-6 反压期间完整结果与事务身份保持

注：同一事务 seq1321 于 181597 接收，181599–181605 连续 7 拍反压，完整 128 位结果 0x00000000000000000000000000001A87 及 ROB/pdest/控制字段保持；181606 只有一次 FU 输出握手。图中分别保留父级 inready 与 FU vready；181606 后者恢复而前者仍为 0，同拍旧 P 事务 1322 转入 R。

来源与范围：0f fullcore/core-01/control/run.log.gz，原行 15012–15044；raw SHA256 c7351619b11e61171009c1dc997c08dd8a575f7c1508d79f7484df72711c6cb7。版本范围：历史 EMU 2d1db77cc407…，非 clean-02 acf672…；接收到响应 9 拍含 7 拍反压，不是 ROB 提交或新仿真。

2.7.5 取消、复位及环形年龄

redirect 按 RobPtr 的年龄语义分别判断 P 和 R 是否需要取消。ROB 索引的数值在环回时不能直接代表年龄，测试需要覆盖 flag 变化、不同 flushItself 设置和接受顺序与年龄关系不同的情况。较老幸存事务继续完成，较年轻匹配事务释放有效状态。

同一拍发生输出 ready 和匹配取消时，取消决定该事务是否仍可形成响应。reset 则清除槽有效状态，并检查接口握手被抑制。真实父级还需验证已经关钟的空槽可以正确复位，以及取消后重新唤醒的第一笔请求不丢失。

任意前缀都满足事务守恒。接受数等于已响应数、已取消数、复位丢弃数和当前在途数之和。当前在途数最大为 2。这个关系能够发现尾部漏计、幽灵响应和被取消事务再次输出等错误，并要求正常结束时排空。

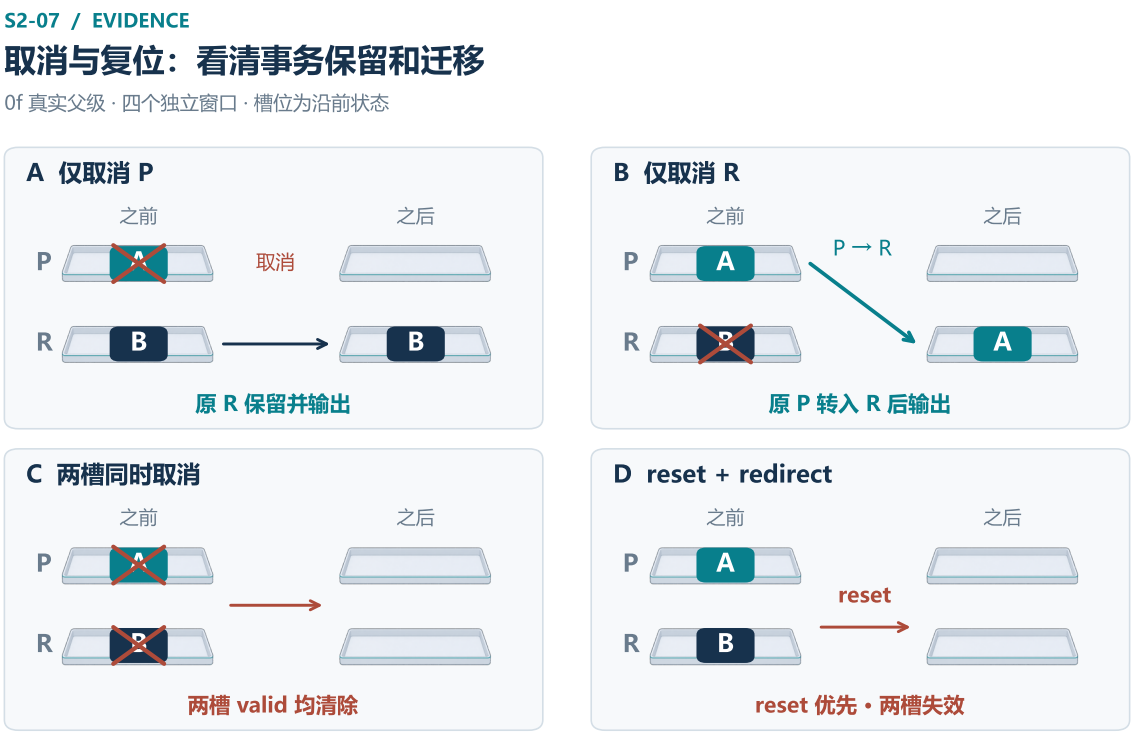


图 2-7 部分取消、全部取消与复位优先级

注：四个独立窗口展示仅 P 取消、仅 R 取消且 P 转 R、两槽取消，以及 reset 与 redirect 同拍时按 reset 唯一记账。前两例包含 ROB flag 环绕；seq 不能代替程序年龄。状态按各窗口沿前记录抽象展示，原 cycle 及事务身份见附件；空槽仅表示 valid 清除。

来源与范围：green-parent-01/result.log.gz；partial-ROB-48/54/12 及 TE0-reset-7；原行与逐行值见 figures/evidence-20261105/figure-data.json。版本范围：仅 0f 真实父级已记录窗口；被取消请求无错误响应，幸存请求按原身份输出。

2.7.6 共享 DIV 资源与时钟使能

VFEX4 父级在 vdot 和 DIV 之间保持资源互斥，输出选择要求有效完成来源唯一。仅在独立 FU 中验证 ready/valid，无法证明父级在取消和切换时正确释放 busy 和时钟保持状态。最终动态父级验证包含 VDot、VIDiv、浮点 DIV 以及真实 ClockGate；SQRT 取消路径完成静态检查，不能将 DIV 动态记录扩展为 SQRT 动态覆盖。图中子时钟位掩码来自上升沿后采样，不是门控使能波形，也不直接代表功耗。

0f 处理的是匹配 DIV 取消后保持状态的释放。若取消后仍保持时钟使能，空闲期间会观察到多余时钟脉冲。若过早释放，又可能使仍在途的事务冻结。因此测试同时核对取消后的空闲采样和未取消事务的完成行为。



图 2-8 DIV 切换与取消后关钟

注：上部展示先排空 VDot 再接收整数/浮点 DIV；下部展示取消清理边沿后子时钟归零，再接收新 VDot。clocks 为上升沿后采样的子时钟位掩码，1/2/4 分别表示 VDot/VIDiv/VFDiv，不是 gate_enable 信号。

来源与范围：green-parent-01/result.log.gz；switch-0-1-0、switch-0-2-0、DIV-cancel-92/110；完整行号见图数据附件。版本范围：动态覆盖 DIV；SQRT 仅静态取消路径检查。事件箭头不表示时间比例；不宣称功耗测量。

2.7.7 定向、随机和全核测试的分工

定向测试用最小数据隔离某一类错误，随机测试扩大数据覆盖并检查确定性再生成，全核测试验证实际译码、依赖、异常和提交路径。完整 128 位数学比较贯穿各层，而配置、取消和写回身份需在对应控制层检查。

22 组定向和 10000 组随机描述算术层输入集合。117 个不同功能/异常例描述架构测试矩阵，124 次执行包含分片重复对照。这些计数保持各自单位，避免把重复执行累计为新的不同用例。

基础测试报告应对每类输入说明初态、配置、指令字、预期结果、实际检查域和通过条件。负数案例核查低 32 位补码与高 96 位零，极值案例核查位宽正端点，别名案例核查源快照与目的分配，非法配置案例核查精确异常和写入抑制。

(a) Historical combinational RTL suite counts (2026-10-12)

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

(b) Positive transaction: expected 204 and actual 128-bit response

```
[POSITIVE] functional-2  seq=1  ROB=0:39  pdest=103
vs1 (128-bit)      0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)      0xa5a5a5a5a5a5a5a50807060504030201
expected sum       204
expected (128-bit)  0x000000000000000000000000000000cc
actual   (128-bit)  0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

(c) Negative transaction: expected -8 and actual 128-bit response

```
[NEGATIVE] functional-0  seq=4  ROB=0:12  pdest=143
vs1 (128-bit)      0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)      0xa5a5a5a5a5a5a5a50101010101010101
expected sum       -8
expected (128-bit)  0x000000000000000000000000fffffff8
actual   (128-bit)  0x000000000000000000000000fffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

(d) Boundary transaction: expected 131072 and actual 128-bit response

```
[BOUNDARY] functional-0  seq=5  ROB=0:43  pdest=66
vs1 (128-bit)      0xa5a5a5a5a5a5a5a580808080808080
vs2 (128-bit)      0xa5a5a5a5a5a5a5a580808080808080
expected sum       131072
expected (128-bit)  0x00000000000000000000000000020000
actual   (128-bit)  0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-9　完整 128 位算术结果证据（异常截图待补）

注：(a) 组合算术 RTL 统计；(b)-(d) 历史全核观察器 ACCEPT/RESPONSE 配对的字段摘录，按 seq、ROB 和 pdest 绑定。actual 来自原响应日志，expected 按低 8 个 signed int8 重新计算；不是 GTKWave 波形或架构提交截图。非法配置异常码和目的不更新的图片仍待补。

来源与范围：用户提供的算术统计截图及正数、负数、边界响应摘录截图；图中保留原日志路径、行号和 SHA。版本范围：算术 RTL 统计为 2026-10-12 历史记录；0f 全核响应为 2026-10-27 UTC 历史记录；原批次 UTC 容量审计 FAIL 保留。

2.7.8　设计演进与最终采用

初版单请求控制提供功能闭环，T14 改变乘积锁存位置，O1 支持更及时的交接，O2 通过两个弹性槽支持连续接受。H1 将结果状态收窄至 32 位，0f 进一步完善 DIV 取消行为。每项演进都对应其独立的验证和比较合同。

后续 P4 尝试将内部算术缩为一拍，局部验证完成后进入同约束成本筛选。原 47 项门槛中，初版失败 6 项、唯一修订失败 10 项，因而最终报告仍采用 0f。设计决策同时考虑协议、功能、资源与时序，单一指标改善不足以替代完整资格。

2.7.9 指令语义与实现边界的逐层核对

第二阶段的设计审查从软件可见的结果开始，逐层追到硬件内部状态。软件合同首先规定指令从两个向量源读取低位的八个有符号字节，逐项相乘并求和，形成低三十二位结果，同时将结果向量其余九十六位清零。这个定义决定了参考模型、定向数据和写回比较的共同口径。指令编码、寄存器字段和向量配置则决定该结果是否属于当前指令。即使算术值相同，若译码选择了另一类功能单元，或输出进入错误的物理目的寄存器，架构行为仍然失败。

评审时需要区分项目约定和外部通用向量扩展的语义。这里的点积是项目实现的定制指令，测试中的字段布局、异常条件和目标寄存器宽度应以本项目已冻结的输入合同为准。不能因为指令名字中含有向量术语，就自动把标准向量指令所有尾部策略、掩码策略或多元素迭代规则套进该实现。相反，也不能用局部算术单元测试的成功替代对项目译码约定的检查。报告在每一层标出该层能够证明的内容，防止一份波形被重复解释为全链路证据。

完整输入合同应包含指令字、两个源寄存器的全部一百二十八位、目的寄存器编号、向量控制快照、进入功能单元时的事务身份，以及可能同时存在的取消信息。期望输出除了结果数据，还包含有效位、目的物理寄存器、ROB 身份、异常字段和写回使能。这样描述允许复核者从测试向量直接推导预期，而不必依赖某一个实现版本的内部信号名。字段无法由输入推导时，应在测试中明确标成由父级供应的控制元数据。

2.7.10 算术边界为何需要单独覆盖

正端点一三一零七二来自八次负一百二十八乘负一百二十八，恰好越过十八位有符号数的上限。若硬件在归约树末级之前只保留十八位，常规随机样本仍可能大量通过，而这一个构造样本会确定性失败。负端点来自八次负一百二十八乘一百二十七，可检查负数中间值和最终补码。零、正一、负一及交错正负项则分别检查零传播、符号处理和加法顺序。这些用例不是为了增加表格数量，而是分别锁定不同的失效机制。

乘积的符号扩展必须在参与加法之前完成。对有符号八位源直接截取无符号字段，再在加法树末端补符号，不能恢复正确的中间结果。各级加法树的位宽应按数学范围增加，并在每一级检查进位和符号位。最终低三十二位的形成方式与整条向量的高位清零是两个独立操作；两者同时检查，可以区分加法位宽错误、错误的整向量符号扩展和旧目的数据残留。

源寄存器的高半区作为哨兵时，应选择能显著改变错误十六项实现结果的非零数据。只把高半区填成零虽然满足合法输入，却无法识别错误的读取长度。同样，交换两个源所得的数学点积不变，所以对源顺序的检查应依赖源读取使能、寄存器依赖和已知非交换的控制情境，而不能只看最终和。测试把可观测的算术行为与必须在译码或重命名处观察的行为分开，避免把交换律造成的盲点当成覆盖。

2.7.11 配置依赖和写回路径的风险

乱序核心中，相邻指令进入执行单元的时间可能与程序次序不同。若点积指令执行时再读取一个可变的全局 VL 寄存器，就可能得到后来配置指令的值。正确的验证对象是随本条微操作传播的配置依赖以及它与源数据的配对关系。测试用合法配置、非法配置、短间隔切换和取消后的重新配置，分别观察译码、调度和执行端是否维护同一份指令上下文。对于没有实现的配置组合，应检查明确拒绝及异常抑制，而不是让它们悄悄落入默认算术路径。

源寄存器与目的寄存器编号相同，是检验重命名和源快照的有效情境。接受请求时，源值应已固定为该请求的操作数；之后目的写回不得反过来改变在途请求所用的源。目的为零号向量寄存器时，也要依照项目合同检查它是合法的目的字段还是特定控制字段，不能套用整数零寄存器的丢弃规则。只有从译码类型、读端口、执行响应到提交映射均保持一致，测试才可称为覆盖完整写回路径。

异常和取消是写回路径的一部分。非法编码、非法向量配置或父级产生的异常需要与原始事务身份同行，并禁止正常结果在取消后写回。单看算术结果端口会漏掉这类问题，所以检查器同时记录发射、接受、响应和提交观察点。每个观察点只对本层可见的字段负责，之后以身份键将这些记录拼接，核对是否存在错配、重复或消失的事务。该方法也方便将未来新增的异常类型接入同一套审查框架。

2.7.12 两级弹性管线的状态推导

P 槽负责保存尚待归约的乘积与身份，R 槽负责保存已归约的结果与身份。两槽各自拥有有效位，数据寄存器里保留旧值并不表示存在有效事务。计算输入 ready 时，需要综合 P 槽是否空闲、P 能否在该边沿前进、R 是否空闲以及 R 能否在该边沿输出。计算输出 valid 时，则直接以旧 R 有效位为准。把 ready 作为 valid 的条件会在反压时隐藏一个真实存在的响应，造成接口协议错误。

一个周期内可以同时发生三个动作，但动作操作的是三个不同时间层的对象。旧 R 可能响应，旧 P 可能进入新 R，新输入可能进入新 P。审查组合逻辑时，应画出边沿前的状态和边沿后的状态，而不是按源码赋值顺序想象三个顺序执行的步骤。特别是新输入覆盖 P 寄存器的同一边沿，R 所接收的必须仍是旧 P 的乘积。用不同的结果与不同的目的寄存器交错输入，可以直接暴露把新 P 误送进 R 的错误。

容量为二的结论需要纳入取消和复位的额外状态转移。若 R 反压而 P 空闲，允许填充 P；若两槽都满，则输入必须停止。如果 redirect 只取消 R 而保留 P，下一边沿可以由幸存 P 填补 R；若只取消 P，R 可继续等待正常响应。reset 清空两个有效位后，旧寄存器中的比特无须立即清零，但它们不得再次获得有效身份。状态表和守恒关系共同检查这些情况，比只观察几条理想流量波形更可靠。

ROB 环形指针具有索引和环回标志。数值较小的索引在环回之后可能代表较新的事务，因此年龄判断必须使用父级定义的比较语义。定向取消样本要跨过环形边界，并

分别覆盖 flushItself 启用和关闭。测试还需让 P 与 R 的 ROB 年龄和槽位先后关系不完全一致，以防代码误把槽位置当作年龄。对每种情况，先列出应幸存的身份集合，再检查输出序列是否恰好等于该集合。

2.7.13 父级共享资源与门控检查

点积加入 VFEX4 后，父级同时承载除法等已有操作，因而必须检查不同来源的响应选择和 busy 释放。若点积自己的两槽逻辑正确，父级仍可能在 DIV 取消后留下旧 busy，阻断新的点积请求；也可能提前放开共享资源，使两个来源在同一输出端口争用。父级验证使用交替操作、同时反压和匹配取消来观察这些问题，且明确记录输出选路所采用的事务身份。

真实时钟门控单元还引入唤醒边界。门控使能往往根据当前有效状态和新请求组合得出；取消、复位、测试使能与时钟相位的组合可能产生独立仿真里没有的差异。对门控的检查不能只比较最终数值，还要观察空闲时是否出现不需要的时钟活动，以及唤醒的第一笔请求是否完整进入 P 槽。对取消 DIV 的修订也应在同一个父级环境中验证，避免只在点积专用平台上得到局部成功。

仓库中的 O1 与 O2 默认开关保持关闭，避免常规构建无意切换实验实现；本报告选定的 0f 稳定构建则按 clean-02 冻结记录显式启用 O2/T14 并关闭 O1。两种配置不能共用同一性能数字。候选比较必须保存开关、源码与构建摘要，在相同输入和约束下复验算术、取消、门控及父级路径；未通过完整门槛的结果仍标为实验结果。由此可以分别追溯默认构建、正式 0f 构建和其他候选的资格范围。

2.7.14 测试证据如何形成可复核结论

算术定向用例提供对边界和特殊值的解释，随机用例提供更宽的输入分布，父级定向用例提供协议和共享资源证据，全核用例提供架构路径证据。每层使用同一数学参考值，但观察对象不同。随机种子、用例数量、构建标识和检查器版本需要随日志保留；否则一个“全部通过”的汇总行无法说明实际跑过哪一批输入。重新运行时还应核对生成输入与原始日志记录一致。

负控的作用是证明检查器会报错。修改期望结果高位、交换两个事务身份或删除一条响应时，检查器应针对相应字段失败。负控本身并不证明硬件存在这些错误，也不能计入硬件通过数。它限定的是证据系统的灵敏度：如果某种故障注入后仍然显示通过，原本的正向测试即使全部绿色，也缺乏针对该故障的检出能力。报告把两类运行分别标识，便于审阅者正确使用数字。

全核验证要包含真正的译码、配置依赖、读端口、执行单元、写回和提交。只在测试平台直接驱动 VDot 请求不能覆盖编码匹配错误；只读取最终软件寄存器又可能难以定位响应身份问题。因此定向平台与软件用例相互补充。对于尚未取得完整输入合同的分支，应记录缺少哪些构建物、指令字或配置，避免把无法复现解释成实现已经通过。

阶段结论建立在可重新执行的输入和现有日志上，而非口头推断。

2.7.15 优化筛选与交付判据

性能筛选要求候选版本在相同约束、相同工作负载和相同观察窗口下比较。两级弹性管线改善的是连续接受与反压恢复能力，不直接保证单条指令延迟、面积或最高频率也改善。H1 收窄状态位宽主要影响寄存器规模；门控修订主要解决空闲切换和取消后保持。把不同目标的优化混成一个总速度数字，会掩盖功能代价和测量边界，因此报告分别保留吞吐、时序、面积和正确性证据。

P4 的失败说明局部算术变快不足以完成集成资格。它在原有门槛中引入更多失败项，应按失败类型追溯组合路径、协议状态和父级共享资源，而不能只挑选局部速度较好的样本替换基线。最终采用 0f 是基于当前完整筛选结果的工程选择；未来若修订 P4，需要作为新的版本重新执行同一验证合同。这样既保留可优化的空间，也防止未通过门槛的候选无意进入默认交付。

第二阶段交付的核心不是若干通过次数，而是一份可追溯的设计闭环：数学定义对应位宽，指令合同对应译码和配置依赖，事务身份对应弹性管线及取消，父级资源对应门控和共享输出，软件可见结果对应写回及提交。后续阶段可以沿此闭环建立独立模型和性能实验；若其中任一环的输入、观察点或版本发生变化，应先更新合同，再解释新旧结果的可比性。

2.7.16 回归矩阵中的组合关系

测试项数量应服务于失效机理，而不是追求一个孤立的总数。算术极值与随机输入解决数值正确性，配置交错解决依赖绑定，别名解决源快照与目的分配，反压解决有效位和载荷稳定，取消解决年龄与复位优先级，父级混合操作解决共享资源。一个测试可以覆盖多个维度，但记录应指出其主断言及额外观察点。这样，当某一项失败时，能够快速找到最相关的输入与波形；当实现变化时，也能判断哪些测试必须重跑。

为了避免组合爆炸，可先对每一维构造最小可区分情境，再挑选交叉点。比如正常单事务可检查基础算术和响应格式；双事务且不同目的寄存器可检查连续接受；双事务加反压可检查载荷稳定；环回 ROB 加选择性取消可检查年龄语义；取消后首笔 DIV 和点积交替可检查父级释放。并不是所有输入值都要与所有控制状态笛卡尔相乘，但每个高风险交叉都应能在矩阵中定位到明确的用例和预期。

随机测试也需要确定的覆盖边界。种子和生成规则决定能否重放，输入约束决定是否覆盖负数、极值、零、高位哨兵和寄存器别名。若随机生成器默认均匀抽取完整向量，特定十八位溢出端点几乎不可能自然出现，因此定向极值不能由万组随机样本替代。反过来，定向样本也难以代表一般输入组合。报告同时保留两类证据，并分别说明它们为何存在。

2.7.17 从日志到源码的审查步骤

审查设计时，先以数学参考值和事务身份写出预期，再阅读源码中的状态更新条件。对于每个槽位，列出被接受、被推进、被输出、被取消和被复位五种转移的有效位变化，并检查同拍组合。随后在仿真波形中选择至少一条对应轨迹，比较边沿前后状态。只看到最终通过摘要不足以确认源码条件正确，因为测试可能未制造出最危险的同拍组合。

源码评审还要关注“未发生动作”的周期。反压时输出载荷不能漂移，空闲时不应出现有效响应，取消后被清除的身份不能再次出现，时钟门控下未接受的请求不能被错误记账。这些不变量在正常完成日志中可能没有显式文本，因此需要检查断言或观察器是否确实在每个相关周期运行。若检查器只在 ready 为一时观察输出，就无法证明 valid 等待期间的稳定性。

对失败项应保留最短的复现输入、预期与实际差异、关联源码版本和修复后重跑范围。若修复发生在父级输出选择，局部算术通过不能代替父级混合操作重跑；若修复发生在乘积位宽，控制协议检查仍需至少确认没有被状态重构连带影响。回归范围依据修改影响确定，既不机械地仅跑一个用例，也不把所有历史日志不加区分地重新解释为新版本证据。

2.7.18 时序与吞吐取舍的解释

弹性槽提供的是输入与输出之间的缓冲容量，它改善持续请求或短暂下游阻塞时的利用率，却引入额外的状态寄存器和 ready 路径。对独立指令流，流水重叠可以降低平均每条指令的间隔；对严格依赖链，下一条必须等前一条写回，缓冲容量不一定产生同样收益。报告中的性能判断需要与这两类工作负载分别配对，不能只从微架构结构图推出实际加速比。

综合成本也有层级。乘法树位宽、P/R 寄存器规模、控制比较器以及父级选择逻辑分别贡献不同路径和面积。H1 收窄结果状态可能减少寄存器位数，但若增加末端拼接或改变布线，最差时序端点可能转移。P4 缩短内部算术拍数也可能使组合路径穿越更多逻辑，导致父级 ready 或门控使能的余量变差。优化审查需要报告路径端点和约束，而不能把单一面积或延迟变化当作全局结论。

2.7.19 面向下一阶段的接口冻结

第二阶段结束时需要交给第三阶段的是可独立实现的架构语义，而非 RTL 内部的乘法和槽位细节。参考模型应得到指令字、源向量、控制状态和异常条件，并返回预期的目的向量及异常结果。它不应依赖 P/R 具体拍数，否则任何合法的流水优化都会迫使参考模型变化。相反，硬件性能实验需要清楚记录当前实现版本和开关，因为同一架构语义可以对应不同的周期和面积。

软件测试的输入镜像、编译器汇编路径和配置前置条件也应冻结。若第三阶段换用普通 C 接口或新的链接布局，应先核对最终机器指令仍为目标编码，再说明与第二阶段

裸指令测试的共同部分和新增部分。这样，差分验证能够沿用数学与异常合同，性能测量又能诚实反映软件包装的额外开销。两阶段的结果在同一指令语义上衔接，但不会因为软件入口相似就被误认为同一测量合同。

2.7.20 回归失败的归属与收口

回归出现失败时，首先核对失败对应的构建摘要、优化开关和输入集合。若源码未变而检查器改变，应重新判断先前通过记录的有效范围；若源码改变但沿用了旧仿真产物，则本次失败或通过都不能直接归属给新版本。定位时从第一个可复现的错误观察点出发，追溯输入接受、槽位迁移、响应选择和目的写回，尽量保留最短事务序列。后续修订应在同一序列上先证明错误消失，再运行该修订影响到的完整矩阵。

对于多个同时失败的断言，应区分初始错误与连锁症状。例如错误的 ready 可能引起输入重复接受，继而出现结果错位、身份错位和最终计数不守恒；只修补末端比较器不会恢复协议。相反，单个高位结果错误若不影响握手和身份，就应优先检查结果拼接与参考模型的完整向量比较。按因果链归属失败，可以减少针对表面症状的补丁，并使新增测试覆盖真正的触发条件。

收口记录至少包括修订前失败轨迹、源码改动、修订后同输入结果和受影响层级的回归结果。尚未得到父级或全核证据的修订，只能标注为局部修复。若优化开关在正常交付中仍关闭，应在构建配置和报告结论中同时写明，防止读者把候选实验的通过误认为默认实现的通过。这个版本纪律是第三阶段开展配对性能测试和软件集成的基础。

2.7.21 写回身份与功能结果的联合检查

向量点积的数学参考值可以由两个源向量独立计算，但它不能告诉检查器该值是否写入了正确目的。测试平台因此为每个接受事务绑定一个身份记录，包含目的物理寄存器、ROB 位置、指令控制及期望结果。响应到达时先核对身份，再核对完整一百二十八位数据。若先按结果值寻找匹配事务，两笔恰好得到同一结果的请求可能被错误互换而不被发现。零结果、相同源向量和不同目的寄存器的交错请求尤其适合暴露这一盲点。

写回使能与异常字段同样需要随身份核查。异常指令可以在执行路径中占有状态，但不能以正常结果覆盖架构目的；被取消的指令在局部槽位里保留旧数据，也不能因 ready 恢复而重新输出。检查器在每次接受、响应和取消时更新账本，并在结束时核对所有身份的去向。该账本提供了比单纯比较输出顺序更强的约束，也能分辨协议漏响应与结果错误。

完整路径复验还应覆盖目的别名和连续配置变化。若两条指令的源与目的存在交叉别名，重命名阶段必须固定各自的源版本；若配置指令夹在两条点积之间，执行端必须持有各自对应的 VL 与 vtype 快照。最终软件结果可能在少数输入上偶然相同，因而需要结合调度记录、响应身份和异常状态判断。只有这些字段与数学结果共同成立，才可将局部算术通过提升为全核写回路径通过。

2.8 内部 ISA 合同、数学边界和测试向量

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/spec/vdot.md

2.8.1 数学语义与目的位布局

2.8.1.1 源元素编号

从指令执行前的源寄存器状态读取：

```
byte_a[i] = vs1_before[8*i+7 : 8*i]
byte_b[i] = vs2_before[8*i+7 : 8*i]      i = 0…7
signed8(x) = x            if x < 128
           x - 256      otherwise
```

源 [127:64] 不参与运算。内存加载测试使用既有小端裸机环境，数组的第 0 个字节对应源 [7:0]；不能把打印的高位十六进制组误认成 lane 0。

2.8.1.2 精确范围与位宽

单项乘积范围为 [-16256, 16384]，因此八项精确和为：

```
S_min = 8 × (-128) × 127    = -130048
S_max = 8 × (-128) × (-128) =  131072
```

有符号 18 位最大值为 131071，不能覆盖 S_max；至少需要 **19 位有符号数**。原型使用 32 位结果，合法输入不发生 32 位溢出。

RTL 可先采用 8 个 signed 8×8 乘积与加法树：乘积 16 位，按二项、四项、八项归并时分别至少用 17、18、19 位，再符号扩展到 32 位。也可全部以 32 位求和。A 决定结构和流水级，但所有中间结果必须保留符号和足够位宽；不得先用窄位宽加法溢出，再扩展结果。

2.8.1.3 输出定义

```
127                          32 31                  0
+-----------------------------------+--------------------------+
|          全部为 0                 |    S 的 32 位补码        |
+-----------------------------------+--------------------------+
```

示例：S=-8 时，完整目的值为 0x00000000_00000000_00000000_FFFFFFF8。负数只在低 32 位内采用补码，高 96 位仍为零。本文中的“结果字段”固定为 32 位，不把它称为 SEW=8 下的普通第 0 个元素；否则容易错误截断为 0xF8。

这是自定义指令的整寄存器写回约定。vl=8 用来限制合法输入配置，不控制目的只写低 8 个字节；不得由通用 tail/mask 合并逻辑保留旧目的高位，或把高位改成全 1。

2.8.1.4 参考伪代码

以下描述架构效果，不指定时钟周期和具体 NEMU 接口：

```
def execute_vdot(inst, state):
  # 必须先完成编码、向量状态和配置检查；失败则走非法指令路径。
  require_vdot_encoding_and_supported_state(inst, state)

  src1 = read_full_vector_before_write(inst.vs1)  # 128-bit snapshot
  src2 = read_full_vector_before_write(inst.vs2)  # 128-bit snapshot
  total = 0                              # 数学整数，不模拟 RTL 截断
  for i in range(8):
    a = (src1 >> (8 * i)) & 0xff
    b = (src2 >> (8 * i)) & 0xff
    a = a if a < 128 else a - 256
    b = b if b < 128 else b - 256
    total += a * b

  result128 = total & 0xffffffff             # 仅编码低 32 位补码
  write_full_vector(inst.vd, result128)       # 高 96 位因此为零
  # vl/vtype/vxrm/vxsat/vcsr 不变；vstart 仍为 0；向量状态按写回置脏。
```

独立数学模型只计算输入数据与输出；完整指令模型另外实现编码、配置检查和异常，不能把 Python assert 当成处理器异常机制。

2.8.2 合法配置、别名、异常与状态

2.8.2.1 配置合同

表 2-5 配置合同

条件	原型行为及测试要求
VLEN	目标固定为 128；RTL 构建与 NEMU 配置均检查该常量。不承诺跨 VLEN 可移植性
向量状态	运行环境支持并启用 V；mstatus.VS 不能为 Off；沿用处理器已有特权态检查
vtype	vill=0，vsew=000（e8），vlmul=000（m1）；其他非法/保留 vtype 不接受
vl	必须等于 8。vl=0、1、7、9、16 等域外值均不执行点积
vstart	必须为 0；非零值不表示跳过前几个乘积，也不被静默清零后继续执行

表 2-5 配置合同（续）

条件	原型行为及测试要求
vm	必须为 1；vm=0 的相邻编码不属于本版合法指令
vta/vma	tu/mu、tu/ma、ta/mu、ta/ma 均合法；输出仍精确遵循第 3.3 节

推荐首个测试程序使用 vsetivli zero, 8, e8, m1, tu, mu，读取并核对实际 vl/vtype，并确保 vstart=0。该配置指令是准备工作，本身不能让未扩展的处理器识别 vdot。

这是有意收窄的 custom 指令原型域。域外统一非法是团队选择；尤其 vl=0 在本版不作为空操作处理。标准 RVV 指令的通用循环、tail 和 restart 规则不能未经核对直接套入。有关标准向量状态、策略位和重启机制的背景参考 RISC-V Vector 1.0 文档；它没有定义本项目的 vdot 行为。

2.8.2.2 寄存器别名

vd/vs1/vs2 均为 v0…v31，每个操作数只占一个寄存器，无偶数寄存器对齐要求。v0 在本版 vm=1 下可作为普通源或目的，不是零寄存器。

所有源值以指令前状态为准，再写目的。NEMU 应先缓存所需源数据，不能先清空 vd 再逐项读取源。RTL 应正确维护源、目的重命名关系，不能把目的新物理寄存器当作输入。由于完整覆盖目的，算术上不需要旧 vd；物理接入是否存在通用旧目的端口由 A 审查，不得让它改变结果。

2.8.2.3 成功执行与失败执行

成功执行时：

- 只更新一个向量目的寄存器的完整 128 位；其余向量寄存器、标量寄存器、浮点寄存器、内存不变。
- vl、vtype、vlenb、vxrm、vxsat、vcsr 不变；合法初态下 vstart 仍为 0，不能把原有 vxsat=1 清除。
- 沿用现有向量状态置脏机制；不产生浮点异常标志，不访问数据存储器。
- 正常进入下一条指令（本编码长度 4 字节），退休与计数行为沿用处理器正常指令机制。

匹配本指令但配置不合法时，**原型目标行为**为精确非法指令异常（异常码 2）。目的寄存器、运算相关向量 CSR 和内存不产生本指令的部分更新；陷阱 PC 指向该指令。陷阱入口对特权 CSR 的正常更新不属于“不变”范围，tval 填写和异常优先级沿用已验证

的现有机制。测试在状态可达时优先使用单一非法条件，不在本版另造一套异常优先级；存在状态联动时按第 7.2 节记录。

VLEN 不符优先在构建/加载配置时拒绝；若未来允许在其他 VLEN 机器执行相同编码，应另立版本规定其行为。

乱序执行中的请求若被 redirect/flush 取消，不得形成已提交的目的更新。首版不提供从中间 lane 恢复的架构状态；中断与取消复用现有精确提交机制。这些均需全核验证后才能勾选完成。

2.8.3 候选机器编码

2.8.3.1 编码选择

【原型约定】选择 32 位 custom-0 主 opcode 0x0b。官方 opcode 表为自定义扩展预留 custom 空间；这不表示下列 funct 字段由官方分配。标准 opcode 表

表 2-6 编码选择

位段	字段	v0.1 候选值
[31:26]	funct6	000000
[25]	vm	1；本版只支持无掩码
[24:20]	vs2	第二个编码源寄存器编号，0～31
[19:15]	vs1	第一个编码源寄存器编号，0～31
[14:12]	funct3	000
[11:7]	vd	目的向量寄存器编号，0～31
[6:0]	opcode	0001011（0x0b）

汇编记法沿用向量常用顺序 vdot.vv vd, vs2, vs1。数学公式中 vs1/vs2 的名称与位段一致；点积交换律会掩盖源颠倒，所以编码测试还必须检查字段提取本身。

```
MATCH_VDOT = 0x0200000b
MASK_VDOT  = 0xfe00707f
识别条件： (instruction & MASK_VDOT) == MATCH_VDOT
编码公式： MATCH_VDOT | (vs2 << 20) | (vs1 << 15) | (vd << 7)
BitPat：  b0000001??????????000?????0001011
```

例：vdot.vv v10, v8, v9 的候选机器字为 0x0284850b；小端指令字节为 0b 85 84 02。寄存器编号为向量寄存器编号，即使字段位置与 R-type 相似，也不能在主译码中把它们当作通用整数寄存器。

vm=0 或 funct 字段不符时不匹配本版 vdot；若未被其他合法扩展占用，应走默认非法指令路径。不得通过缩小 mask 接受未定义的相邻编码。

2.8.3.2 工具链与冲突检查边界

首版先用独立.S 文件中的.word 0x0284850b 或经过校验的编码宏生成上述机器字。后续封装由 B 负责；宏必须检查寄存器编号范围。未新增助记符支持前，反汇编可能显示原始字或未知指令，应核验机器字和字段，而不能以“打印出了 vdot 名称”作为前置条件。

如使用 C 内联汇编，须另行处理寄存器绑定、clobber 与编译器重排；不要把.word 放进没有约束的内联汇编后假定编译器知道它读写向量寄存器。

2026-10-11 的只读静态检查在锁定版本的 **XiangShan 当前实际译码表和 NEMU 主译码表**中未发现与上述候选 mask/match 的交叠。已有 nemu_trap/TRAP 使用 0x6b，并非 0x0b。rocket-chip 依赖中确有与候选重叠的 CUSTOM0 通配定义，但属于 Rocket RoCC 译码体系，未接入本版香山实际译码表；未来若启用该通路，必须重新审查分配与仲裁。此结论只覆盖已检查的静态模式，不能代替接入后的译码单测与全核回归；候选编码尚未实现。

2.8.4 RTL、NEMU 与验证共同遵守的接入要求

2.8.4.1 锁定版本的阅读入口

表 2-7 锁定版本的阅读入口

子系统	相对路径	需要核查和完成的事情
XiangShan 指令模式	src/main/scala/xiangshan/backend/decode/Instructions.scala	增加候选模式，检查邻近编码和与既有扩展的冲突
XiangShan 向量分类	src/main/scala/xiangshan/backend/decode/isa/bitfield/RiscvInst.scala	isVecArith 当前只识别 OP-V；custom-0 不会自动被识别为向量算术
XiangShan 译码控制	src/main/scala/xiangshan/backend/decode/DecodeUnit.scala	正确的向量源/目的类型、配置依赖、合法性检查、微操作和写回控制
XiangShan 执行接入	src/main/scala/xiangshan/backend/fu/FuConfig.scala，fu/wrapper/，fu/vector/	选择可复用通路或新单元、执行资源、结果位宽、完整目的写回和取消机制
NEMU 主译码	src/isa/riscv64/instr/decode.c	分派 custom 主 opcode；检查向量预分类和执行入口，不能只加 RVV 子表

表 2-7 锁定版本的阅读入口（续）

子系统	相对路径	需要核查和完成的事情
NEMU 向量实现	src/isa/riscv64/instr/rvv/decode.h、exec.h、vcompute.h、vcompute_impl.c	编码、配置检查与独立算术实现；避免误复用通用 tail/restart 行为
NEMU 寄存器访问	src/isa/riscv64/instr/rvv/vreg.h、vreg_impl.c	有符号源读取、先读后写、完整 128 位写入
NEMU 声明与差分状态	src/isa/riscv64/include/isa-all-instr.h、src/isa/riscv64/difftest/ref.c	执行入口声明、向量和 CSR 状态同步，确认实际比较范围

这些是源码阅读入口，不要求机械修改表中的每一个文件。来源：固定 XiangShan 译码目录、固定 NEMU 主译码、固定 NEMU RVV 目录。

以下是本版源码核查发现的具体接入风险；行号对应表中固定硬件/NEMU 提交：

- RiscvInst.scala:93 的向量分类需要识别精确 vdot 模式，不能把全部 custom-0 都归为向量。DecodeUnit.scala:897 的向量禁用检查还依赖 FU 分类，因此只加 BitPat 不够。
- DecodeUnit.scala:775 的输入带 vtype/vstart，未直接带实际 vl；vl=8 限制须通过正确的 VL 依赖，在能取得 srcVConfig.vl 的路径检查，例如 fu/vector/VecPipedFuncUnit.scala:54，并接通非法指令异常输出。
- DecodeStage.scala:236 对 v0 目的有单独写使能，重命名也有相应通路。允许 v0 就必须覆盖它，不能只测 v8/v9/v10 后认定全部寄存器均可用。
- NEMU 的 instr/rvv/decode.h:167 在既有 vopivv 子表中把 funct6=0 解释为 vadd，且不再检查主 opcode。**不能把 custom-0 直接分派到现有 OP_V 执行子表**；应建立独立 vdot 执行分派，可以复用字段解析和寄存器访问。状态检查/置脏参考 instr/vp.c 的 vp_enable()、vp_set_dirty()。

2.8.4.2 硬件接口边界

A 在 T06 维护 RTL 接口与接入设计（2026-10-12 已形成第一版草案），B/C 复核具体端口与时序，人工评审待记录。本文只锁定必须实现的性质，不虚构固定周期数：

1. 运算请求携带正确的两个向量源值和请求身份；配置检查必须使用与该指令对应的向量状态。

2. 只有请求真正被接受后才进入执行；输出反压时保持结果及身份，不能丢失、重复响应。

3. 正常完成形成一个完整 128 位架构目的结果；内部微操作数和流水级由接入设计决定。

4. 结果与目的物理寄存器、ROB/取消标识保持一致；flush 后不能提交已取消的结果。

5. 全寄存器写回绕开或正确配置通用 tail/mask 合并逻辑，保留高 96 位清零要求。

6. 功能单元延迟和发射间隔分别定义、分别测量；本文不预先承诺“一周期”或加速倍数。

2.8.4.3 NEMU 与交付约束

B 先写独立数学模型，再实现 NEMU。不要把 RTL 的加法树、中间截断或共享错误复制成参考模型；不能通过跳过 vdot 的 DiffTest 比较获得“通过”。C 应注入一次可控错误，证明比较路径能发现目的值或高位清零错误。

共享参考库与 standalone NEMU 应分开构建，保存各自提交、配置和实际加载路径，避免覆盖已有环境产物。NEMU 修改应提交至团队可访问的 NEMU 仓库或采用明确可复现的补丁交付；确定方式并更新版本引用后，其他成员才算能够完整复现。

2.8.5 测试与验收合同

本节是**待实现的测试清单**。表中期望值由数学定义得出，不是已完成的仿真结果。所有功能测试必须比较完整 128 位目的值。

2.8.5.1 定向算术向量

[x]×8 表示重复 8 次。列表按 lane 0→7 排列，未另行说明时源高 64 位为零；输出高 96 位在所有用例中均为零。

表 2-8 定向算术向量

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D00	[0]×8	[127]×8	0	00000000
D01	[1]×8	[1]×8	8	00000008
D02	[1,2,3,4,5,6,7,8]	同 A	204	000000CC
D03	[-1]×8	[1]×8	-8	FFFFFFF8
D04	[-1]×8	[-1]×8	8	00000008
D05	[127]×8	[127]×8	129032	0001F808
D06	[-128]×8	[127]×8	-130048	FFFE0400

表 2-8 定向算术向量（续）

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D07	[-128]×8	[-128]×8	131072	00020000
D08	[1,-2,3,-4,5,-6,7,-8]	[8,7,6,5,4,3,2,1]	0	00000000
D09	[-128,0,0,0,0,0,0,0]	[-1,0,0,0,0,0,0,0]	128	00000080
D10	[0,0,0,0,0,0,0,-128]	[0,0,0,0,0,0,0,127]	-16256	FFFFC080

2.8.5.2 数据布局、状态与控制

表 2-9 数据布局、状态与控制

ID	测试内容	判定标准
L01	逐 lane 一热：只令 A 的 lane i 非零，B 的 8 项互不相同，i 遍历 0～7	每次等于对应那一项乘积，检测遗漏和单侧错位
L02	保持低 8 项不变，两个源高 64 位分别换成随机值、全 FF、交替 55/AA	结果完全相同
L03	独立目的寄存器预填全 FF，再执行 D03	目的完整值为 00000000_00000000_00000000_FFFFFFF8
L04	不重叠、vd=vs1、vd=vs2、vs1=vs2、三者相同；含 v0/v31	使用实际执行前源快照算期望；别名不能使源被提前破坏
L05	四种 vta/vma 组合	每种均得到相同完整目的值，保留原配置
S01	vxrm 四种值、vxsat 0/1 组合，预置后执行合法指令	算术结果不变；上述状态及其 vcsr 映射保留
S02	检查 vl/vtype/vstart、非目的寄存器与内存	除正常提交、向量置脏等既有机制外，无额外副作用
E01	逐项改变 SEW、LMUL、vl、vstart、vill，或关闭向量状态	模型显式拒绝；完整指令实现发生非法指令异常，无部分结果

表 2-9 数据布局、状态与控制（续）

ID	测试内容	判定标准
E02	vm=0、逐个翻转固定 funct 位；合法寄存器字段遍历	只识别精确 mask/match；确认相邻模式未被误译码为 vdot
C01	连续请求、输出反压、结果依赖、同一目的连续写、flush	无丢失/重复/错序提交，取消后无已提交写回
R01	原 hello/vadd 及相关向量回归	保持正确；vdot 新路径不破坏旧指令

L01 无法检测“两侧同时采用相同排列”的内部置换，因为点积对这种置换本来就不变；不要为无法观察且不影响语义的内部排列编造失败标准。CSR/非法配置测试应使用合法准备指令并保存实际 CSR 值，避免测试准备过程就已失败。

非法配置须通过实际可达状态构造：用标准 vset* 指令设置非法 vtype 时，vill=1 与 vl=0 联动，此例只验证目标 vdot 的非法异常及无部分更新，不能据此声称单独覆盖了 vill 检查或异常优先级。测试非零 vstart 时，应在向量配置、加载等准备完成后最后写入，避免被准备指令清零。向量状态关闭案例也应先完成向量数据准备，最后再关闭 VS。

2.8.5.3 随机验证与证据

- 软件模型/算术单元起步使用固定种子 0x20261011，至少 10,000 组 signed int8 输入；全核冒烟先用上述定向用例，再选 32 组固定随机用例。数量是初始工程安排，不是官方评分门槛。
- 保存生成器版本、实际输入、完整期望值、合同版本。仅有随机种子不一定能跨语言复现随机数序列。
- 交叉检查交换两个源、同时重排两边低 8 项等不变量。涉及 int8 取负时排除 -128，避免把不可表示的 +128 当成合法输入。
- 失败应输出指令字、PC、配置、源/目的编号、指令前源值、完整实际/期望目的值；定向用例输出用例 ID。
- 独立模型、RTL 单元、NEMU、全核 DiffTest 分别保存结果，不能用某一层通过替代其他层。

软件读取结果时要覆盖全部 16 字节。例如执行后切换到 e32,m1,vl=4，保存四个 32 位元素，再恢复约定的配置；C 应区分“被测指令副作用”和“测试程序为读回结果而改变的配置”。只保存 e8 的 lane 0 会漏掉错误。

2.8.6 RVV 性能基线必须保持的等价性

【题目明确】需要用现有 RVV 指令组合完成相同功能，并比较延迟。候选基线按“8 位有符号输入 → 16 位精确乘积 → 32 位扩宽归约”设计，例如研究 vwmul.vv 与 vwredsum.vs 的组合；中间切换的 SEW/LMUL、寄存器分组和零初值必须匹配。这里不是已通过验证的汇编实现。

B/C 验收基线时必须逐项确认：

1. 只计算源低 8 项，从零开始，不把旧目的或源高半部分加入。

2. 不使用会在 8/16 位提前截断的组合；D06、D07 必须通过。

3. 最终目的低 32 位及高 96 位与第 3 节完全相同；普通归约指令的尾部策略不能代替显式验证。

4. 双方按共同约定的可观察入口/出口状态验证功能等价：目的 128 位结果及向量 CSR 必须一致，vl/vtype 恢复实际入态。允许破坏的临时（scratch）寄存器必须预先列明并排除于比较集合，源/目的别名用例另行证明等价。该口径不能被称为“除目的外所有架构状态都保持不变”；第 4 节的严格副作用限制仍适用于单条 vdot 本身。

5. 分别报告核心算术和上述合同范围内完整序列的周期；必要的清零、配置切换、恢复和搬运开销不得只从基线一侧隐藏。装载、存储是否计时，两边保持一致。

6. 同一硬件基线、同一输入和测量边界下比较；声明测量的是依赖延迟还是吞吐。Verilator 主机运行秒数不能当作处理器周期加速比。

具体序列与测量事件由 B/C 在后续 team/spec/measurement.md（待创建）中确定。本文不承诺加速比或获奖分数，先以完整性、正确性和可复现证据为验收顺序，再做流水线和扩展优化。

2.8.7 歧义清单与变更流程

下列清单供 B 整理后，通过比赛已有咨询渠道向命题方确认。**截至本版没有在仓库归档答复，也没有代表团队发出外部消息。**

表 2-10 歧义清单与变更流程

编号/优先级	需要确认的问题	本版候选及受影响内容
Q01 / P0	图中的 8 项是否为最终验收语义？是否明确取低编号 8 项？	固定低 8 项；影响读取范围、算术规模、测试和基线
Q02 / P0	“vd 第一个元素”的位宽是什么？是否接受低 32 位精确结果、高 96 位清零？	第 3.3 节；影响目的格式、写回和完整结果比较

表 2-10 歧义清单与变更流程（续）

编号/优先级	需要确认的问题	本版候选及受影响内容
Q03 / P0	是否要求精确结果？是否另有截断、饱和或旧 vd 累加规则？	精确从零求和；影响数据通路与边界结果
Q04 / P0	vl/LMUL/mask/vstart 的必测范围是什么？是否接受第 4 节的受限原型域？	固定 e8/m1/vl8/vstart0、无掩码；影响控制及异常范围
Q05 / P1	vta/vma 是否影响“其他通道清零”？是否要求源目的重叠支持？	策略位不改变完整结果，全部单寄存器别名支持
Q06 / P1	是否有指定或禁止的指令编码？是否允许使用 custom-0？	第 5 节；影响 RTL、NEMU 与汇编封装同步
Q07 / P1	延迟比较是否有指定输入/出口状态、计时边界或基线要求？	按第 8 节先验证等价，再分别报告各项开销

如获得回复，在 team/spec/clarifications/（待创建）保存日期、来源和原文/截图，再在表中补充结果。没有答复不等于官方认可。等待期间可按明确标记的原型版本开展模型、算术单元和用例工作。

行为变更流程：B 提出“旧行为→新行为、依据、受影响用例”；A/C 评审；升级合同标识；同一轮更新 RTL、NEMU、编码封装和测试数据。旧版本结果仍保留版本标签，不能混入新版性能统计。

2.9 两槽弹性状态、父级协议及验证不变量

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/optimizations/o2/CONTRACT.md

2.9.1 接口与状态

方案 A 保留 FuConfig.piped=false、UncertainLatency()；这里 piped 字段属于现有框架配置约束，不表示新模块内部不能有两个流水槽。该字段若为 true，现有 FuConfig 要求 certain latency，不能为名称好看而更改时序合同。

新候选使用独立的 FuncUnit 实现（可复用 VecFuncUnitAlias 读取输入），不得直接依赖 VecNonPipedFuncUnit 的一份 RegEnable/DataHoldBypass 输出身份；也不得套用要求 certain latency 的 VecPipedFuncUnit。

表 2-11　接口与状态

槽位	有效状态与数据	必须一同保存的身份
P：乘积槽	valid、8 个 16 位有符号乘积、输入合法性	接受时的 ROB、pdest、vecWen/v0Wen、需要输出的完整 vpu 控制及 perfDebugInfo；字段清单按 FuncUnitOutput 逐项核对
R：结果槽	valid、完整 128 位结果、异常向量	从 P 原样转移的同一包身份，不能从当前输入取得

非法请求照常占一个槽并终结一次，不能在算术或元数据通路上提前丢弃。v0 与普通向量目的互相交替时，各自写使能及 pdest 不得串线。所有输出控制、异常及 perf 信息由 R 提供；输入在反压期间任意变化也不能改变旧响应。

给父级提供明确的功能状态端口：两个实际 valid 或其精确 occupancy/empty 归约。时钟门控和派发使用这些功能状态，不能用 printf、调试 seq、tap 日志或推测的最近一条 ROB 替代。若父级为验证而保存副本计数，必须每拍断言与 FU 真实状态一致。

2.9.2　每拍转移定义

以下量在同一时钟边沿前计算，ready/valid 遵循 Decoupled 握手。P、R 各自调用该请求的 robIdx.needFlush(io.flush)，保留硬件 RobPtr 的 flag 与环绕语义。

killP = P.valid && P.rob.needFlush(flush)

killR = R.valid && R.rob.needFlush(flush)

liveP = P.valid && !killP && !reset

liveR = R.valid && !killR && !reset

out.valid = liveR

out.fire = out.valid && out.ready

rReady = !liveR || out.ready

move = liveP && rReady

pReady = !liveP || rReady

in.ready = pReady && parentAllowsVDot && !flush.valid && !reset

in.fire = in.valid && in.ready

nextR.valid = (liveR && !out.fire) || move

nextP.valid = (liveP && !move) || in.fire

当 move 为真，R 锁存 P 的归约结果和整包身份；当 in.fire 为真，P 锁存新输入的乘积和整包身份。move 使用边沿前的旧 P，绝不使用同拍新输入。reset 强制两个 valid 及父级占用清零；无效槽的数据无需清零，但不得影响可见输出。

表 2-12 每拍转移定义

边沿	接受	槽间转移	输出握手
E0	A 进入 P	无	无
E1	B 进入 P	A 进入 R	无
E2	C 进入 P	B 进入 R	A
E3	D 进入 P	C 进入 R	B

表中为从空管线开始、持续 ready 且无取消的目标时序，不是测得的 O2 波形。已有 R 但输出阻塞时，空 P 仍可接收一条；两槽满且 R 阻塞时拒绝输入。R 被取消而 P 幸存时允许 P 进入腾出的 R；P 被取消而 R 幸存时不能误清 R。

重定向当拍统一不接受新请求；未被取消的旧 R 可正常握手，幸存 P 可转移。被取消请求的 KILL 优先于它的 RESPONSE，二者不能同时计数。无关重定向不能破坏已有停顿响应。RESET 优先级最高，复位当拍无输入/输出握手。计数时使用 effectiveKillP=killP&&!reset、effectiveKillR=killR&&!reset；RESET 丢弃数取边沿前两个 valid 之和，reset 与 flush 同拍不能把同一请求同时记为取消和复位丢弃。

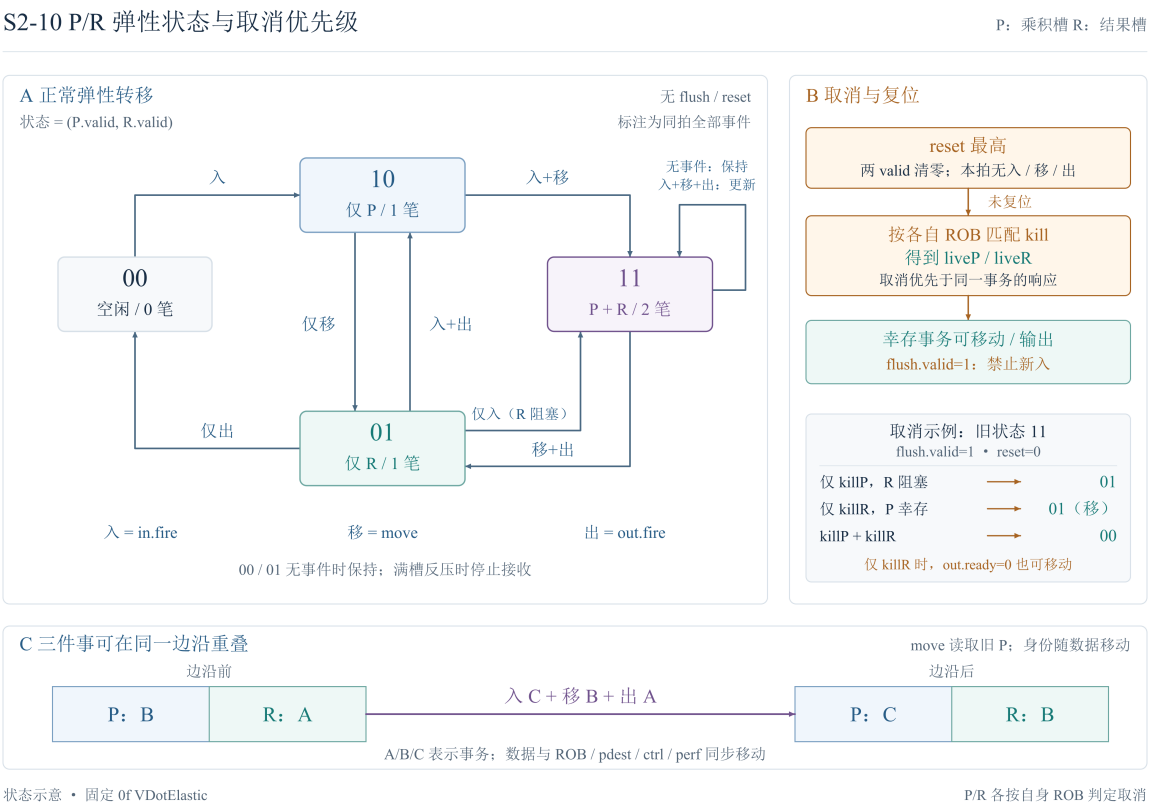


图 2-10　P/R 弹性状态与取消优先级

注：状态位按 (P 有效,R 有效) 排列，区分输入接收、P 至 R 移动、输出握手和保持；reset 及 ROB 取消优先处理，事务身份随数据移动。

来源与范围：用户提供示意图；依据：本文状态方程、0f 实现及 O2 合同不变量。版本范围：历史合同仅作依据，图中所有行为以最终 0f 核对。

2.9.3　父级资源、派发与除法切换

当前 VFEX4 含 VDot、VIDiv、VFDivSqrt；共享输出是 one-hot 选择和 ready 广播，缺少多 FU 同时完成仲裁。本候选维持互斥：

1. 有任意 vdot 在途时，只能向 vdot 追加请求；DIV 禁止接收。

2. 任意 DIV 在途时，vdot 禁止接收，保持已有 DIV 串行行为。

3. vdot 最后响应或取消当拍不切换 DIV；在下一拍确认两个槽确实空、无旧 vdot 输出后，才可接受 DIV。

4. DIV 完成/取消后也必须确认旧输出已经撤销、子单元具备合法接收状态，再放行 vdot；不依赖取消当拍旧 DIV valid 立即变低的假设。

5. 所有跨 FU 切换都不得出现两个 out.valid 同时有效。DIV 自身已有取消当拍行为需原样观察并按现有全核合同解释；不能为了让新检查器通过而改写旧预期。

6. 持续 vdot 到来可能使排空切换延迟；不承诺新的公平调度。混合负载测试须记录等待，有限输入停止后必须能排空并完成 DIV。

在 O2 专用的 VFEX4 路径局部选择被选 FU 的 ready，或采用等价的原子派发实

现；不修改所有 EXU 共享的 Dispatcher 语义。现有 Dispatcher 把所有 ready 相与，但 child.valid 并未与总 ready 相与，不能仅放宽父级 busy 而让 child 单独接受。必须断言：

parent.in.fire == OR(all child.in.fire)

PopCount(all child.in.fire) <= 1

child.in.fire -> parent.in.fire && selected(child)

父级不能继续用单个 busy 和最近输入 RobPtr 表示多个 vdot。vdot 占用从真实槽状态取得；DIV 占用及身份独立保存。任何一笔 vdot 响应/取消只释放该事务，不能把另一个在途请求一起“释放”。

输入选择只依赖已注册占用、当前合法选择及下游 ready；不得构成 valid/ready 组合环。选择后未握手的输入仍由上游保存，FU 不得提前更新身份。

2.9.4 门控、唤醒与写回

- 当前不定时延门控只用一个 uncer_en_reg，out.fire 且没有 in.fire 时会清零；O2 仍有 P 请求时这会错误关钟。vdot 专用使能必须覆盖当前任一槽有效、当前输入接受以及必要复位/清除边沿。
- 当前槽有效时，即使 flush 将其全部取消，也必须保留处理本次清除所需的时钟边沿；排空后再关门。父级状态逻辑位于未门控时钟域。
- 只对 O2 vdot 改门控;DIV 及其他 EXU 保持现状。真实父级测试使用真实 ClockGate，不能用常开时钟替代最终验证。必须明确 reset 在实际门控使能/复位传播中的接法，并覆盖空槽且已关钟时 reset、非空时 reset、cancel/reset 同拍及随后重新唤醒；抽象状态公式不能证明复位脉冲确实到达门控后的寄存器。
- A 保留既有不定时延完成/唤醒与实际写回仲裁 ready。某次输出未被接受时，R 必须保持全包稳定；不得假定写回总是 ready。
- 不新增固定延迟提前唤醒、写回预留或旁路承诺。若发现现有 issue 反馈限制 II=1，先记录并评估，不自动扩大到调度重构。
- 若转选 B，必须另修合同并重审 FuConfig、ctrlPipe、调度/唤醒、寄存器读端口及写回预留；禁止在本合同下悄悄迁移执行路径。

2.9.5 观察协议与不变量

另立 O2_SCHEMA version=1，以未门控全局周期记录，不让 O1 单在途检查器“放宽后”兼任 O2。每次父级接受产生唯一观察 seq，槽间转移携带 seq；seq 仅用于观察，不参与硬件功能。

最低事件：父级接受与等待（请求种类、选择、ready 及阻塞原因）、P_ACCEPT、P_TO_R、R_STALL、R_RESPONSE、P_KILL、R_KILL、RESET 丢弃、槽占用、DIV 开始/结束和重定向。按周期处理旧事务终结、移动和新接受，保留原始输出顺序，不假设 printf 先后决定硬件先后。

必须满足：

- 任意前缀：接受数 = 响应数 + 取消数 + reset 丢弃数 + 当前在途数；同一事务恰好属于一个终结类别。
- 当前在途数在 0..2；两个槽不能持有同一 seq；输出来自 R 中那个身份。
- 幸存输出按接受顺序，不允许重复/幽灵响应；完整 128 位及异常/写使能与独立模型一致。
- 多槽取消逐笔按 ROB 年龄判断，覆盖年龄顺序与接受顺序相反、flag 环绕、flushItself 两种情况。
- 反压保持整个 R 包稳定；只有匹配取消/reset 可以撤销旧响应。
- 父子握手一致，父级 out 与被选子输出一致，FU 输出 one-hot，门控不能冻结未终结请求。
- 正常运行结束无悬空请求；主动故障终止只能标部分检查，不能记为完整通过。

2.9.6 分阶段验收与负控

表 2-13 分阶段验收与负控

门禁	必须实际完成的内容	进入下一步的条件
G0 评估/合同	旧轨迹复算、两条接入路线审查、候选范围及停止条件	允许做隔离原型，不代表 O2 采用
G1 算术与弹性 FU	原 22+10000 完整 128 位向量；连续满管线、两槽反压/腾空；合法非法交错、v0 交错；部分/全部取消、reset 和环绕	无反压时真实 II=1 且最早响应 2 拍；事务守恒与反例检出通过
G2 真实父级	真 ExeUnit+VDot+VIDiv+VFDivSqrt+ClockGate；全部 FU 切换、满管线反压、cancel 与切换重合、持续供给及最终排空	无误派发/双完成/误关钟；其他 FU 受影响范围回归通过

表 2-13 分阶段验收与负控（续）

门禁	必须实际完成的内容	进入下一步的条件
G3 全核	新 O2 EMU、正常 NEMU；hello/vadd、117 唯一功能/异常矩阵、T12 等价控制压力；故障参考库及正常恢复	全部 DiffTest/完整状态通过，新观察事件覆盖达标
G4 公平性能	冻结相同 16 镜像、参考库、工具配置，O1/O2 各自实测 custom 与 RVV；同合同四场景和相同样本规则	有实际净收益、退化可解释、结果与身份绑定可复算
G5 采用与交付	O3 应用数据、O4 结构/时序证据、O5/T17 队友干净检出构建和人工复核	选择最终模式、冻结后再集中编写 T16/T18 材料

定向负控至少包括：交换 P/R 的 pdest 或 ROB；篡改高位结果；漏取消其中一槽；误杀另一个幸存槽；重复/丢失响应；R 反压时修改字段；父子握手不一致；双 FU 输出有效；有请求时误关钟。区分真实硬件故障注入与日志/检查器故障注入，不以后一种冒充前一种。

性能采用条件：先争取冻结 batch128 相对 O1 再降至少 10%（362.50→不高于 326.25 cycles），这是本合同新增的工程投入目标，不是预言、官方标准或正确性通过线；必须同时报告全部场景，其他保留场景均值退化超过 2% 先调查。若 RVV 保持 678，这个目标约对应 2.078×，正式加速比仍用候选自身实测 RVV。达到 2× 本身不自动通过采用门禁。

2.10 本轮已提供截图补充

本节收录已有补充截图，并按各自历史版本和证据层级解释。正式图位已补齐；尚未提供的原始波形或专门异常截图，其补证范围按正文保留。

(a) Fixed 0f VDotProducts source excerpt

```
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

(b) Fixed 0f VDotReduce source excerpt

```
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

图 2-11　固定 0f 算术实现源码摘录

源代码截图辅助解释 8 路乘法和归约；不替代 S2-02 数据通路图。

Host model checks only; no RTL/NEMU execution in this command



图 2-12 主机端参考模型检查画面

35 个测试方法通过；该命令只检查主机模型和数据，不授 RTL 或 NEMU 资格。

Historical 0f observer log excerpt; NOT a GTKWave waveform

```
S2-05 SUPPORTING EVIDENCE: VDOT TWO-SLOT EVENT LOG EXCERPT
Historical 0f clean-02, dense-batch128, 2026-10-27 (UTC).
Observer event log excerpt; this is NOT a waveform screenshot.
Selected fields copied from the source lines listed below; fields omitted.
Source: /home/riscv/riscv-lab/software-import-20261026/clean-02/fullcore-run-01/runs/dense-batch128/run.log
Source SHA256: 18f472d75feef0378bec71c64793ffd2297bc37c47a5b501597f222da174b98f

line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
Source result.json reports O2_DENSE_ELASTIC_AND_WINDOWS_PASS and exit_code=0.
Original batch UTC capacity audit FAIL retained; see R5/G5 acceptance RESULT.md.
```

图 2-13　两槽重叠事务日志摘录

保留 10986 周期 ACCEPT seq2、MOVE seq1、RESPONSE seq0；本图为历史日志字段摘录；S2-05 现为父级逐拍记录的事件重绘，未提供原始波形截图。原批次 UTC 容量审计 FAIL 保留。

(a) Slot registers and 32-bit result state

```
    require(!cfg.writeFflags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

(b) Handshake and cancellation conditions

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

(c) State update and old-P transfer

```
PS C:\WINDOWS\system32>
                       > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

图 2-14　固定 0f 两槽状态与控制源码摘录

依次展示寄存器、握手与取消条件、状态更新；源代码画面辅助解释实现；S2-06 至 S2-08 的控制行为及证据范围以各图所列原始记录为准。

第三阶段协同仿真与性能分析文档

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

工程版本为 0f ＋ firstdot V2 ＋原 C8，初稿日期为 2026 年 11 月 2 日。

3.1　验证与评估目标

本阶段通过独立 NEMU 语义模型与香山 RTL 协同仿真验证指令行为，并采用等价 RVV 实现评估周期收益。工程评价同时考虑完整结果、异常、状态恢复、控制正确性、软件调用边界及硬件成本。

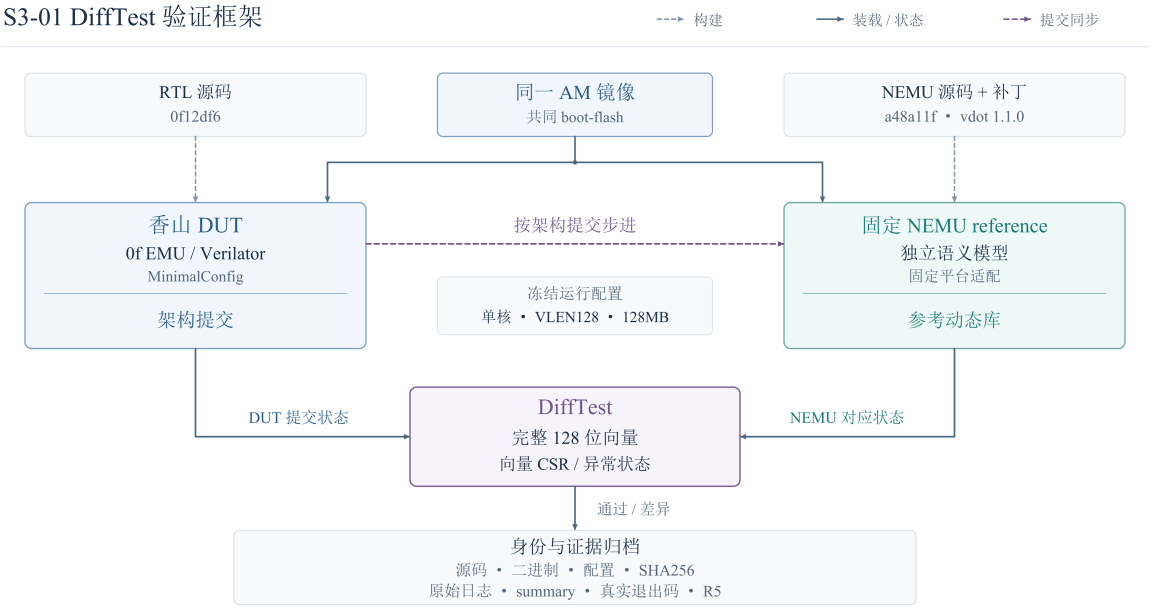


图 3-1　RTL 与 NEMU 独立模型的 DiffTest 框架

注：同一 AM 镜像分别驱动 RTL DUT 和固定 NEMU 独立模型，在架构提交边界比较完整 128 位向量结果及 CSR，并归档版本、配置与运行证据。

来源与范围：用户提供示意图；依据：冻结 EMU/NEMU 身份、差分协议和 R5 结果。版本范围：稳定 0f ＋固定 reference；同一镜像。

3.2　NEMU 扩展

NEMU 以 a48a11f208ff81ad765591d83271d1b339fb3861 为基点，使用 vdot 补丁 1.1.0 及 xs-16ae9dd-vdot-runtime-tval-v1 平台配置。补丁绑定 7 个变化文件，SHA256 为 6c16 5bf5f94ef0bfb4f894a5f06eb6d620d4a7e0e12f3b1400fa72811f5d24db。

执行过程先匹配 custom-0 编码并检查合法向量状态，再快照读取两源，逐项转换 signed int8、从零求和并写回完整 128 位结果。独立软件求和有助于与硬件乘法/归约结构交叉验证。

平台适配对齐非法指令 xTVAL 行为。固定香山中，译码已知非法与 FU 发现 VL 非法的路径具有不同 xTVAL 表现，reference 按同一架构谓词建模。对应检查仍覆盖异常码、陷阱 PC、xTVAL 及目的状态。全核实验使用已验证 reference 角色，特权验证范围以现有 M 模式记录为准。

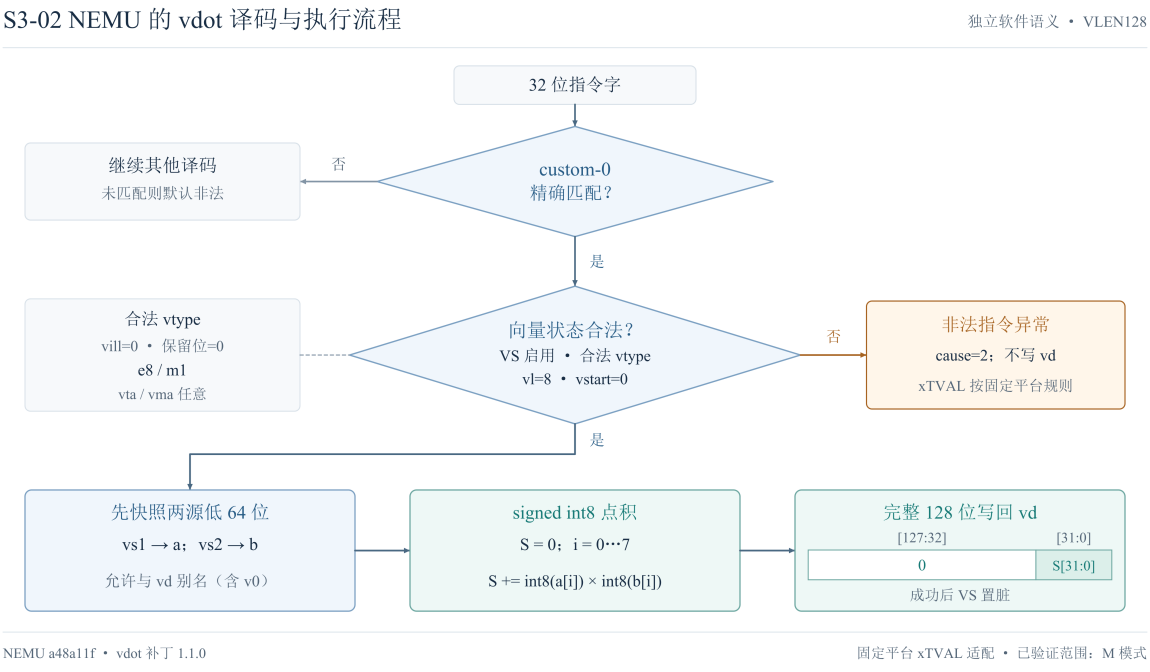


图 3-2　NEMU 的 vdot 译码与执行流程

注：精确匹配 custom-0 编码并检查合法状态后，先快照两个源，再完成 signed int8 点积和完整 128 位写回；非法配置进入异常路径，平台 xTVAL 限制按正文保留。

来源与范围：用户提供示意图；依据：team/patches/nemu 及冻结 reference 实现。版本范围：按已实现版本绘制；平台 xTVAL 范围按正文保留。

3.3　差分、拒错与恢复

DiffTest 将硬件提交后的架构状态与 NEMU 对应状态比较。除正常算术结果外，本项目检查完整向量写回、向量 CSR、寄存器别名和非法状态路径。

负控对 vdot 结果的 bit100 注入错误，检验高 96 位比较是否真正生效。图 S3-03 展示 2026 年 10 月 24 日原 0f 记录中的 negative-vdot 拒错与 functional-0 同镜像恢复两次运行，绑定 EMU 2d1db77c…与 BIN b8b7e1a7…。原高 64 位的第 36 位对应完整向量的 bit100；首次失配 PC 为 0x80001bdc，机器字 0x0294050b，目的 v10。恢复正常 reference 后，15 个计划用例全部通过并以 GOOD TRAP、exit 0 结束。这组图示证据覆盖拒错与恢复两次运行。

后续 R5/G5 在另一组冻结运行身份下选择原 13 项记录与 1 项新控制诊断组成验收，原始记录重新解析后通过。全核功能矩阵包含 117 个不同用例及 124 次分片执行。新控制诊断检查 131072 字节寄存器结果。算术模型、局部 RTL、全核作业和应用矩阵的数量按其范围分别记账。

R5 clean-02 原宿主 UTC 并发审计存在 80.119ms 表观四路重叠，旧 FAIL 记录保留。补证以同 boot/PID 单调包络完成所选审计，组合 UTC 峰 3、补项单调峰 1。系统 UTC 稳定性仍待建立，性能采用 DUT guest 周期。

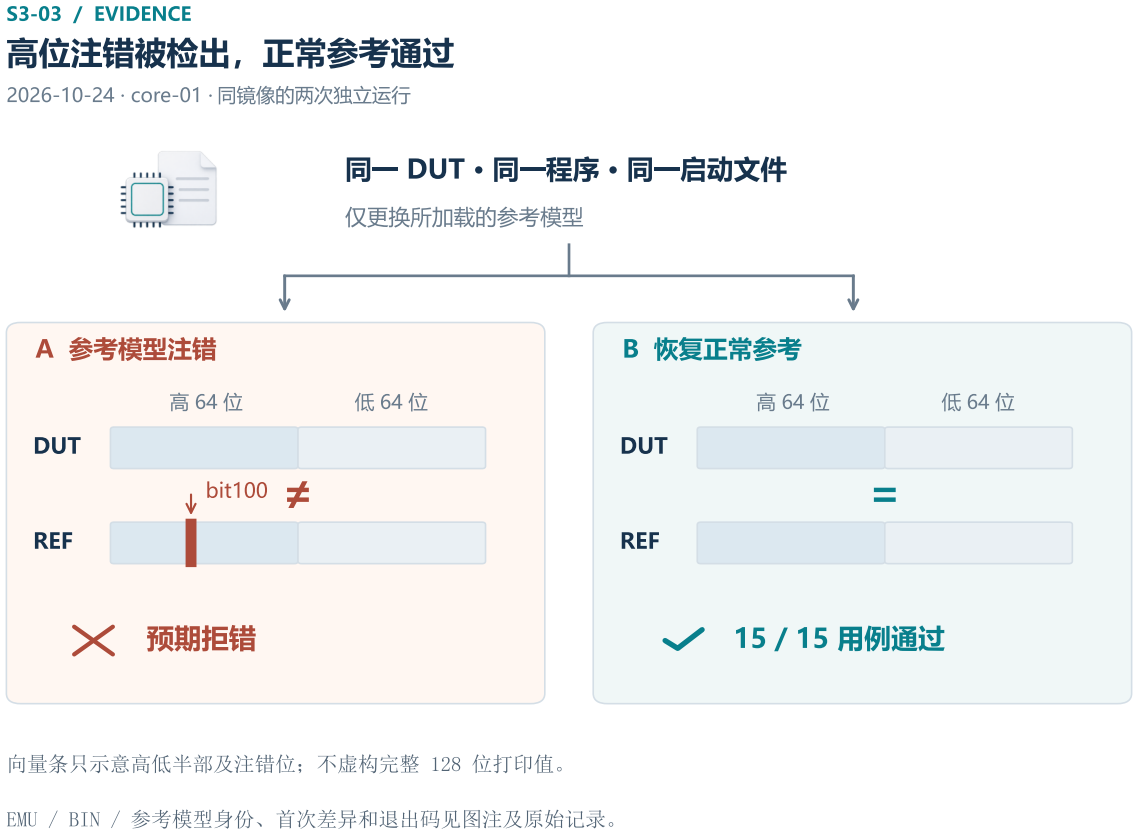


图 3-3 0f 原始 bit100 拒错与同镜像恢复记录

注：2026-10-24 core-01：负控首次失配 PC 0x80001bdc、机器字 0x0294050b、目的 v10；原高 64 位 right/wrong 十六进制打印见附件，第 36 位对应完整 128 位 bit100。恢复正常 reference 后 15/15 与 GOOD TRAP、真实 exit 0；负控 exit 1。

来源与范围：e2c2e8d 固定归档：team/optimizations/o2/r3-div-gate/fullcore/evidence/20261024-fullcore/files/runs/core-01/{negative-vdot,functional-0}/{stdout.log,result.json}；图数据附件保存原文、Git blob 和 SHA256。版本范围：历史 EMU 2d1db77c…与 BIN b8b7e1a7…；不是 R5 clean02/acf672…日志，不添加本组独立 replay，不把 ABORT 事件 PC 等同首次失配 PC；仅图示原记录，无新增仿真。

3.4 等价 RVV 基线与测量方法

已知配置的 RVV 单次点积通过 vwmul.vv 得到宽化乘积，再切换到 e16、清零目的向量，使用 vwredsum.vs 形成 32 位总和，最后恢复入口向量状态。目的清零覆盖完整向量，宽化归约保留已清零的高位。RVV 批处理共享允许共享的配置开销。

两路径计算相同输入和完整输出，检查声明的寄存器副作用、CSR 以及缓冲区护栏。相同镜像使用相同窗口规则、数据和循环次数。正式周期按三组数据、每组四个正式窗口统计，首次及预热另存，独立重放用于一致性检查。

加速比 S = C_RVV / C_vdot。相对旧版的周期降低率 R = (C_old − C_new) / C_old。

在固定香山实现中，fence 与 CSR 读组成的测量边界经实际入队、采样、退休记录核验。每窗校验四条 CSR 指令的 PC、机器字和 ROB 身份，cycle 差与全核 tick 差对应，instret 差与实际退休/ELF 展开计数一致。绝对计数器按整数解析。原记录中的首次、预热和失败样本完整保存。

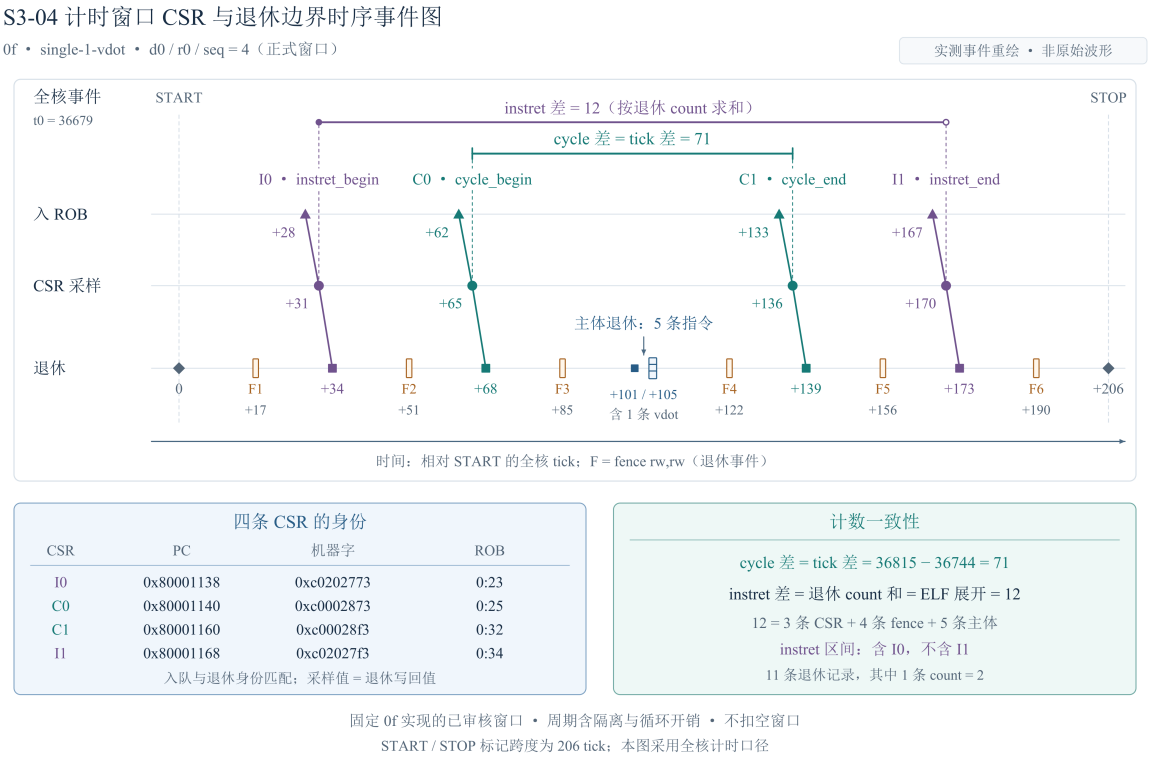


图 3-4　计时窗口 CSR 与退休边界

注：本图为既有观察器事件的时序重绘，并非原始波形截图。所示窗口 cycle 差为 71、instret 差为 12；入 ROB、CSR 取样与退休事件分别标示。

来源与范围：用户提供示意图；依据：team/spec/measurement.md 及固定 T13/O2 观察器窗口证据。版本范围：按实际审核过的窗口绘制；不以局部 FU 周期替代全核计时。

3.5　硬件微基准结果

下表来自 0f 同镜像对照合同，均为正式窗口均值。

表 3-1　硬件微基准结果

场景	vdot cycles	RVV cycles	加速比
单次点积	71.000	85.250	1.200704×
依赖链 256	1857.000	5440.500	2.929725×
四路独立 batch128	221.000	678.000	3.067873×

表 3-1 硬件微基准结果（续）

场景	vdot cycles	RVV cycles	加速比
加载/计算/存储 stream64	1127.500	1133.000	1.004878×

独立批量可以利用 O2 的连续接收能力，依赖链受到前后数据依赖约束，stream 场景还承担访存及循环开销。结果显示收益随工作负载改变。

O1→O2 的独立同镜像比较中，batch128 由 362.5 降至 221 周期，减少 39.0345%。H1 及 0f 修复分别保持各自 96 对正式窗口周期相同。阶段间比较采用各自冻结合同。

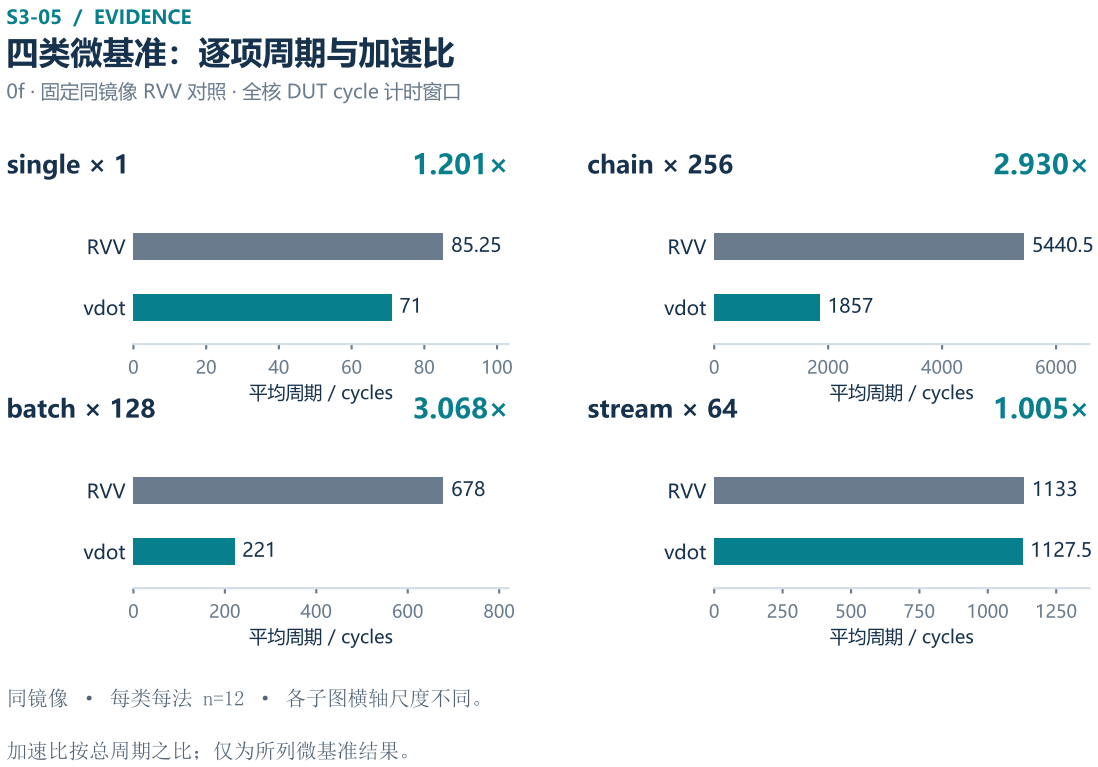


图 3-5 四类微基准周期与加速比

注：四类窗口分别从零比较 RVV 与 vdot 平均周期，每法每类 12 个正式样本；加速比为周期总和之比。各分面使用自己的周期横轴尺度，stream64 仅约 1.005 倍。before/after 配对窗口周期相同。batch 为四路独立、共 128 次点积；修复前后指 8b5024 到 0f 的 DIV 关钟修复，非 vdot/RVV 两方法。

来源与范围：0f r3-div-gate/performance/results/summary.csv；样本与配对核查见同目录 samples.csv、paired-windows.csv 和 verification.json。版本范围：同镜像、固定单核单线程合同的局部微基准；不减空窗口，不等于完整应用或 LLM 收益。

3.6 软件调用与 C8 结果

firstdot 以首段点积初始化累加状态，后续段按约定求和。C8 在已验证 8×32 路径中复用配置，提供普通 C 显式入口和 RVV 回退。输入为对齐的 288 字节 A||x，输出为 8 个

16 字节完整槽。

表 3-2　软件调用与 C8 结果

同合同 8×32 实现	完整调用 cycles
firstdot	282.25
原 C8 配置复用 vdot	273.00
本批最快合格 RVV	299.00

C8 相对 firstdot 减少 3.2772%，相对同批最快 RVV 为 1.095238×。该结果来自三数据集完整矩阵，普通 C 验证、软件包和回接分别补齐调用、ABI、拒错、搬移与最终镜像等价资格。

原合格镜像、复制对照、SDK 接入版 BIN 均为 31440 字节，SHA 相同。PT_LOAD、48 条调用表项与 12 组输入数据一致，ELF 差异位于调试节。627 项保护输入支持复用原功能证据。后续任意新应用按实际输入生命周期和最终链接重新界定资格。

4×8 和 16×64 采用原 firstdot 入口，4×13 选择更快的 RVV。C8 普通调用接口、固定对象及最终镜像身份可由原接入记录核对。

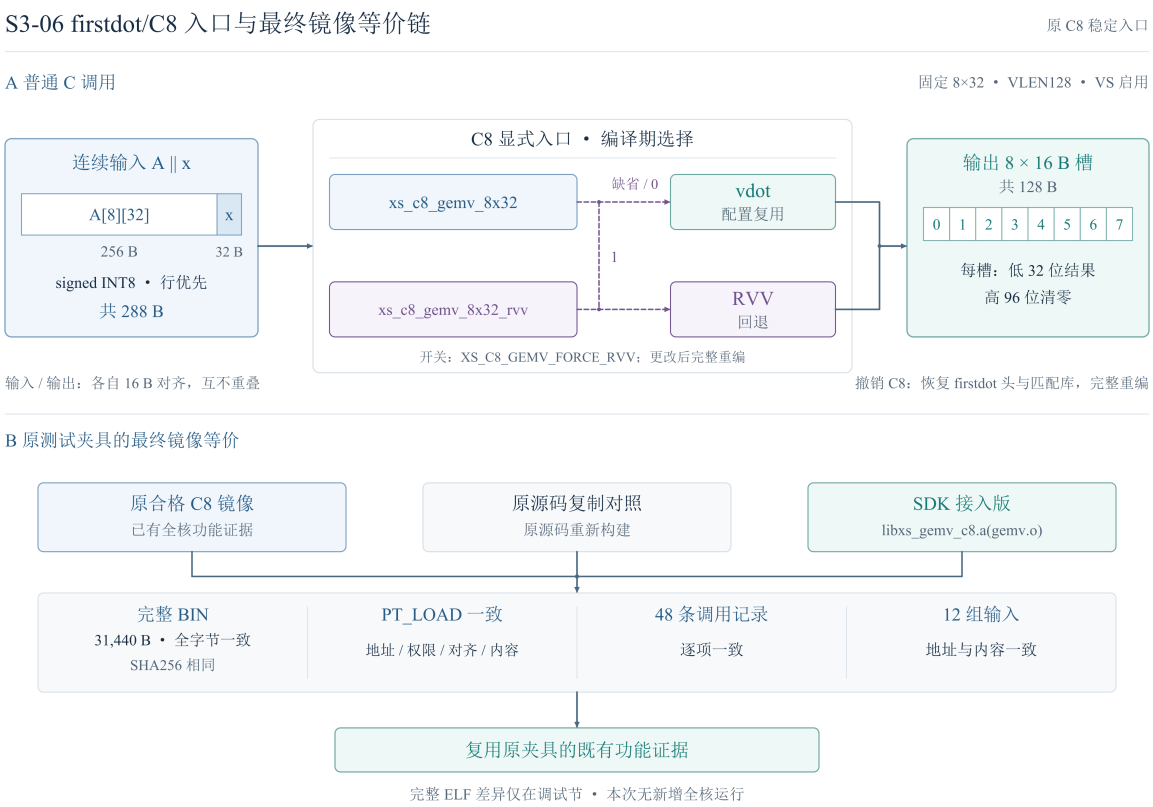


图 3-6　firstdot/C8 入口与最终镜像等价链

注：原 C8 入口读取对齐的 288 字节 A 与 x，产生 8 个 16 字节结果槽，并保留 RVV 回退。原镜像、复制对照与 SDK 接入通过 BIN、PT_LOAD 及调用表进行静态等价核查。

来源与范围：用户提供示意图；依据：C8 CONTRACT、relink RESULT 和对应冻结 manifest。版本范围：原 C8 稳定入口；不引入 split-input 或分段归约候选。

3.7　面积、时序与设计成本

使用 Nangate45 Typical 1.10V/25°C，Yosys0.9、ABC1.01 及 OpenSTA2.0.17。主档周期 1ns、负载 5fF，采用固定线负载模型，另有 0.5ns、2ns 及 1fF、20fF 敏感性档。

表 3-3　面积、时序与设计成本

指标	FU	完整 VFEX4 父级
标准单元面积 μm²	17,298.246	284,614.946
nominal r2r 到达 ns	1.163404	3.261218
nominal 最差 setup 裕量 ns	−0.268520	−2.393998

父级面积包含 FU 及 DIV。负裕量表明 1ns 参考目标仍有时序违例，整核 Fmax、功耗和布线后时序为待测。H1 相对 O2 独立 FU 面积降低 2.33386%，父级降低 0.137349%，同时保留部分路径退化。0f 的取消关钟修复以正确性为依据，其周期不退化与成本变化分别留档。

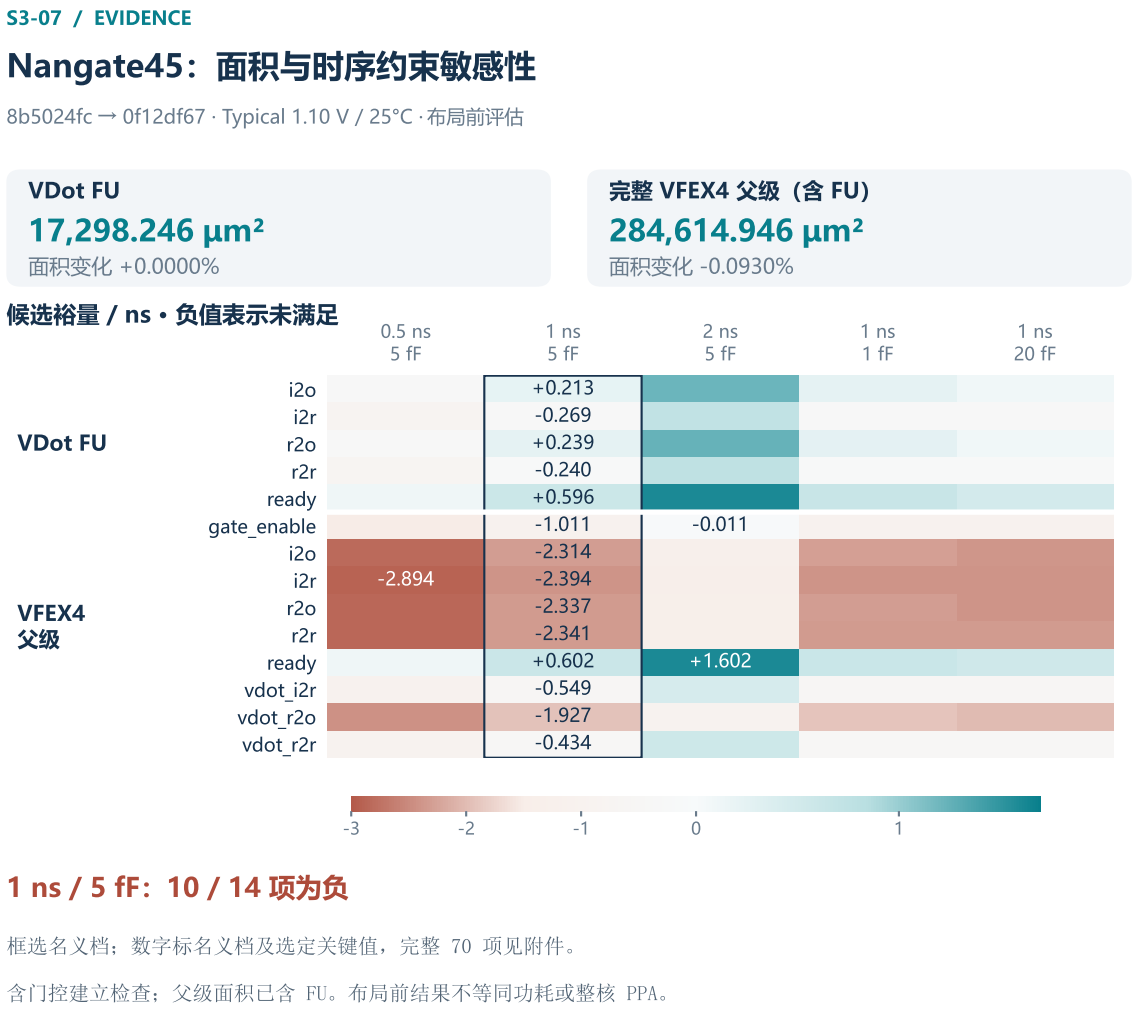


图 3-7　Nangate45 面积与五档 STA 报告证据

注：面积卡片按 8b5024fc 到 0f12df67 比较；父级包含 FU，不能相加。热图以色阶展示五档 70 项时序裕量（包含门控建立检查），数字标注 nominal 及选定关键值；完整 70 项数值见附件。框选 nominal 1ns/5fF 档，其 14 类检查有 10 类负值。I/R/O 分别为输入、寄存器、输出；完整起止点和精度保留于附件。

来源与范围：r3-div-gate/cost/results/summary.json 与 timing.csv；源身份、SHA 及完整数值见 figures/evidence-20261105/figure-data.json。版本范围：Nangate45 Typical 1.10V/25°C，布局前理想时钟/线负载；没有整核 PPA、布线后时序、功耗或 Fmax 结果。

3.8　优化探索及最终取舍

表 3-4　优化探索及最终取舍

探索	结果	最终处理
P1 / P1-R1 输入分离	锚点超出 1% 门槛	HOLD
P4 一拍两槽	成本原 47 项中 6 项失败	NO-GO
P4 唯一修订	47 项中 10 项失败	NO-GO

表 3-4 优化探索及最终取舍（续）

探索	结果	最终处理
C8 分段归约	同轮观测 273.75→273 周期，锚点及 3% 收益未过	关闭该候选
P3 FP32→QKV	正常校准在限额内取得前缀，0 完整计时窗，7200 秒完整上界未闭合	暂存 HOLD

上述试验支持最终选型，正式性能表使用已获资格的稳定结果。参数扩展提供 INT16 独立单元证据，当前工程的应用演示范围为点积微基准及固定 GEMV 调用。

3.9 结论

本项目完成自定义指令硬件接入、独立 NEMU 扩展、分层功能验证及等价 RVV 周期分析。O2 提升独立吞吐，H1 降低结果状态成本，0f 处理 DIV 取消关钟，firstdot 与 C8 改善固定软件调用。工程通过固定源、镜像、输入及原始证据绑定实现复核，材料中如实列出失败、时序违例和验证边界。

NEMU 补丁说明 • 测量合同 • 0f 性能与成本 • 版本数据对照

3.10 最终版本与性能证据身份

- 资料与稳定代码发布提交为 eca41d7cd959f5f3c4f240e382919c492d1ccb96。
- 生产硬件提交为 0f12df675378556670afd0dd2153f47fcb330610。
- clean-02 EMU 的 SHA256 为 acf672eb39f836da4000b253849e3214089e9c8e5198060a57d2cc6bbd393da9。
- reference 的 SHA256 为 62b1707c52a4c323468481e0c1863b79f244d9a3fd54aeb7b7d2ecb5f01a35ca。
- C8 最终 BIN 的 SHA256 为 56805506eb98b68269c73a3645d8f75dff73b697e30dec5f93ccc4716089ad34。

本稿保留最终版本、同合同测量和各层原始证据的关系，报告中的周期收益由已有实验支持。

3.11 工程实现与验证方法补充

3.11.1 架构参考模型的独立性

NEMU 参考实现从架构指令字、源向量和向量配置计算结果，按软件顺序完成八项有符号乘积求和。RTL 以并行乘法、归约和寄存状态实现相同语义。两者采用不同内部组织，可以减少同一结构错误在参考和实现中同步出现的风险。

参考模型必须独立确定目的结果、配置合法性和异常行为。它不得读取 DUT 输出反推正确值。平台适配可对齐架构允许的实现选择，例如固定平台的 xTVAL 规则，但应通过指令和架构状态描述条件，保留其他异常和目的状态检查。

reference 动态库、standalone 可执行模型和故障注入参考库分别具有不同用途。最终差分使用已验收 reference 的真实二进制及来源，负控只对约定位置注入错误。仅凭相同文件名或基线提交，无法确定实际加载库是否相同，因此每次记录包含完整路径和 SHA。

3.11.2 从全核日志判断完成状态

正常程序的有效证据至少包含实际启用 DiffTest、约定测试数量和顺序、完整功能检查、GOOD TRAP 及一致退出码。一个 wrapper 返回 0，但内部日志在 summary 前截断时，仍可能属于中断或部分执行。项目保留历史失败，并在组合验收中明确选择哪些已完整闭合的记录。

负控的完成条件与正常任务不同。bit100 错误应绑定实际 vdot 的 PC、机器字和目的寄存器，并出现预期差分。若只观察到注入标记而未观察到差异和终态，就只能说明执行到了注入附近。恢复正常库后，还需要同镜像正常完成，排除程序或环境本身已损坏。

3.11.3 为何要审计计时边界

乱序处理器可能在较早指令未完全提交时执行某些只读 CSR。若直接把两次 rdcycle 包围的 C 函数耗时理解为严格串行范围，读数可能受到指令重排、边界等待和调用历史影响。本项目在固定香山实现上使用 fence 构建边界，并通过观察器核对实际分派、采样和退休。

四条 CSR 分别承担 cycle 和 instret 的起止采样。cycle 差需要与全核 tick 差一致，instret 差需要与真实退休及有限循环展开一致。对大于 2 的 53 次方的绝对计数，先解析为整数再作差，避免浮点格式损失低位。UART 和完整状态比较按既定计划安排在窗口外，窗口内工作量由 ELF 和调用计划确定。

屏障和循环开销保留在两个实现的共同窗口中。小任务的固定开销占比更大，因此单次点积 71 周期不能直接解释为 FU 内部延迟。空窗口和整数/除法校准用于核对边界

行为，报告遵循原合同统计完整窗口。

3.11.4 配对样本和加速比的含义

给定同一合同下的 RVV 周期 C_RVV 和 vdot 周期 C_vdot，配对加速比为 C_RVV/C_vdot。周期降低率与加速比之间存在换算关系，降低 39.0345% 对应旧新周期比约 1.6403，而相对另一条 RVV 基线的 3.067873 倍使用不同分母。表格必须同时写出原始周期和比较对象。

每场景三个数据集各有四个正式窗口，首次和预热保留但不混入正式均值。重放核查同镜像、同序号和计数结果是否一致，不作为额外独立样本。跨合同的不同链接地址、调用入口和输入生命周期可改变测量上下文，不能把两张表中最好的一格随意拼成新结论。

3.11.5 普通 C 调用和软件合同

C8 使用编译期别名连接已验证的完整对象。调用者提供连续的 A||x，输出按 8 个 16 字节槽保存。接口不会自动检测 CPU 是否支持自定义指令，调用环境、VS 状态、VLEN128、对齐和生命周期由调用者负责。

C8 返回 VL8、vtype0、vstart0、vcsr7，原向量配置不自动恢复。将该函数嵌入更大的软件时，要在调用合同中明确允许改变的状态。若新增 dispatcher、CSR 包装、对象裁剪或不同最终链接，即使底层函数体未变，也需要重新判断受影响的功能和性能范围。

firstdot 与 C8 在 8×32 同批中的 282.25 和 273 周期说明配置复用产生 3.2772% 的改善。最快合格 RVV299 周期用于该批分母。其他尺寸没有全部采用 C8，最终接口选择由已验证数据决定。

3.11.6 对象等价和镜像等价的层次

汇编源相同并不自动保证目标对象相同，工具链、参数和重定位都可能改变机器码。对象相同也不自动保证完整 ELF 相同，启动、库、数据和地址布局仍会变化。因此包化接入分为对象、可重定位链接、最终装载内容和真实调用几个层次。

C8 接入时真正从静态库抽取 gemv.o，链接映射和依赖文件证明没有绕过 SDK。最终比较先检查 BIN、PT_LOAD、BSS、符号、48 项调用表及 12 组数据，再使用去调试 ELF 一致作为独立补充。这样既解释了调试路径变化，也保留了运行内容等价的严格依据。

3.11.7 面积和时序结果怎样支持取舍

综合面积是固定 Liberty 库下实际映射单元面积之和。FU 包含点积和接口状态，VFEX4 父级还包含 DIV 及共享控制。父级已包含 FU，二者的数值不能相加。不同工艺库的面积变化应单独表述，当前最终成绩采用 Nangate45 同条件数据。

STA 的到达时间、要求时间和裕量共同描述路径。到达较小不一定意味着全设计最差裕量改善，因为起终点、时钟预算及输出延迟不同。最终报告同时列出整体、VDot 子集、ready 和门控使能，保留负裕量和端点切换。

五档敏感性保持同一映射网表，改变周期或负载检验结果对约束的响应。它不能替代多工艺角、布线寄生、hold 和物理实现检查。1ns 主档仍有违例，后续若开展时序优化，应保持原对照并说明新约束。

3.11.8 优化失败也需要可复核结论

P1 与 R1 有局部收益信号，但锚点超过冻结 1% 门槛。P4 一拍候选减少内部等待，却在父级 ready 等路径上产生代价。分段归约减少配置指令，但同轮点积完成进度并未相应提前，收益未过预设门槛。这些结果都支持保留稳定版，而不是继续扩大未闭合候选。

P3 的情况属于预算 HOLD。正常校准取得首窗前缀但 0 完整窗，完整 14 调用的保守上界没有闭合。原供体收益和前缀吞吐不能直接成为 0f 的完整应用成绩。该方向保留证据并暂存，正式表采用已有合格结果。

3.11.9 同机干净复现和最终工程验收

干净复现从独立源码、依赖和构建目录恢复，核对 EMU、reference、AM/klib 及应用身份，并逐窗比较结果。firstdot 的 768 窗口与 96 普通 C 输出已复现一致。该过程证明固定环境和输入下的可恢复性，执行者与缓存复用情况仍在记录中说明。

R5 总验收采用原 13 项和补充 1 项组合，保留原宿主墙钟审计失败。通过单调时钟补证收口所选占用审计后，系统 UTC 稳定性仍单列限制。工程版本、功能资格、性能数字、成本结果及人工复核分别表达，使最终交付可以逐项核对。

3.11.10 独立参考模型的输入与输出合同

第三阶段需要把硬件执行结果放在独立于 RTL 的架构坐标中检验。参考模型只读取当前指令编码、架构源寄存器内容、向量配置和异常上下文，不读取硬件的乘积树、槽位或 ready 信号。其计算步骤按八个有符号字节分别取值、相乘、以足够宽度求和，再构造完整一百二十八位目的值。这样，即使 RTL 内部换成另一种归约树或流水安排，参考仍依据相同的软件可见合同判断结果。

独立性还包括错误路径。非法编码或非法配置要根据指令和架构状态判定，不能在参考侧直接复制 DUT 报告的异常种类。若平台对某一类异常附加的 xTVAL 有固定实现规则，适配必须只针对已明确的条件，同时继续比较异常 PC、目的状态和其余异常字段。扩大豁免范围虽然会减少差分噪声，却也可能遮住真正的译码或写回缺陷。每一条适配都应在报告中给出触发条件和允许变化的字段。

参考库文件名无法代表其内容。构建时的编译参数、源码状态或上一次产物残留，都可能让同名库加载不同模型。因此正常、负控和恢复三类运行分别记录实际加载路径与校验摘要。负控库应只在约定目标上改变一个可识别结果位，并证明差分在预期的 PC 与指令字附近触发；恢复运行再回到原始库，验证相同镜像可以正常收尾。这一组实验同时约束硬件结论和检查器本身。

3.11.11 差分测试的结果如何判读

差分日志需要按事件链阅读：程序启动、差分启用、目标指令执行、参考比较、程序终止。仅有 GOOD TRAP 不保证每条目标指令真正进入差分，也不保证加载了目标参考库。仅有参考注入标记不保证比较器检出了变化。有效记录还必须包含完整的测试计数、预期终态和进程退出状态。若日志截断于摘要之前，应标成未闭合，不能从已经出现的若干通过行推断全程成功。

负控选择高位结果位有明确目的。点积的合法结果只占低三十二位，高九十六位应为零；对高位注入错误时，若差分仍通过，说明比较可能只看了低位或错误地复用了硬件输出。测试同时记录目的寄存器和实际机器指令，确保故障发生在本次点积上，而非另一个向量操作。差分失败应发生在预期的检查点，其他启动或环境错误引起的失败不构成有效负控。

恢复实验使用同一程序与输入，排除故障注入修改了持久状态、镜像或依赖文件的可能。正常运行和负控运行的差异应尽量只限于参考库内容。若重新链接程序、改变启动参数或切换依赖版本，就不能把两次结果直接视为单变量实验。通过这组约束，报告能够说明差分检查覆盖了特定结果位和特定执行位置，而不会把它夸大为对所有硬件故障的证明。

3.11.12 计数窗口的工程定义

性能实验的基本单位是一个明确的指令窗口，而不是主机墙钟时间。窗口从哪条边界指令之后开始、在哪条边界指令之前结束，决定了其中包含哪些初始化、循环、加载和存储开销。报告给出的周期与退休指令数必须使用同一对边界；否则即便两个数字都正确，也不能组成可解释的性能样本。窗口描述还应指明输入数据的生命周期和结果校验位置，让后续重放知道哪些工作发生在计数内。

在乱序核心上，读取 cycle 与读取 instret 的四条 CSR 指令可能与邻近计算重叠。项目使用固定平台上的屏障来约束窗口，并以观察器核对实际分派和退休。屏障属于测量

合同的一部分，其开销保留在被比较的两种实现中。不能在结果出来以后为了更漂亮的数字只从一个实现扣除屏障开销。小任务尤其容易受到固定开销影响，因此报告把完整窗口周期称为应用级样本，而不将其解释为功能单元的裸延迟。

原始绝对计数可能远高于普通浮点数能够精确表示整数的范围。读取日志时先按整数字符串解析绝对 cycle 和 instret，再作差和汇总；若先转换成浮点，再相减，低位会在输入阶段丢失。重放核查也应以整数差值为准，并将原始起止计数保留在可查的证据中。这个处理属于测量正确性，而非统计呈现细节。

cycle 差与全核 tick 差的一致性，是对窗口边界的独立交叉检查；instret 差与实际退休范围的一致性，是对调用计划的交叉检查。两者不应被互相替代。若 cycle 一致而退休数不一致，可能存在函数版本、循环展开或路径变化；若退休数一致而 cycle 不一致，则需要检查反压、缓存状态或计数边界。报告在解释原因前先保留原始差异，避免用未经观察的微架构故事覆盖数据。

3.11.13 配对比较和数据汇总

每一个加速比都要求一对同合同样本：相同数据、相同调用次数、相同镜像边界及相同计数定义。先计算或列出每一对原始周期，再按预先规定的方法汇总，避免把一种实现的最好窗口与另一种实现的最差窗口相除。首次运行与预热用于检查环境稳定性，正式窗口进入均值；重放用于验证可重复性，不作为额外独立样本增加权重。

“周期降低百分比”和“加速倍数”采用不同公式。前者用旧周期减新周期后除以旧周期，后者用旧周期除以新周期。若比较对象从 O1 变为 RVV 基线，数值即使在同一场景也会变化。所有图表应同时标出分子、分母、周期单位和工作量，使读者可以用原始数字复算结论。表格之间若链接地址、输入生命周期或调用入口不同，应标成不同合同，不能跨表挑选局部最佳值再组合成新的总体收益。

平均值本身不显示离散程度。审查时需要保留各正式窗口的独立值、顺序和数据集标识，确认均值不是由缺失窗口或重复窗口形成。若样本之间存在顺序效应或明显异常，应先检查重放、日志和环境，而非自动删除异常点。当前报告的性能结论针对这些固定工作负载、镜像和仿真设置；其可比性来自明确的配对规则，不来自对任意程序的速度保证。

3.11.14 四类工作负载各自回答的问题

单次点积样本能够展示调用、配置和计数边界共同作用下的最短工作量，但固定开销占比最高。即使观察到约七十一周期，也不能把该数值当作乘法树或归约寄存器的响应延迟。单次样本更适合核对功能接入与调用成本，对连续接收能力的判断仍需使用批量场景。解释任何单次优化时，都要先查看是否改变了配置指令或边界指令数量。

独立 batch128 通过四个轮换的目的寄存器组织无真实数据依赖的点积，使两个弹性槽可以连续接收请求。O1 与 O2 同镜像的完整窗口由 362.5 降至 221 周期，降低 39.0345%。

这一结果支持 O2 改善该场景的独立吞吐；复核时还应确认寄存器轮换确实消除了别名相关，且两版数据、循环控制和计数边界一致。依赖链需要等待上一条结果写回，不能直接套用这一收益。

chain256 刻意把上一条结果作为下一条源，测的是反馈路径对整窗吞吐的限制。结果值可能因数据演化趋零，但寄存器依赖仍然存在。完整窗口 1857 周期包含发射等待、读端口、功能单元、写回与下一条指令的可执行时间；两拍算术响应只是其中一环。将两拍乘以 256 得出的 512 拍并非完整窗口预测值，也不能把两者差额全部归因于某个单一模块。

stream64 包含连续的加载、地址更新、计算和存储。vdot 为 1127.5 周期，RVV 为 1133 周期，收益接近 1，说明此合同下缩短点积主体对完整窗口影响较小。该结果不能单独证明瓶颈一定是缓存，也不能外推至所有数据规模。进一步定位需要增加访存等待和功能单元占用观察；目前只能陈述固定工作量的实测差异及其范围。

四类场景共同构成性能解释的边界。单次样本对固定开销敏感，独立批量对连续接受敏感，依赖链对反馈延迟敏感，流式场景对计算之外的工作敏感。相同硬件在不同场景中出现不同收益是预期现象。报告逐项说明工作量和比较对象，避免用表现最好的一类代表全部应用，也避免用表现最弱的一类否定已验证的局部吞吐改善。

3.11.15 软件接口、等价验证与状态影响

C8 接口把已经验证的汇编对象接到普通 C 调用路径，使应用不必直接管理每条自定义指令的发射。调用合同规定输入布局为连续的 A 与 x、输出为八个十六字节槽、对齐及生命周期由调用方保证。平台支持、VS 使能和 VLEN 长度也属于调用前提。接口并不自动探测不支持的处理器，因而移植到新平台前需要重新确认指令编码与向量状态约定。

函数返回时 VL、vtype、vstart 与 vcsr 具有明确的最终值。这些变化可能影响后续向量代码，因此调用方要么接受该状态，要么在外层显式保存并恢复。把保存恢复逻辑加入函数内部会改变指令窗口和性能结果，不能在不重测的情况下把旧成绩移植到新接口。firstdot 与 C8 同批比较的二八二点二五和二七三周期，解释了配置复用的局部价值，但这仍受相同八乘三十二调用合同限制。

软件包化的等价检查分层进行。首先确认静态库实际抽取了预期对象，而非链接到开发目录中另一份同名文件；再确认重定位、符号解析、装载段、数据和调用表与被比较版本一致。调试信息或构建路径变化不一定改变运行内容，因此使用去调试 ELF 和实际 PT_LOAD 内容来区分非运行差异。若 BIN、BSS 或入口发生变化，应按变化影响重新评估，而不能只依赖对象源文件相同这一事实。

输出数据比较覆盖调用的每个槽，并且需要核对槽边界。某些错误不会改变第一项结果，却会把后续结果写到相邻地址；单一标量校验无法发现。输入和输出缓冲区的别名、对齐及有效期也应依照合同处理。当前验收证明冻结的调用方式与已验证汇编对象相接，并不自动证明任意别名模式、任意矩阵尺寸或动态派发策略都获得同等覆盖。

3.11.16 成本分析与后续优化资格

面积来自指定单元库、综合脚本和约束下的映射结果。父级 VFEX4 的面积已包含点积子模块及其他共享逻辑，不能把父级与子模块面积相加作为整机成本。时序则要看具体路径的到达时间、要求时间和裕量；单一最差端点可能随版本切换。若某次修改使点积内部路径缩短，却使父级 ready 或门控路径变长，最终时序结论必须按父级最差路径陈述。

一纳秒主档仍存在负裕量，说明当前映射结果未达到该约束。不能因为某一子集路径为正就称整模块闭合，也不能把敏感性扫描的较宽松档位当作原主档结果。工艺库、时钟周期、负载和报告提取规则都应成对记录。Nangate45 与 ASAP7 属于不同库和映射条件，适合单独研究趋势；在缺少一致的物理和时序前提时，不宜把不同库中的数值作为纯工艺缩放比。

优化候选进入正式成绩前，需要同时通过功能正确性、异常与取消、差分测试、可复现构建和固定约束性能筛选。P1、P3、P4 展示了不同的未闭合方式：有的收益不足预设门槛，有的完整调用窗口没有取得，有的局部加速伴随父级代价。把失败证据保存下来能减少重复试验，也能明确下一轮修订应解决什么问题。候选未取得完整资格时，稳定版仍是报告的正式基准。

3.11.17 复现、限制与交付判断

独立检出从源码、依赖和构建目录重新恢复，可以发现依赖于工作区残留的偶然成功。复现记录应包含源码版本、工具链、构建参数、参考库摘要、应用镜像、随机种子和正式窗口集合。若某项产物不可重新生成，需要明确指出缺失输入和仍可复用的历史证据，而不是把已有日志当作当日执行结果。复现结论针对固定输入和固定环境，不能代替新配置、新工艺角或新应用的验收。

系统 UTC 时间异常与仿真周期计数是不同问题。补充的单调时钟占用审计可以说明选定运行的持续时间，但不能反向修复原始墙钟记录。报告保留原失败项及补证过程，使审阅者能区分运行结果、证据采集质量和最终工程判断。最终验收也因此采用逐项状态：功能、差分、性能、软件接口、成本和复现分别说明，避免一句“全部完成”掩盖仍需后续工作的限制。

第三阶段的总体结论依赖从指令语义到应用窗口的连续证据链。独立模型验证软件可见结果，负控确认关键差分比较有效，计数窗口审计确认周期含义，配对实验限定性能结论，软件对象与镜像等价限定调用接口，综合与 STA 限定物理成本解释。任何一个环节改变，相关结论就需要按影响范围重新验证。保留原始输入、失败记录和版本身份，使未来优化能够在同一合同上推进，而不会靠记忆重建基线。

3.11.18 性能数字的实际使用边界

报告中的每一个周期数都服务于一个明确决策。独立批量的结果回答是否值得保持连续接收能力；依赖链的结果回答反馈路径是否仍是限制；流式场景回答计算优化能否在更完整的软件工作中保留收益。若实际应用的矩阵尺寸、访存模式、配置频率或调用批次与这些场景不同，就需要建立新场景再测，不能把现有倍数按指令条数线性外推。真实应用还会受到缓存、调度和系统活动影响，固定仿真微基准只提供受控的比较起点。

性能图的纵轴和表格的单位也应保持一致。周期均值可以用小数表示，是因为多个整数窗口求平均，而不是某次执行出现半个周期。对单一窗口的结果，应保留整数原值；对组均值，说明参与平均的窗口数和工作量。若两种实现的正式样本数量不等，不能直接比较未经说明的均值。数据图应能回到原始日志，复核每一个点所对应的镜像、场景与样本序号。

对于一纳秒主档负裕量，工程上的含义是当前综合与静态时序条件下仍有未闭合路径。它不会自动推翻功能仿真中已经通过的指令语义，但会限制将设计称为满足该目标频率的交付。下一轮时序优化应从具体端点、逻辑层级和扇出入手，并在相同库、相同约束下重跑完整父级。若目标频率、输出负载或映射库改变，报告应重新列出比较基准，而不是把新结果覆盖旧记录。

把软件接口纳入性能复验时，还要判断函数前后的状态处理是否属于用户实际要付出的成本。若应用每次调用前都重新设置 VL，窗口内应包含该操作；若一批调用共享配置，应该在批量合同中清楚说明配置移出窗口的条件。两种合同均有实际用途，却对应不同的性能问题。firstdot 与 C8 的比较因此固定在同一批次和同一调用约束，避免把一次性配置节省误写成每条点积的算术速度变化。

最后，报告的优化建议应保持与证据强度一致。可以依据已通过的同镜像对照选择稳定版本，并把未过门槛的候选留作下一轮研究；可以依据负裕量列出需要进一步收敛的父级路径；可以依据流式场景的有限收益提出增加访存观察。当前材料不能证明这些后续措施一定达到目标，因此建议与已完成结论分别列示。这样的区分让项目既能继续前进，也能保留对已验证结果的准确描述。

3.11.19 从异常样本到结论修订

第三阶段的性能与功能证据来自不同运行，异常样本不能简单地从平均值中删除。若某个正式窗口周期明显偏离同组其他窗口，应先检查该窗口的起止计数、数据集、镜像摘要和全核退出状态，再核查是否发生重放差异。确认属于采集错误时，应保留原始样本和排除理由，并重新执行预定数量的正式窗口；若属于真实的体系结构或资源行为，就应保留并解释其对均值和离散程度的影响。事后只保留有利样本会破坏配对比较的意义。

功能差分出现异常时，应先确认 reference 和 DUT 使用同一指令字及配置状态。若差异落在目的高位，可沿完整结果布局检查硬件清零和参考模型写回；若差异落在异常

字段，应核对已约定的平台适配条件；若失败发生在目标指令执行之前，则优先检查镜像与启动路径。定位顺序与故障发生位置保持一致，能够避免为无关问题修改模型或放宽比较器。

成本结果也需要版本配对。面积报告与 STA 报告只有在同一源码、相同开关及相同综合网表下才可共同描述一个候选；若面积来自一轮映射、时序来自另一轮映射，两者不能组成同一个设计点。父级最差路径随逻辑变化发生端点转移时，应同时保留旧端点和新端点的信息，说明优化是否只是将压力转移。后续若引入物理布线或其他工艺角，应将新结果作为新的验证层级，而非覆盖当前库下已复核的数值。

3.12 周期测量边界与统计合同

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/spec/measurement.md

3.12.1 两条路径必须计算同一结果

低 8 个 signed int8 元素分别相乘，在足够宽的有符号整数中求和；目的向量低 32 位存二补码结果，高 96 位清零。检查完整 128 位，不用截断标量或摘要替代。vdot 不修改其他向量寄存器；RVV 只允许破坏事先声明的 scratch。

复用来源为 zufan@f090665130107282be0990a7664aebf26e491b87。原文件与 SHA-256 位于 team/references/zufan-t13/，本轮重新构建和运行，不继承对方的 PASS 为本轮验收结果。

表 3-5 两条路径必须计算同一结果

RVV 路径	点积主体指令数	向量 scratch	入口与出口
已知配置单次	5	v2、v3	e8/m1/VL8、tu/mu；退出恢复相同配置
通用配置单次	7	v2、v3，另用 t3/t4 保存 CSR	保留原 VL/vtype，测试四种 tail/mask policy
独立四路批处理	14 / 4 个点积	v16–v23	e8/m1/VL8、tu/mu；退出恢复相同配置

已知配置序列：vwmul.vv → 设置 e16/m1/VL8 → vmv.v.i vd,0 → vwredsum.vs → 恢复 e8/m1/VL8。e16 下 8 个元素正好清零整个目的寄存器；宽化归约使用 tu，保留已经清

零的高位。必须先乘后清目的寄存器，以支持源/目的别名。通用序列增加 csrr vl/vtype，末尾用 vsetvl 恢复。四路批处理共享两次 vtype 设置，不人为放大基线开销。

每次调用保存并比较全部 32 个向量寄存器，按声明排除 RVV scratch；检查 VL、vtype、vstart、vxrm、vxsat、vcsr、实际循环次数以及缓冲区前后 canary。访存组逐个比较全部 128 位输出，并检查未使用输出空间。C 判定器使用真实源文件进行原生及 UBSan 位翻转测试。

3.12.2 为什么不能直接使用一对 rdcycle

DecodeUnit.scala 的普通 CSR 表带 waitForward/blockBackward；但 Rename.scala 282–299 行会解除大部分只读 CSR 的这些限制。本轮初始观测确实看到 cycle/instret 在 ROB 未清空时进入，标志为零。因此不能由解码表单独推导 rdcycle 串行化，更不能把旧无校准窗口当作指令延迟。

本轮在**软件测量边界**加入 fence rw,rw，不改变生产核执行规则。该固定版本的 fence 使用等待旧指令、阻止后续分派和 flush 的实现，作用必须通过实际日志确认。这是针对固定香山实现的实验方法，不是声称 RISC-V ISA 的普通内存 fence 在任意处理器上都能序列化计算和 CSR。

窗口结构如下（所有实验分支相同）：

```
fence; START; fence
read instret_begin; fence
read cycle_begin; fence
执行显式汇编主体（含循环控制）
fence; read cycle_end; fence
read instret_end; fence
STOP; fence
保存结果；完整状态检查；打印
```

观察器以编译开关 VDOT_T13_TRACE=1 启用，仅打印 CSR 实际采样、计数器入 ROB 和窗口内退休信息。与 T12 观察器一起构建，不改数据通路、频率、复位、忙闲接口或 NEMU 计时。硬件源码提交为 4460300e9557ad20ac152fa67b1827c5d62802dd；工作负载另列实际构建提交，不能用后来的文档提交冒充重新构建。

每个窗口必须同时满足：

1. 精确匹配四条计数器指令的 PC、机器码、ROB 身份与顺序。

2. CSR 读值与对应退休写回数据相等，采样位于入队和退休之间。

3. 每条计数器的前一 fence 已退休，后一 fence 在计数器退休后才退休。

4. cycle 差值等于硬件全核采样 tick 差值。

5. instret 差值等于真实退休计数，也等于从 ELF 机器码展开有限循环得到的指令数。

6. 完整窗口及 UART 记录数量、顺序与 manifest 一致；真实 exit0、DiffTest 启用、完整成功 summary 后 GOOD TRAP。

绝对计数器可能大于 2^53，所有解析与差值计算使用 Python 整数。禁止经过 JavaScript Number 或浮点再保存绝对值。FU 自身时钟存在门控，使用已验收 T12 的 parent VFEX4 accept 逐事务归一化，不能直接拿 FU 局部 tick 减 ROB tick。

空窗口、8 条整数指令、32 条依赖除法用于校准。**不从测量结果直接减掉空窗口开销**；小样本的屏障、取指和循环开销占比必须说明。

3.12.3 工作负载矩阵

表 3-6 工作负载矩阵

场景	n	点积数	含义
single	1	1	已知配置完整调用窗口
chain	16、64、256	n	目的向量作为下一次源，观察依赖限制
batch	1、8、32	4n	每轮四个独立目的，比较共享配置的 RVV 批处理
stream	1、16、64	n	两个 128 位加载、点积、128 位存储以及指针更新
generic	1、16、64	n	原 vtype=192 的 CSR 保留对照

链式实验的反馈值是上一结果的完整向量表示，后续数值可能收敛到零。它用于固定依赖关系的微基准，不代表真实模型的数值分布。批处理四个目的独立且共享输入，用于消除人为增加的配置开销；不将其宣传为任意矩阵乘法性能。

三组输入包括正极值、负极值和混合符号；高半区设置非零哨兵，访存组加入固定种子的随机数据。功能矩阵使用原始 22 组定向和前 32 组随机数据，轮换七种别名与四种 policy；这 54 对不是全部笛卡尔积。另用固定 NEMU 对全部 10,022 组数据重新检查精简基线，明确与全核执行层分开计数。

每个配置同一镜像内运行 6 轮配对：首次样本、预热、4 轮正式样本；顺序按 AB/BA 交替。首次不是人为清空缓存后的“冷缓存”；预热也不保证全部微架构状态完全相同。每种配置单独分片，两个实现的入口均 64 字节对齐；代码长度、地址和寄存器压力的自然差异保留并记录。

每个性能镜像以固定 seed=1 在全新 EMU 中运行三次。重放比较同一镜像、同一 seq 的周期和退休指令数；用于确定性复核，不作为独立统计样本，不给 p 值或统计显著性。

宿主允许 8 个进程并行；每个 DUT 仍是同一个单核、EMU_THREADS=1 配置。宿主秒数仅用于安排任务，不参与加速比。

3.12.4 汇总与结论边界

保留全部首次、预热、正式窗口。逐配置、逐数据集报告原始周期、周期/点积、指令数、正式样本 min/median/max；配对加速比为 RVV cycles / vdot cycles，减少比例为 1 - vdot / RVV。不以最优个例冒充总体，不在不同场景、不同 n 或不同数据集之间错配。若 vdot 变慢，照实记录。

报告层次区分：局部 FU accept→response；完整序列的窗口；独立批处理的点积吞吐；包含加载/存储的循环。FU 局部 E3/II4 不是全核指令周期。端到端窗口含相同的隔离与循环开销，不等同于纯算术单元延迟。

本轮不推断 Fmax、面积、功耗、PPA、实际 LLM 加速或官方比赛得分。没有 STA/综合结果，不把周期减少转换成频率提升。正式规范及队友人工复核仍按团队流程处理。

3.13 O1 与 O2 同镜像验证过程

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/optimizations/o2/performance/README.md

3.13.1 本次实测范围

两模式使用原 16 份冻结镜像，实际新跑 36 项（32 次首次镜像运行＋4 次代表 batch 镜像重放），478 个校验窗口，192 条正式性能窗口。每场景每模式每方法为 3 数据集 ×4measured ＝ 12 条样本；first/warmup 和重放完整保留，重放不增加独立样本数。

周期降低按 1-sum(O2 custom)/sum(O1 custom)，加速比按该模式本次实测 sum(RVV)/sum(custom)。每组逐 dataset/repeat/seq/镜像配对，机器码、工作量及退休指令一致；未扣循环/屏障或空窗开销。历史 362.50/678 没有作为本次统计输入。single 不是 FU 固有延迟，batch 四路同输入独立目的不代表任意 GEMV 收益。

每个作业通过真实 DiffTest、约定输出/128 位高位/非 scratch 寄存器/CSR/哨兵、计数器与全局周期窗口核验，以及各自 O1/O2 完整协议和有限排空门禁；致命错误文本显式拒绝。冻结 manifest 明确允许各内核的 scratch 寄存器范围，scratch_control 也须通过；不把允许破坏的 scratch 当成全部 32 寄存器逐位相同。额外 4 次 batch 重放逐 seq 周期与退休数一致。30 项主机测试通过；全部原始 stdout/stderr、真实退出码、版本、输入和检查器摘要保留。

3.13.2 硬件与工具身份

表 3-7 硬件与工具身份

项目	O1	O2
实测 EMU SHA-256	6699225b76ceae47906074102 6683bb03de069c1e10a4a94e4 63304ba1e322d0	ac0123280bf6c742d27a7650505bc 1d91db350d1519806edfd0be9c7ea 208a57
构建源码提交	a239dace720b32216e83fb5a5 14930d334c69a26	b218cbc6e6b135e4ff219a902a9284 a59d3c82c9
来源	本轮从相同硬件输入重建	复用 G3 已验收固定 EMU
O1_HANDOFF / O2_ELASTIC	1/0	0/1
O1_TRACE/O2_TRACE	1/0	0/1

429 项硬件源码与 21236 项依赖逐字节相同，MinimalConfig/单核/VLEN128/单线程 EMU；共同 FUSED_INPUT1、T12_TRACE0、T13_TRACE1。各自合法协议观察器只提供只读观察，共同 T13 提供全局周期/退休计数。

路径归一化后的实际 Verilator/C++ 命令、七类工具版本日志一致；实际 LLVM19.1.7 编译器由构建过程/proc 记录并核对。j2、12G JVM、O3/O0；最多两个单线程 EMU。新 O1 采用外置产品目录，原 G3 的 1687 份非对象生成文件已恢复并验证原始 SHA，旧 O2 EMU 不覆盖。硬件构建身份、运行工具身份与后续文档/封存提交分别记录，不以文档提交冒称重构建。

3.13.3 交付复核

Windows 原生 Python 3.13.2 全量离线复算通过，覆盖 36 项作业、478 个窗口、192 条正式性能窗口及 4 项重放。见原生复算回执。它复核封存记录，不另跑 EMU，也不替代 Linux 构建来源审核。

交付时补齐了原冻结功能生成器依赖的两份模型语料，逐字节取自运行工具提交 63b95a5 并绑定 Git blob 及 SHA-256；原镜像、测量日志和 646 份主封存 payload 均未改变。原 30 项主机测试及新增 3 项语料绑定检查共 33 项通过；首次封装复算失败和修复后成功记录均保留。Windows 请用 Python UTF-8 模式执行：

python -X utf8 -B performance/portable.py –archive evidence/20261019-performance-ab –campaign campaign-01 –output /absolute/new/portable-verification.json

上述相对路径适用于 Windows 交付目录；仓库内对应 team/optimizations/o2 下的路径。

3.14 0f 硬件性能与五档成本详细记录

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/optimizations/o2/r3-div-gate/PERFORMANCE_COST_RESULT.md

3.14.1 受测身份与公平条件

表 3-8 受测身份与公平条件

对象	修复前	修复后
实现	O2 两槽＋ H1 低 32 位保存	O2 ＋ H1 ＋ DIV 取消关钟修复
硬件源码	8b5024fcbd96986871cc40ce8c9fb71a67c495eb	0f12df675378556670afd0dd2153f47fcb330610
EMU SHA256	e37b3c267cae29036bda0b6952a17db2523ca0b3a573d45281341d4207df8747	2d1db77cc407486a2387a4a69da2498b68d44980239025eebcc02370c02826e8

硬件 429 份 Git 源码、21236 份依赖、每版 1672 份生成产物、实际编译命令和七类工具版本均经核验。生产源码仅 ExeUnit.scala 不同。

同一 MinimalConfig/单核单线程/O2 与 T13 观察配置、128MB、seed=1、正常 NEMU 和 flash；208 份冻结软件载荷不变。

正常 NEMU SHA256：efbed0774addc27eac38af1a19ab5d9c9dd5f179b5015f25ae7b80b966185bd4。

完整生成物还包含已审查的 Git 标识、只读诊断日志、顶层注释及标量 FDiv 公共子表达式内联差异；没有声称全设计形式等价。

3.14.2 同镜像周期

表 3-9 同镜像周期

场景	修复前 vdot 均值	修复后 vdot 均值	两版 RVV 均值	两版 RVV/vdot
单次点积 1	71.000	71.000	85.250	1.200704×
依赖链 256	1857.000	1857.000	5440.500	2.929725×

表 3-9 同镜像周期（续）

场景	修复前 vdot 均值	修复后 vdot 均值	两版 RVV 均值	两版 RVV/vdot
四路独立批量 128	221.000	221.000	678.000	3.067873×
加载/计算/存储 64	1127.500	1127.500	1133.000	1.004878×

单位均为 DUT 全局 cycles。每场景三组数据 × 四轮，vdot 和 RVV 分别配对；192 正式窗口形成 96 对，每对差值均为 0，退休指令与工作量相同。

共 478 个校验窗口，保留 first/warmup；四次额外 batch 重放完全一致，不加入正式样本。36 项实际 EMU 均通过完整 128 位结果、DiffTest、GOOD TRAP、窗口计数和 O2 生命周期/排空检查。

加速比为本轮各场景 sum(RVV cycles)/sum(vdot cycles)，不扣计时开销，不用宿主时间。**本修复没有提高 cycles 加速比，也没有使这些固定微基准退化。**

详见逐窗差值、全部样本、复算回执。

3.14.3 同约束综合与面积

修复前复用已封存 H1 的 205 份完整原始生成/综合/STA 载荷并逐字节核验；修复后实际重新生成、综合和 STA，未拿旧值冒充新运行。

两边使用相同完整生产接口、相同未修改 O4 脚本、Yosys0.9/ABC1.01/OpenSTA2.0.17 及其动态库、Nangate45 typical 25°C/1.1V 库。

ABC 目标 1000ps、BUF_X1 驱动、5fF 映射负载。面积是 Liberty 单元面积之和，单位 μm²，非芯片布局面积。

表 3-10 同约束综合与面积

边界	修复前面积	修复后面积	增量	增量比例	DFF 数量	ICG 数量
fu	17298.246	17298.246	+0.000	+0.000000%	1742→1742	0→0
parent	284879.882	284614.946	-264.936	-0.092999%	17171→17172	3→3

父级映射 DFF 增加 1 个，已回溯到既有整数 DIV 的 Bypass_final_rem_plus_d_reg[2]：旧网表该位无驱动和消费者，新网表保留 DFF_X1。两版对应 DIV 生成 RTL 相同，没有新增 RTL 寄存器声明或 VDot 状态位；这是整体综合优化/裁剪结果的差异，不能把总面积变化简单等同于新增几个门的局部成本。详情在归档 dff-review.json。

两版 VDot 结果仍保存 32 位，公开输出 128 位高 96 位清零，算术与控制/perf 寄存器分组不变；父级 DIV 输出不能被误当作高位常零。

FU 生成 5 份文件完全相同。父级 58 份生成文件仅 ExeUnit.sv 和 LogPerfEndpoint.sv 不同；后者只改变打印顺序，并被实际 SYNTHESIS 定义排除。

3.14.4 STA 及敏感性

同一映射的五档条件为 1ns/5fF、0.5ns/5fF、2ns/5fF、1ns/1fF、1ns/20fF。理想时钟、0.05ns uncertainty 与 transition、0.1ns 输入/输出 delay、0.05ns 输入 transition、5K_hvratio_1_1 线负载。parent testEnable=0，不额外排除 ready/reset/flush 路径。

每版 70 条类别/档位路径对照，父级网表规范化经逐 bit 拓扑绑定；未知单元和动态未约束端点拒收。以下 nominal 到达时间包含对应外部输入延迟；同类最差端点可以变化，不能解释为同一物理路径。

表 3-11 STA 及敏感性

边界/路径	修复前 arrival(ns)	修复后 arrival(ns)	差值 (ps)	修复后 slack(ns)
fu/i2o	0.636728	0.636728	+0.000	0.213272
fu/i2r	1.190323	1.190323	+0.000	-0.268520
fu/r2o	0.611317	0.611317	+0.000	0.238683
fu/r2r	1.163404	1.163404	+0.000	-0.240117
fu/ready	0.253743	0.253743	+0.000	0.596257
parent/gate_enable	1.855397	1.888362	+32.965	-1.010883
parent/i2o	3.128214	3.164359	+36.145	-2.314359
parent/i2r	3.194971	3.314103	+119.132	-2.393998
parent/r2o	3.185769	3.187058	+1.289	-2.337058
parent/r2r	3.166914	3.261218	+94.304	-2.341112
parent/ready	0.295424	0.247821	-47.603	0.602179
parent/vdot_i2r	1.511395	1.474949	-36.446	-0.549064
parent/vdot_r2o	2.832115	2.776701	-55.414	-1.926701
parent/vdot_r2r	1.345889	1.356106	+10.217	-0.433760

70 条类别/档位比较中，到达时间变大 30 条、变小 15 条、相同 25 条。全部差值和

端点见完整时序 CSV。

nominal 所报路径中，修复前有 10 类负 slack，修复后有 10 类。流程审计 PASS 不等于 1ns 收敛；本轮没有获得整核 Fmax、布线后时序或功耗。

最差端点回溯已保存 path-endpoint-review.json：前后 r2r 均是 VFDiv 的 fsm_q[0] 到 quo_m1_iter_q_f64_0[1]，i2r 最差起点从 vsew 输入变为 src 输入；gate_enable 最差起点从 vfdivs_1 切到 vfdivs_0，终点仍是浮点 DIV 的 ICG。不能因其属于 DIV 就忽略父级时序退化，也不能把最差路径切换解释为同一固定路径的变化。

3.14.5 结论与限制

本次修复解决匹配 DIV 取消后的无效关钟保持；前轮严格局部测试已证明 48 取消窗口/144 个后续采样拍无多余钟脉冲。新全核回归与本轮周期检查进一步支持保留其行为正确性和固定微基准性能。

这是门控正确性修复，不是新的吞吐提速。面积/时序取舍必须按父级完整映射结果判断，不能将消除时钟脉冲直接换算成功耗百分比。

建议保留这项修复作为工程候选：周期未退化、空闲关钟缺陷已消除、父级面积小幅下降；但它不是时序全面改善版。冻结版本前应由队友复核 nominal 父级 r2r 约 +94ps、i2r 约 +119ps 及门控使能约 +33ps 的取舍。若下一项专门做时序优化，应另立候选和同约束对照，不改写本轮结果。

当前本地源码仍为 0f12df6；未自动切换团队默认版本、未推送、未代签人工 ACK；本轮不转授 GEMV、LLM、T17 或其他硬件版本的资格。

3.14.6 复查、失败记录与交付

新增主机检查共 62 项：周期配对 30、验收身份 6、沿用成本审计 20、归档拒错 6，均通过。保留首次预检的验收 JSON 字段适配失败，以及生成范围检查首次遗漏日志模块的拒绝；二者均发生在检查工具/范围审查，不是 EMU 或综合功能失败，未删除原记录。

原始周期根：/home/riscv/riscv-lab/o2-r3-div-gate-performance-20261025。

原始综合根：/home/riscv/riscv-lab/o2-r3-div-gate-cost-20261025。

原 H1 完整 build：/home/riscv/riscv-lab/o2-h1-build-before-r3-div-gate-20261024。

原始 stdout/stderr、真实退出码、命令、源/依赖/镜像/工具/库哈希及所有路径报告随归档保存；EMU/NEMU 和 EDA 可执行文件保留在本地，用 SHA 绑定，不放进日常 Git 提交。

只读复算入口为 performance/recheck.py 和 cost/compare.py；必须传新的输出路径。归档字节核验使用 package_evaluation.py –verify ARCHIVE –receipt NEW_JSON；字节检查不能冒充重新仿真或重新 STA。

下一步是依据本轮成本结果完成工程取舍与队友复核，用户授权后再发布 zjhwork。本轮没有自动开展新的 RTL 优化。

3.15 C8 调用 ABI、对象及回退合同

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/optimizations/o2/integration/firstdot-v2-0f/package/c8/CONTRACT.md

3.15.1 入口与选择

表 3-12 入口与选择

入口	目标
xs_c8_gemv_8x32，开关缺省或 0	config_reuse_gemv_8x32_reuse_vdot
xs_c8_gemv_8x32，XS_C8_GEMV_FORCE_RVV=1	config_reuse_gemv_8x32_reuse_rvv
xs_c8_gemv_8x32_rvv	始终 config_reuse_gemv_8x32_reuse_rvv

XS_C8_GEMV_FORCE_RVV 最终展开只能是字面量 0 或 1；空值、表达式和未知标识不受支持，编译时拒绝。独立 guard 为 XS_C8_SELECTED_GEMV_H，API 版本 XS_C8_GEMV_API_VERSION=1。XS_C8_GEMV_PROFILE 是静态合同标签，不能检测 CPU 能力或不同翻译单元的旧对象。改变宏或版本后，完整重编所有相关翻译单元。

旧 XS_GEMV_FORCE_RVV 和 xs_gemv_* 不参与本包选择。包内不重新定义旧 guard、版本、别名。应用按所选版本使用匹配的旧库；C8 库不能代替 firstdot 库。

3.15.2 普通 C ABI

函数原型为 void entry(const int8_t *packed_ax, void *output_slots)。已验证运行环境为当前香山昆明湖 V2/Nexus-AM 裸机 RV64、lp64d，**VLEN 恰好 128（vlenb=16）**，VS 已启用。默认路径要求该硬件实现项目的自定义 vdot 编码。编译器支持 RVV、misa.V 或本包标签均不足以证明自定义指令存在。无 Linux 用户态运行资格。

- 输入：有符号 INT8 行优先 A[8][32]，紧接 x[32]，共 288 字节。

- 输出：8 个 16 字节槽，共 128 字节。每槽低 32 位为 INT32 点积结果，高 96 位清零。不得改为紧密 int32_t[8]。
- 两块缓冲区各自 16 字节对齐、有效且不重叠；输入不会被修改。调用者负责生命周期、数据打包及平台初始化。
- 沿用原 RV64 C ABI：栈及被调用者保存寄存器约束不变；向量寄存器和允许的调用者保存状态会改变。
- 返回状态为 vl=8、vtype=0（e8/m1/tu/mu）、vstart=0、vcsr=7，不保留原向量配置。

底层 vs1r.v 存整个向量寄存器，而输出行步长固定为 16。VLEN 256 会越过槽边界；不能把该包当作 VLEN≥128 通用实现。强制 RVV 只改变调用目标，完整库仍包含未使用的 vdot 函数，也不授予任意 RVV 平台运行资格。

3.15.3 构建与资格边界

独立 GNU 汇编参数固定为 -march=rv64gcv -mabi=lp64d -mno-relax -c。完整 gemv.o 为 27,160 字节，SHA256 为 f8bc13daeead98a414f313600a5778c96ac95a66b5bcb5c714ea3b98e0d25ab3。确定性 libxs_gemv_c8.a 只含一个 gemv.o，保留 16 个函数、原节和重定位；config_reuse_gemv.h 从合格 gemv.h 原字节复制，仅改产物文件名。

当前资格链绑定软件源基点 c823a8786e052c6e26f6915563c42b6d730dd8e3、选中 RTL 0f12df675378556670afd0dd2153f47fcb330610 和 clean-02 配套身份。完整输入、工具身份及验收结果以 INPUTS.json、验证器输出和版本数据档案记录为准。构建成功本身不代表资格链通过。

静态验收覆盖完整对象、单库成员、缺省/0/1 两别名的 GNU 调用对象和可重定位链接等价、实际符号解析、错误配置拒绝及 SDK 搬移。它不产生新的最终 ELF、目标运行、周期、面积或 STA 数据；example.c 仅为编译/链接示例。只有上述完整等价及身份检查通过，才可明确复用原矩阵和 C8 普通调用证据，不能改写旧记录或当成新应用验证。

新增 wrapper、分派、CSR 保存、裁剪/混合对象、编译参数或最终布局变化，以及新实际应用，需要按影响补独立目标运行和必要的公平性能比较。叶函数字节相等不能单独授予新最终映像的功能或周期资格。

3.15.4 回退

本包内设 XS_C8_GEMV_FORCE_RVV=1 或显式调用 xs_c8_gemv_8x32_rvv 后完整重编。撤销 C8 选用时，恢复原 firstdot 头和匹配库并完整重编。其余尺寸未提供 C8 稳定别名；不要将本包的 8×32 结论推广到其他形状。

3.16 C8 最终链接和镜像等价验证

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/optimizations/o2/integration/firstdot-v2-0f/package/c8/relink/RESULT.md

3.16.1 接入内容

- 原主程序只把底层头的包含改为 selected_gemv_c8.h。
- 8×32 reuse vdot/RVV × 三数据集，共 6 个函数表项改用 xs_c8_gemv_8x32 / xs_c8_gemv_8x32_rvv。其余 42 项、数据、期望值、顺序和校验不变，没有重新生成测试向量。
- 真正从 libxs_gemv_c8.a(gemv.o) 链入完整合格对象；SDK 程序源目录没有 gemv.S，对象目录没有绕过库的 gemv.o。GNU 依赖文件和 link map 确认实际包含 SDK 头、实际抽取库成员。
- 在全新输出中分别构建原源码复制对照和 SDK 接入程序，保留原 AM/klib 和链接次序；只有输出路径和记录链接来源的选项不同。

源码见 source，重建和完整核验见 README。

3.16.2 三方等价结果

比较对象：原已通过全核的 C8 镜像 → 本轮原源码复制对照 → 本轮 SDK 接入版。

表 3-13 三方等价结果

项目	结果
完整 BIN	三者全字节一致，均 31,440 字节
BIN SHA256	56805506eb98b68269c73a3645d8f75dff73b697e30dec5f93ccc4716089ad34
入口	均 0x80000000；ABI flags 及 ISA 属性一致
PT_LOAD	同地址、权限、对齐及完整内容；file30,024B，memory31,440B，零填充 1,416B
装载节	所有 SHF_ALLOC 节的位置、大小、权限、对齐及内容一致，包括 BSS

表 3-13 三方等价结果（续）

项目	结果
函数	16 个公共函数及启动/主程序等 5 个函数的地址、大小和机器码一致
测试表与数据	48 条 64 字节调用记录与原计划逐项匹配；12 组输入数组地址及内容一致
GNU 链接	无未解析符号；map 与实际子命令证明使用 SDK 库成员
完整 ELF 原字节	不同；差异仅在.debug_info、.debug_line、.debug_line_str
独立 GNU 复核	分别 objcopy –strip-debug 后，三份完整 ELF 全字节一致
去调试 ELF SHA256	7e26e295fb020a63334026f768ed4825372266465b4e6008973de966fefc04ab

调试节保存源文件/头文件路径及行号等信息，构建目录与头文件发生变化会改变这些记录。保留原 ELF，没有通过删除调试信息来掩盖装载差异：主检查器已先完整比较装载内容和计划，去调试检查是独立补充证据。

原 ELF SHA256：02159c7ef78cb31eb24cf706bf2974eadea313710563f235f2b11a7bb922de56。

复制对照 ELF SHA256：cfad14278ee4a3590de9bcbe7589b808fd91959156cef93ba92a78a90db0d99b。

SDK 接入 ELF SHA256：f0ddf1167c961101e70f58cfd012ae788d2965324ad057aa4ade5af1de5bc42e。

3.16.3 检查器拒错与输入保护

7 项负控全部被拒绝：实际 ELF 代码字节改坏、调用表改坏、启动入口改变、BSS 大小改变、ISA 属性改变、输入计划改变，以及真实 GNU 链接错误空库。另有只改调试字节的正控，确认不会将纯调试变化判为行为改变。错误镜像只用于主机检查，没有送入全核。

627 项完整输入与封存运行证据绑定当前 0f、EMU/NEMU 及 GNU 工具。原 33 个矩阵进程和 3 个 C 进程的审计关系及全部封存输出树保持；本轮没有重复解析相同原始日志或伪造新 PASS。当前操作保存了真实新 GNU 构建和最终镜像等价证据。

成功流水线：2026-10-27T21:59:40.771873+08:00 至 2026-10-27T21:59:43.182911+08:00，exit0。两次 GNU 构建，8 份主流水线命令回执，另 3 份去调试 GNU 检查回执。流程首次通过，负控和原始文件全部保留。

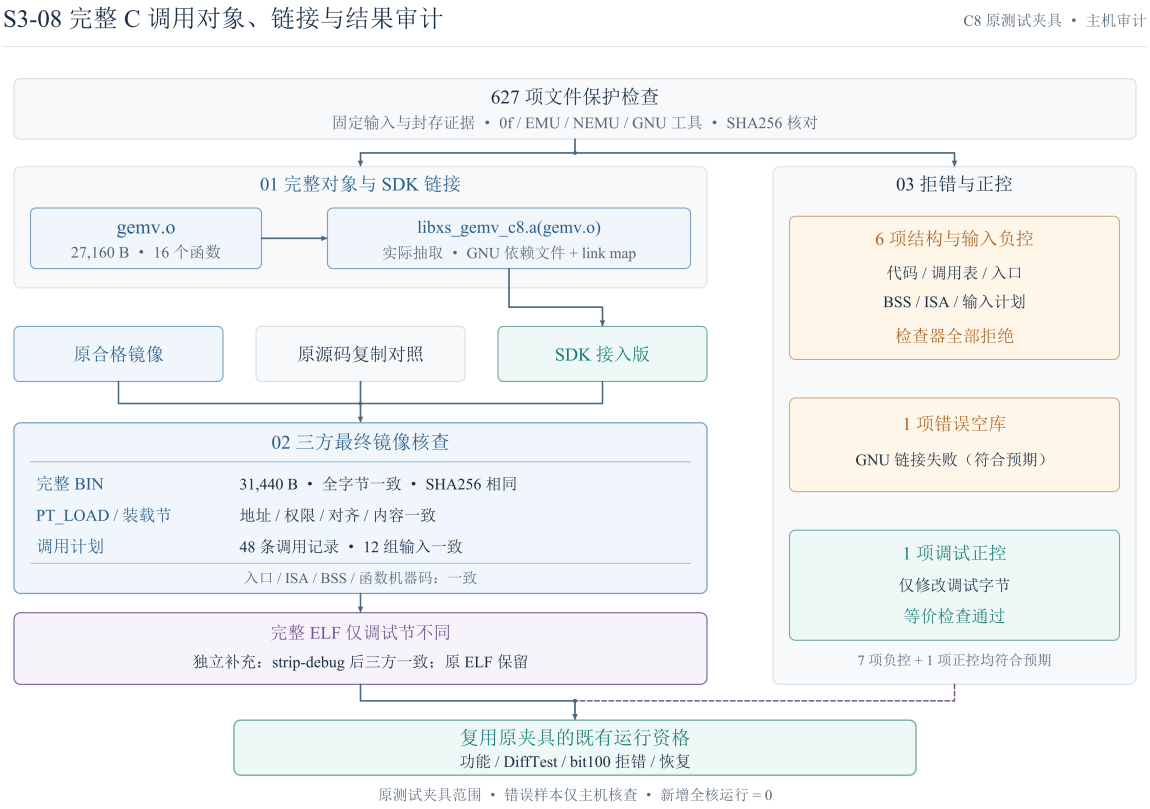


图 3-8　完整 C 调用对象、链接与结果审计流程

注：审计流程示意图归纳 627 项保护检查，以及三方 BIN、PT_LOAD、48 条调用记录和 12 组输入的一致性；完整 ELF 可因调试节不同而异，静态等价支持复用既有功能结果。

来源与范围：用户提供示意图；依据：原 C8 relink RESULT 及三方等价结构化记录。版本范围：静态等价支持复用既有功能结果；不是新增全核运行。

3.16.4　已完成与未新增的结论

当前 C8 包已从“对象/可重定位链接合格”推进到“接回原可运行 C 程序、完整镜像等价”。原 16 函数、48 例/两轮 96 正常调用（8×32 占 12 例/24 调用）的功能资格可明确复用；不是本轮重新执行这些调用。

新增 EMU/NEMU/全核运行均为 0。当前验证的是原测试夹具的包化接入，不是新的实际应用或 LLM 推理。没有新的周期、面积、STA 或功耗测量。原 273 cycles 与 1.095238× 属于第 25 条性能矩阵，不是本 C 程序新测数据。

最小软件包接入闭环已完成。后续选择具体应用后再固定新的输入/调用路径/最终镜像合同；需要新性能结论时使用同路径 firstdot/RVV 及必要的集成前对照。另可继续 4×8、16×64 控制差异只读归因；4×13 保留 RVV。生产 RTL 和旧默认均不因本次接入改变。

3.17 R5 组合验收和证据复用

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/optimizations/o2/integration/firstdot-v2-0f/acceptance/RESULT.md

3.17.1 clean-02 墙钟问题的处理

原 14 项全核的 UTC 容量检查出现 80.119ms 表观四路重叠，原 FAIL/WALL_CLOCK_LIMITED 记录永久保留，未修改时间戳或门槛。按事前合同仅重新执行 o2-control-diagnostic，其余 13 项全部从原始日志重新解析。

最终组合 14 项通过功能、NEMU 差分、O2 覆盖和负控后同镜像恢复。**13 项复用＋1 项新执行，不是重跑 14 项**。选定记录的严格 UTC 并发峰值为 3（上限 3）；新单进程以同 boot/PID 的单调时钟包络验证，峰值 1，包络 1497.000 秒。该耗时是宿主审计数据，不用于 DUT 加速比。

新控制诊断检查 131072 字节寄存器结果，退出 0；独立离线审计再次通过。原 13 项没有单调时钟记录，不能追溯宣称原批次的实际 OS 并发已测量。

本次 UTC 间隔与单调包络仍相差-62.888356 秒（UTC 更短）。因此墙钟稳定性仍未证实；新占用审计依赖同 boot 的单调包络，而非容差修正 UTC。该现象如实留档，不把旧批次或系统时钟标为已修复。

3.17.2 验收矩阵

表 3-14 验收矩阵

项目	状态	依据及范围
identity	PASS	0f 生产源码 398 项与 clean 副本、选定 Git 版本一致；完整工作树冻结 24157 项；冻结不等于提交；HEAD 不能代表未提交软件材料
stage1	PASS_REUSED	当前 hello/vadd 原文重验；原阶段一 12 文件 SHA 及 220295598B 波形定位；波形为 2026-10-09 vadd；团队标识仍是占位符，正式材料需成员确定

表 3-14　验收矩阵（续）

项目	状态	依据及范围
spec	IMPLEMENTED_INTERNAL_CONTRACT	SEW8/VLEN128，8 对 signed int8 低 32 结果、高 96 清零；编码与异常合同有设计说明；Q01–Q07 待官方澄清；不声称官方确认全部内部语义
model	PASS_NEW_HOST_CHECK	原模型及确定性向量一致性重新检查；22 定向 +10000 随机未变；主机模型不代替 RTL
fu-parent	PASS_REUSED_RAW_REPARSED	0f 真实 FU 与 VFEX4 父级；完整 128 位、反压/取消/身份/异常、严格 DIV 关钟及 4+16 负控；复用 0f 原 167 载荷；未新执行 RTL 仿真，不转授 C1/C2/C3
fullcore	PASS_COMPOSITE_13_REUSED_PLUS_1_NEW	固定原 13 项重新解析 + 新 o2-control-diagnostic；117 不同/124 次功能例，O2 覆盖及负控恢复；组合验收不是 fresh14；原 14 项 UTC 容量 FAIL 保留；本次 UTC 间隔比单调时钟短 62.888356 秒，系统墙钟稳定性未证实，新占用审计采用单调包络
microbench	PASS_REUSED	0f 相对修复前 96 对正式窗完全一致；独立 batch128 为 221 vs RVV678 cycles，3.067873×；历史 0f 构建的固定微基准；不是当前 GEMV 或 LLM 端到端加速
firstdot	PASS_REUSED_INPUTS_REVALIDATED	干净 firstdot 四尺寸：768 窗口/96 普通 C 输出复现一致；前三选 vdot、4×13 选 RVV；33 矩阵 +3 普通 C 为既有运行，本轮没有新性能

表 3-14 验收矩阵（续）

项目	状态	依据及范围
c8	PASS_REUSED_FINAL_IMAGE_RECHECKED	627 项输入闭合、C8 包与旧合格普通 C 最终 BIN/PT_LOAD/入口/48 表项/12 数据组一致；273 vs 最快 RVV299 cycles/1.095238× 来自既有性能矩阵，不是本次新计时
parameter	PASS_REUSED_IN_ORIGINAL_SOURCE_ENVIRONMENT	独立 s16x8/r64 扩展归档复算；独立实验算术和包装；生产指令及 NEMU 仍 INT8，未新增全核 INT16
cost	EVIDENCE_PASS_TIMING_NOT_CLOSED	0f 同约束面积/五档 STA 资料 1008 文件回读；FU17298.246、父级 284614.946 微米平方；不是新综合；1ns 负 slack 保留，整核 Fmax/布线后时序/功耗 NOT_TESTED
reproduction	PASS_REUSED_SAME_HOST	独立源/依赖、EMU/NEMU/AM/klib 及 25 应用实际重建已完成，身份再次核验；同机 Agent 复现，工具与下载缓存复用；不是另一成员/另一机器复现
entry-rollback	PASS_EXPLICIT_ENTRY	C8 独立静态库入口及 RVV 回退、firstdot 原入口与 O1/B0 历史对照保留；不新增混合 dispatcher、不变公共默认；改链接/数据/工具链需按影响再验

3.17.3 当前可使用的性能结论

C8 8×32 完整调用窗口为 273 cycles，最快同场 RVV 为 299，**1.095238×**；较 firstdot 282.25 少 3.2772%。这些来自既有 4 尺寸 ×3 数据集矩阵，本次只是验收和身份复核，没有新增跑分。四路 batch128 的 3.067873× 来自另一固定微基准合同，不代表应用端到端收益，不能与 1.095238× 混为一个总分。

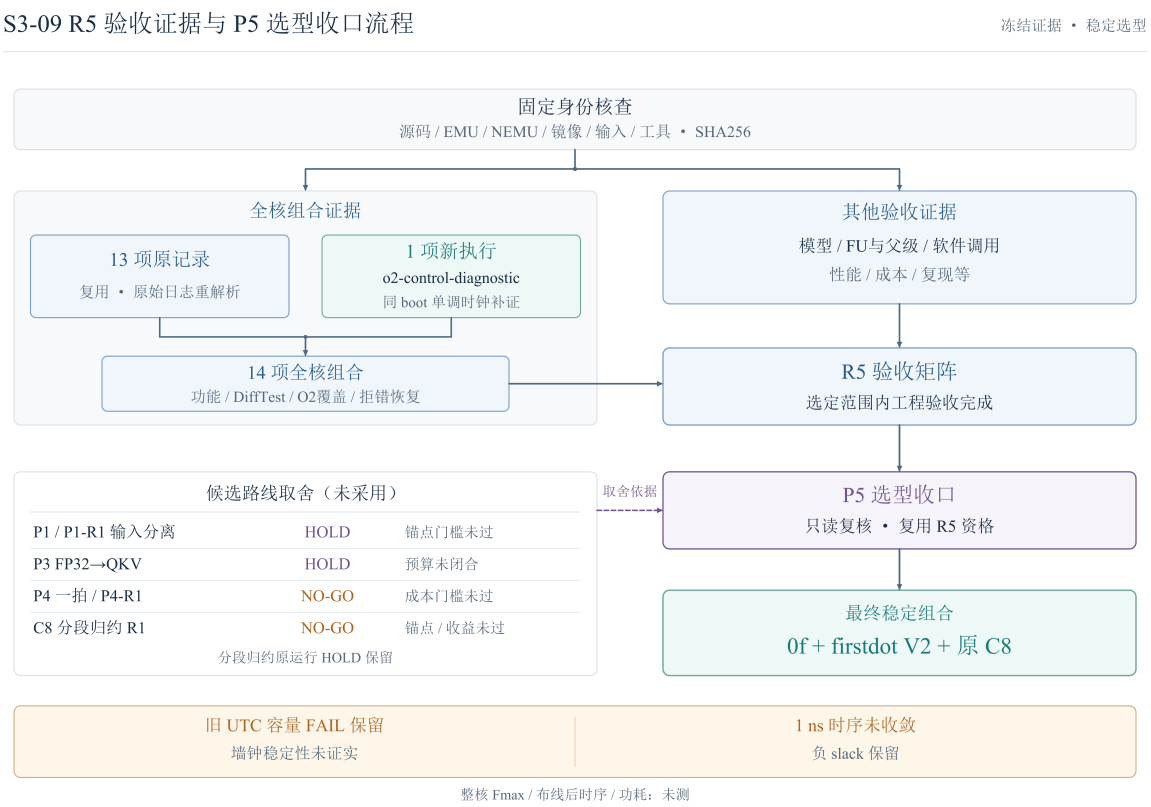


图 3-9　R5 验收证据与 P5 选型收口流程

注：收口流程区分 13 项既有证据复用和 1 项补充执行；最终组合为 0f 加 firstdot V2 加原 C8，P1/P3 维持 HOLD，P4 及分段归约未采用，原 FAIL 与 1ns 负裕量边界保留。

来源与范围：用户提供示意图；依据：R5 验收矩阵、P5 RESULT 和现有版本数据档案。版本范围：证据复用与新执行分开；不暗示未采用候选进入稳定代码。

3.17.4　已知边界与下一步

1ns STA 尚未收敛，功耗、整核 Fmax 和布线后时序未测；保留真实成本取舍。独立 INT16 参数扩展不等于生产全核 INT16 支持。4×8/16×64 配置复用控制漂移仍未归因，未纳入新采用项；4×13 保留 RVV。规格 Q01–Q07 待官方澄清，团队名称仍待确定；正式报告/PPT/视频和成员复核由团队另行安排。

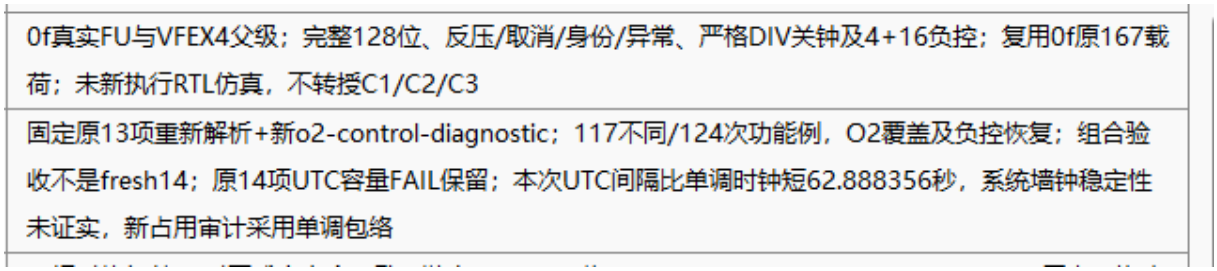
工程可以以此冻结候选为保底版本，不再为了收尾新增应用或强行优化。后续可由用户安排发布 zjhwork、成员复核与比赛材料，或另选一个有明确收益假设的优化。未授权的发布本轮不执行。

冻结文件 24157 项，生产源码 398 项与 0f 及 clean 副本一致。快照在 /home/riscv/riscv-lab/software-import-20261026/r5-g5-acceptance-01/frozen-workspace.tar.gz，已逐项回读，含工作树文件但不含 Git 历史和外部依赖。使用/回退见 README，逐项身份见数据记录，原始新增证据见 manifest。各失败尝试也已封存，没有覆盖旧 28 条数据档案。

3.18 本轮已提供截图补充

本节仅收录已有截图，原始波形、框架图和其他缺图占位仍保留。各截图按照历史版本和证据层级解释。

(a) FU/parent and full-core scope: evidence column of R5 matrix



(b) Cost scope: evidence column of R5 matrix

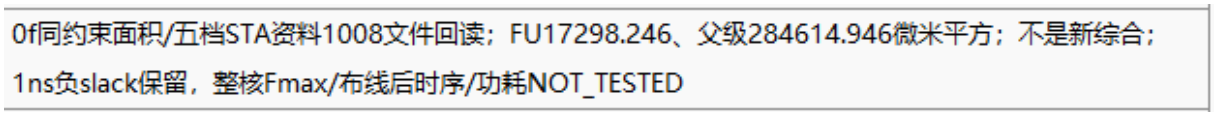


图 3-10 历史验收与成本摘要画面

本图为历史报告摘要截图。截取验收矩阵的依据及范围列；局部/全核证据复用口径、1ns 时序未收敛分别保留。不替代原 bit100 拒错、STA 路径截图或性能图。